СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

ГЛАВА 1. СОВРЕМЕННОЕ СОСТОЯНИЕ И ПЕРСПЕКТИВЫ РАЗВИТИЯ ЦИФРОВЫХ ДВОЙНИКОВ И КИБЕРФИЗИЧЕСКИХ СРЕД 5

1.1 Киберфизические системы и среды 7

1.2 Технология цифровых двойников 14

1.3 Перспективы применения цифровых двойников в рамках корпоративных киберфизических сред 22

ГЛАВА 2. РАЗРАБОТКА И ИССЛЕДОВАНИЕ МОДЕЛИ И ТЕХНОЛОГИИ ЦИФРОВЫХ ДВОЙНИКОВ ДЛЯ КОРПОРАТИВНЫХ КИБЕРФИЗИЧЕСКИХ СРЕД 31

2.1 Методы и технологии создания цифровых двойников 32

2.2 Структура цифрового двойника 39

2.3 Модель интеграции цифрового двойника в киберфизическую среду 49

ГЛАВА 3. РАЗРАБОТКА АРХИТЕКТУРЫ ПРОГРАММНО-АППАРАТНОГО КОМПЛЕКСА ЦИФРОВЫХ ДВОЙНИКОВ 56

3.1 Аппаратно-программный комплекс цифровых двойников 56

3.2 Модель предметной области на основе онтологий и семантических сетей 69

3.3 Структура клиентского программного обеспечения 74

ГЛАВА 4. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ЦИФРОВОГО ДВОЙНИКА 78

4.1 Технология программной реализации цифрового двойника 80

4.2 Вычислительный эксперимент по оценке производительности цифрового двойника 93

4.3 Реализация интерфейсов взаимодействия цифрового двойника с киберфизической средой 101

ЗАКЛЮЧЕНИЕ 108

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 109

ПРИЛОЖЕНИЕ А 116
ПРИЛОЖЕНИЕ Б..... 118
ПРИЛОЖЕНИЕ В 120

ВВЕДЕНИЕ

Киберфизические системы представляют собой интегрированные системы, объединяющие физические объекты (датчики, оборудование) с цифровыми алгоритмами для двустороннего взаимодействия: данные от физических компонентов обрабатываются в реальном времени, а цифровые решения корректируют их работу. Цифровые двойники в свою очередь являются виртуальными копиями физических систем, синхронизированные с реальными объектами через IoT и AI. Они обеспечивают мониторинг состояния, прогнозирование сценариев и автоматическую оптимизацию процессов. В контексте управления учебными аудиториями цифрового университета двойники становятся ключевым инструментом:

— планирование ресурсов. Анализ расписания, загрузки аудиторий и потоков студентов для автоматического перераспределения помещений.

— техническое обслуживание. Прогноз износа оборудования и планирование ремонтов без прерывания занятий.

— аналитика. Визуализация данных об использовании пространств для принятия управленческих решений.

Цифровые двойники трансформируют управление аудиториями в прогнозно-адаптивный процесс, создавая интеллектуальную образовательную среду.

Цель работы заключается в совершенствование технологий формирования киберфизической среды цифрового университета.

Для достижения поставленной цели необходимо решить следующие задачи:

— исследование технологии цифровых двойников, используемых для формирования киберфизической среды цифрового университета;

— разработка концептуальной модели цифрового двойника учебной аудитории, интегрируемого в единую киберфизическую среду университета;

— разработка программно-аппаратного комплекса макета цифрового двойника учебной аудитории.

Используемые методы: методология многоаспектного моделирования, метод создания комплексных моделей, метод создания параметрических моделей, методология объектно-ориентированного программирования.

Применяемые технологии: В основе разработки цифрового двойника учебных аудиторий используется язык C/C++ для программирования микроконтроллеров ESP32, которые собирают данные с датчиков (температура, освещение, CO_2) и управляют устройствами. Логика цифрового двойника реализуется на C# в среде Unity, обеспечивая 3D-визуализацию помещений и интеграцию с реальными данными. Для связи компонентов системы применяется REST API (на базе .Net Framework), который передает данные между ESP32, базой данных MySQL и клиентскими приложениями. Трекинг объектов в аудиторий выполняют Bluetooth-маячки (BLE), интегрированные с ESP32. Интерактивный интерфейс создается в Unity 3D с отображением параметров среды, состояния оборудования и потоков людей в реальном времени.

ГЛАВА 1. СОВРЕМЕННОЕ СОСТОЯНИЕ И ПЕРСПЕКТИВЫ РАЗВИТИЯ ЦИФРОВЫХ ДВОЙНИКОВ И КИБЕРФИЗИЧЕСКИХ СРЕД

Киберфизические системы и связанные с ними технологии признаны универсальными инструментами, обладающими значительным потенциалом для внедрения инновационных решений, способных оказывать трансформирующее воздействие на ключевые сектора мировой экономики, включая промышленность, транспорт, энергетику и здравоохранение [1].

Особенностью развития современных технологий является их переход к самоуправляемости благодаря использованию интеллектуальных программно-аппаратных структур. Это приводит к постепенной передаче управленческих функций от человека к техническим системам, что знаменует глобальные изменения — формирование техносферы с элементами искусственного интеллекта, становящейся движущей силой эволюции.

Понятие киберфизических систем (КФС) носит всеобъемлющий характер: при определённых условиях большинство технологических явлений современности может быть отнесено к этой категории, что позволяет включать в неё широкий спектр устройств и концепций.

К числу ключевых составляющих КФС относятся:

— робототехнические системы, включая интеллектуальных роботов;

— интернет вещей (в том числе его промышленная версия);

— высокоавтоматизированные транспортные средства и системы управления транспортом;

— беспилотные летательные аппараты;

— концепция «умного города»;

— интеллектуальные медицинские устройства.

Цифровизация современных технологических систем подразумевает поэтапное внедрение информационных компонентов с интеллектуальными функциями в технические объекты и процессы. Эта тенденция проявляется

повсеместно — от детских игрушек и бытовых приборов с микрочипами до сложных технологических комплексов в нефтехимической отрасли.

Ещё одним аспектом КФС можно выделить его интеграцию в Индустрию 4.0 наряду с использованием CALS-технологий. КФС интегрируют физические объекты с цифровыми алгоритмами и IoT, обеспечивая автономность производственных процессов, а CALS-системы (Continuous Acquisition and Lifecycle Support) управляют данными на всех этапах жизненного цикла продукции — от проектирования до утилизации. Вместе они создают сквозную цифровую экосистему: КФС в режиме реального времени собирают данные с оборудования (температура, износ, параметры работы), а CALS структурирует эту информацию, связывая её с конструкторской документацией, производственными регламентами и логистическими цепочками. Например, изменения в чертежах автоматически обновляют цифровые двойники станков, а данные о качестве сырья из CALS используются КФС для корректировки параметров обработки. Это позволяет сократить время перенастройки линий под новые продукты, прогнозировать поломки оборудования и отслеживать соответствие процессов нормативным требованиям. Интеграция КФС и CALS обеспечивает сквозную аналитику: от влияния производственных параметров на срок службы изделий до оптимизации логистики на основе данных IoT-датчиков. Ключевые преимущества — снижение затрат за счёт превентивного обслуживания, повышение гибкости производства и минимизация ошибок благодаря синхронизации данных между инженерами, цехами и поставщиками [2].

1.1 Киберфизические системы и среды

Ключевой особенностью киберфизических систем являеться способность к самооптимизации и прогностическому управлению на основе анализа данных в реальном времени без прямого вмешательства человека. Архитектура КФСред строится на многоуровневой структуре, включающей физический, сетевой, вычислительный и когнитивный уровни. Физический уровень охватывает IoT-устройства, роботов и промышленные машины, такие как конвейерные роботы на «умной» фабрике или датчики вибрации станков. Сетевой уровень обеспечивает связь через технологии промышленного интернета вещей (IIoT) и 5G/6G-сети с микросекундной синхронизацией данных. Вычислительный уровень сочетает периферийные вычисления для локальной обработки информации и облачные платформы для сложного моделирования, например, прогнозирования износа оборудования на нефтяных вышках. Когнитивный уровень представлен ИИ-алгоритмами, такими как нейросети, которые обучаются на исторических данных и принимают решения, например, перераспределяя энергию в сетях при пиковых нагрузках.

Цикл работы КФСреды начинается со сбора данных через миллионы сенсоров, фиксирующих параметры вроде температуры или давления. Затем данные агрегируются и фильтруются на периферийных устройствах для устранения шумов. На этапе анализа ИИ идентифицирует аномалии, такие как риск поломки турбины, и строит прогнозные сценарии. Далее клапаны или двигатели, получают команды на корректировку процессов, а система оценивает результат через обратную связь, обучая модели. Примером реализации КФСреды может служить «умная больница», где хирургические роботы с тактильной обратной связью, датчики жизненных показателей пациентов и системы доставки лекарств интегрированы в единую экосистему. Цифровой двойник больницы отражает состояние всех систем — от вентиляции до расписания операций, а ИИ-алгоритмы динамически планируют процессы, например, переносят операции при перегрузке

энергосети. В случае аварийного отключения энергии система автоматически перенаправляет резервное питание в реанимацию, приостанавливая менее критичные операции.

Стимулом для создания интеллектуальной техники выступают психофизические ограничения человека. Согласно некоторым исследованиям, в современном обществе происходит трансформация традиционного понимания счастья: теперь оно ассоциируется с комфортностью жизни, обеспечиваемой цифровизацией быта. В сферу человеческой деятельности вовлекаются не только объекты, повышающие комфорт (их перечень растёт экспоненциально), но и услуги, энергетические и информационные ресурсы. Это провоцирует взрывное усложнение окружающих человека технологий.

Однако развитие сложных систем порождает проблемы их эксплуатации и обслуживания. Ошибки в проектировании или обслуживании могут приводить к авариям и техногенным катастрофам, сопровождающимся экологическим ущербом, разрушениями и человеческими жертвами.

Несмотря на это, интеллектуализация техники обеспечивает эволюционный прорыв человечества. Продолжение данной тенденции ведёт к тотальной цифровизации среды, чей потенциал может сравняться, а в перспективе — превзойти возможности человеческого интеллекта.

В киберфизических системах физические производственные процессы реализуются в материальном пространстве, а их виртуальное отображение — в компьютерной среде. КФС преобразуют производственные задачи из физического мира в цифровые двойники, что позволяет управлять материальными объектами через обратную трансформацию данных в действия. Например, операции сверления, сварки или перемещения объектов осуществляются в физической реальности, тогда как их управление — в информационном пространстве.

Сложность таких задач выходит за рамки создания традиционных автоматизированных систем с встроенными компьютерами. Современная задача заключается в гармоничной интеграции двух типов сущностей:

— Физических (механические, электрические, биологические, химические и экономические системы);

— Их виртуальных цифровых образов.

Предшественниками КФС стали беспроводные сенсорные сети, автоматизированные системы управления технологическими процессами (АСУ ТП), системы реального времени и распределённые вычисления.

Концепция КФС развивает идеи технической кибернетики, ориентированной на управляемость, устойчивость и безопасность систем, а также расширяет рамки классической физики, включая искусственно созданные системы, имитирующие принципы живой природы.

Основой киберфизических систем является интеграция вычислительных ресурсов и физических процессов для достижения эмерджентности — возникновения качественно новых свойств, отсутствующих у отдельных компонентов. Целью такой синергии становится оптимизация системных связей и создание уникальных функциональных возможностей [3].

Классификация КФС по отраслевому признаку включает применение в энергетике, нефтегазовой отрасли, промышленности и других сферах. По масштабу они варьируются от нанороботов до «умных» заводов, объединяя аппаратные компоненты и виртуальные вычислительные модули. Каждый элемент КФС может взаимодействовать с передовыми технологиями: аддитивным производством, блокчейном, виртуальной и дополненной реальностью, облачными вычислениями.

Термин «КФС» применяется как к отдельным устройствам, так и к системам, подразумевающим интеграцию вычислений в физические процессы. Технически КФС схожи с системами типа GRID (согласно ГОСТ Р 55768), задачами которых являются виртуализация и управление ресурсами в

распределённых гетерогенных средах. В КФС эти задачи решаются через промышленный интернет вещей, межмашинное взаимодействие, а также технологии распределённых вычислений (туманные и облачные). Ожидается, что развитие облачных технологий сократит задержки передачи данных и усилит потенциал интеллектуальных устройств [4].

Одновременно КФС служат инструментом решения проблем автоматизированных систем: распределённости, надёжности, безопасности и автономности работы в сложных условиях. Концепция подразумевает использование систем мониторинга для анализа производственных объектов с последующей оптимизацией их жизненного цикла.

В результате производственные активы становятся интеллектуальными и автономными. Для выполнения производственных задач в заданные сроки, с заданными объемами и удовлетворения, в частности, требований, при низких затратах и с сокращенным временем производства требуется информационная синхронизация взаимодействия активов в реальном времени во всех возможных режимах их работы. Важным достижением 2024 года стало принятие российского ГОСТ Р 71576-2024 «Системы киберфизические». Он вступил в силу с 1 января 2025 г. Концептуальная модель КФС, согласно ГОСТу, теперь представлена в следующем виде.

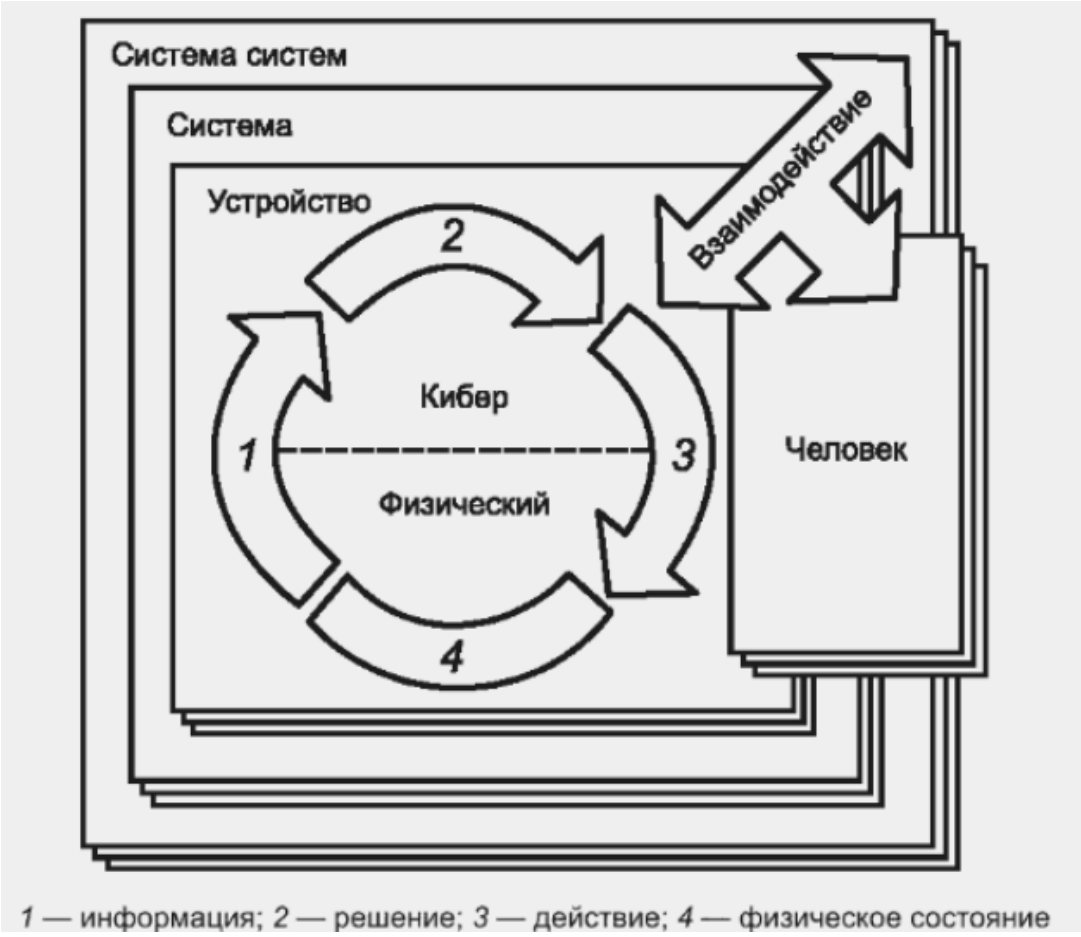

Рисунок 1 – Концептуальная модель киберфизических систем, согласно ГОСТ Р 71576-2024

В ГОСТ Р 71576-2024 особо выделено, что к системам КФС могут относиться как отдельные киберфизические устройства (например, «умная» лампочка), так и комплекты из одного или нескольких киберфизических устройств, которые в результате совместного применения образуют КФС (например, «умный дом»). Более того, системы КФС могут включать несколько подсистем КФС, каждая из которых может состоять из одного или нескольких киберфизических устройств. ГОСТ описывает также и архитектуру киберфизических устройств через пятиуровневую систему [5]:

— уровень 1: «Коммуникационная среда».

— уровень 2: «Конверсия».

— уровень 3: «Киберуровень».

— уровень 4: «Самопознание».

— уровень 5: «Самоконфигурация».

Рисунок 2 – Функции киберфизических систем (КФС) по ГОСТ Р 71576-2024

На уровне «Коммуникационная среда» устройства способны к самоподключению и самоконтролю, что является базовым слоем киберфизических систем (CPS). Этот уровень использует умные датчики и исполнительные механизмы для реализации самонастройки и контроля. Важность восходящих и нисходящих каналов связи возрастает при интеграции полевого и производственного уровней, особенно в контексте бизнес-процессов, таких как ERP, CRM и SCM.

Слой «Конверсия» CPS исключает возмущения из данных полевого уровня с помощью глубокого анализа и облачных вычислений. Эти инструменты помогают решать проблемы, например, вычислять оставшийся срок службы активов и выявлять причины незапланированных остановок оборудования. Данные от устройств и умных систем становятся основой для самоопределения критических проблем.

На уровне «Киберуровень цифровых двойников» каждая физическая сущность создает виртуальный «двойник», который контролирует состояние активов с помощью методологии «Time-Machine». Этот двойник проводит самоанализ и готовит управление физической сущностью.

Уровень «Самопознание своей сущности» предоставляет результаты самооценки в графическом формате, указывая на потенциальные проблемы. Уровень исполнительных действий преобразует интеллектуальные решения в действия на уровне активов, с возможностью переконфигурирования системы на основе приоритетов и рисков. Для анализа состояния активов используются алгоритмы и программное обеспечение.

Уровень «Самоконфигурация» переводит интеллектуальные решения в реальные действия, позволяя киберфизической системе взаимодействовать с физическим пространством.

Одним из ключевых вызовов остаётся обеспечение безопасности и надёжности. Киберугрозы, такие как атаки на медицинское оборудование, требуют внедрения аппаратного шифрования и изолированных подсетей. Отказоустойчивость достигается дублированием сенсоров — например, установкой трёх независимых датчиков давления на газопроводе. Этические и правовые аспекты включают вопросы ответственности за ошибки ИИ-диагностики и конфиденциальности биометрических данных, что решается через блокчейн-технологии и гомоморфное шифрование. Энергоэффективность, потребляющая до 20% ресурсов объекта, оптимизируется за счёт предиктивного обслуживания и «зелёных вычислений», использующих возобновляемую энергию.

Перспективы развития КФСред связаны с симбиозом искусственного интеллекта и квантовых вычислений, что ускорит анализ данных в тысячу раз для мгновенного реагирования в аварийных ситуациях. Внедрение нейроморфных чипов, имитирующих структуру нейросетей, позволит снизить энергопотребление алгоритмов.

1.2 Технология цифровых двойников

В общем понимании, цифровой двойник (Digital Twin) представляет собой сложную цифровую математическую модель, которая имеет определенную степень сходства с реальным объектом, производством, материалом или конструкцией. Чаще всего он описывается трехмерными нестационарными нелинейными уравнениями [6].

Цифровой двойник нужен, чтобы смоделировать, что будет происходить с оригиналом в тех или иных условиях. Это помогает, во-первых, сэкономить время и средства (например, если речь идет о сложном и дорогостоящем оборудовании), а во-вторых — избежать вреда для людей и окружающей среды.

Некоторые эксперты выделяют три типа двойников: цифровые двойники-прототипы (Digital Twin Prototype, DTP), цифровые двойники-экземпляры (Digital Twin Instance, DTI) и агрегированные двойники (Digital Twin Aggregate, DTA):

— прототип (DTP) — представляет собой виртуальный аналог реального объекта, который содержит все данные для производства оригинала;

— экземпляр (DTI) — содержит данные обо всех характеристиках и эксплуатации физического объекта, включая трехмерную модель, и действует параллельно с оригиналом;

— агрегированный двойник (DTA) — вычислительная система из цифровых двойников и реальных объектов, которыми можно управлять из единого центра и обмениваться данными внутри.

Погрешность между работой виртуальной модели и работой реального объекта не должна превышать 5%. Возьмем, например, двигатель — при воздействиях на него в цифровой среде поведение модели должно отклоняться от поведения реального объекта не более чем на 5% [7].

Описание программной среды, способной размещать приложения цифрового двойника, может быть реализовано путем определения двух различных классов агентов, а именно:

— агенты процесса: сущности, которые взаимодействуют с данными процесса физических объектов или процессов в общем, и переводят эти данные в промежуточную форму, интерпретируемую другими агентами, и из неё;

— агенты симуляции: объекты, которые взаимодействуют с модулями симуляции и переводят свои данные в промежуточную форму и обратно.

Агенты в технологии цифровых двойников могут быть реализованы в виде программных алгоритмов, скриптов, искусственных интеллектов или других автоматизированных сущностей. Они служат для обеспечения актуальности и точности цифрового двойника, делая его отражением реальных изменений и событий в реальном мире.

Это понятие также может включать в себя идею "агентного моделирования", где агенты в цифровом двойнике моделируют поведение реальных объектов или систем, чтобы предоставить более точное представление их функционирования [8].

В контексте технологии цифровых двойников, термин "агент" обычно означает программное средство или компонент, которое действует от имени или в интересах цифрового двойника. Этот агент может выполнять различные функции, такие как сбор данных из реального мира, обновление информации в цифровом двойнике, анализ данных, принятие решений или взаимодействие с другими системами.

Поскольку современные системы проектирования и производства уже в значительной степени оцифрованы, каждая уникальная реализация продукта в физическом мире оставляет за собой след в киберсфере; цифровую тень, которая включает все созданные во время реализации данные об операциях, условиях, процессах и т.д. Цифровая тень отдельного экземпляра продукта

затем может быть связана с цифровой мастер-моделью — еще одним цифровым артефактом, состоящим из данных и функций, общих для всех экземпляров данного класса продукта. Таким образом, рождается новый цифровой двойник.

Оснащенный моделями, которые могут симулировать различные подсистемы, созданный цифровой двойник теперь должен иметь возможность «испытать» каждое возможное событие, с которым его физический аналог может столкнуться в течение своей жизни. Сюда могут входить рабочие условия, которые изначально не рассматривались во время создания первоначального дизайна продукта. При этом набор симуляций, составляющий вычислительную часть цифрового двойника, постоянно получает данные датчиков, полученные от физического устройства. В контексте промышленных систем, склонных к отказам компонентов, это может позволить разработать приложения профилактического обслуживания, прогнозирования срока службы компонентов и даже динамическую активацию различных механизмов самовосстановления.

Как было упомянуто ранее, цифровой двойник обладает исключительными свойствами: быть чрезвычайно точным и всегда актуальным. Наиболее важно то, что представление о цифровом двойнике, как об эквиваленте некоторого физического объекта, выделено в обоих определениях. Программное обеспечение, в отличие от физических машин, может быть спроектировано таким образом, чтобы пользователи могли свободно наблюдать за внутренним состоянием выполнения в любой момент времени, не влияя на сам процесс.

Концепцию цифрового двойника можно рассматривать как расширение временного окна симуляции до всего жизненного цикла продукта. В то время как традиционно моделирование рассматривалось в качестве инструмента в основном для проектных операций, парадигма цифрового двойника позволяет использовать возможности симуляции на этапах поставки, эксплуатации и даже вывода из эксплуатации.

На рисунке 3 представлена обобщенная схема применения модели в виде цифрового двойника в системе управления. В быстродействующую цифровую модель, отражающую ситуацию объекта моделирования, вводятся исходные управляющие данные. Полученные результаты моделирования анализируются, изменяются и вновь поступают на вход модели. Таким образом, осуществляется активное прогнозирование и получение оптимального решения, которое через средства сопряжения формирует управляющие воздействия, поступающие на объект управления с цифрового двойника в системе управления [9].

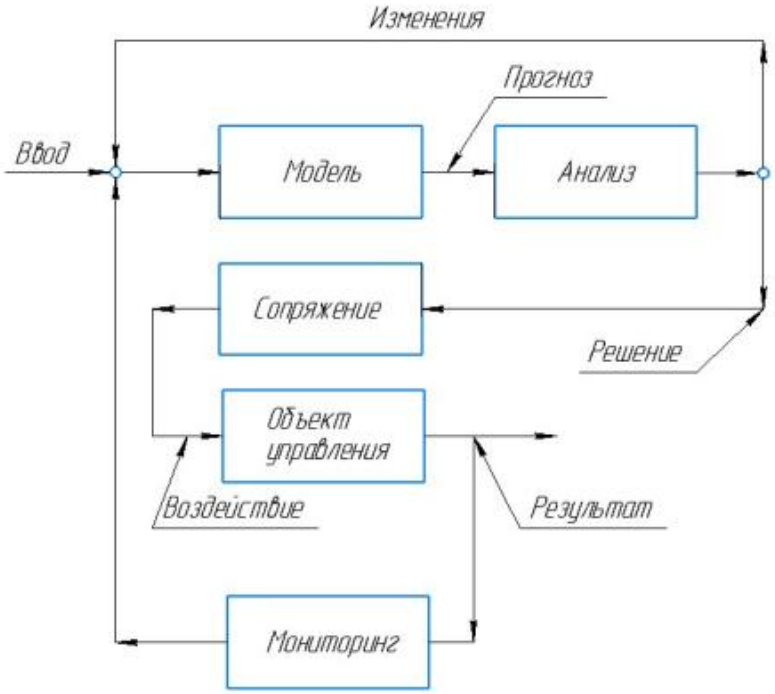

Рисунок 3 - Обобщенная схема применения модели в виде цифрового двойника

Производится постоянный мониторинг результатов указанных воздействий. Данные мониторинга вновь вводятся в модель, и цикл управления повторяется. Быстродействие модели обеспечивает возможность выполнения «обратной» задачи моделирования и получение оптимального решения в реальном масштабе времени. В качестве средств сопряжения модели с объектом управления может выступать оператор. Наличие цифрового двойника обеспечивает получение оператором оптимального решения при выборе управляющих воздействий.

Применяемый при создании цифровых двойников математический аппарат зависит от сложности объекта и степени формализуемости задачи.

Различают два вида моделей, применяемых при создании цифровых двойников: это модели временных рядов и модели предметной области.

Модели временных рядов – это такие модели, которые обеспечивают поиск зависимостей будущих значений состояний объекта от истории изменения этих состояний и построение прогноза по этим зависимостям. Эти модели универсальны для различных предметных областей и не зависят от физической природы переменных данного ряда.

Модели предметной области используют законы данной предметной области. В них используются математические, логические и вероятностные свойства, присущие конкретной предметной области. Среди моделей предметной области особо следует выделить системы, использующие агентное моделирование [10].

Рынок программного обеспечения проектирования и функционирования цифровых двойников насыщен предложениями различных разработчиков. Существуют как максимально универсальные, так и имеющие выраженную отраслевую специализацию программные продукты. Отдельное внимание заслуживают программы, предназначенные для моделирования жизненного цикла и создания цифровых аватаров.

Следует подчеркнуть, что разработка и совершенствование программного обеспечения для цифрового моделирования опирается на программные комплексы, которые являются основой для внедрения цифровых технологий, уже широко применяемых на практике. Эти технологии включают в себя облачные вычисления, сбор и обработку больших данных и многое другое.

Обзор и краткая характеристика программных продуктов (программных комплексов) для проектирования и создания цифровых двойников изделий, объектов, систем представлена в таблице 1.

Таблица 1 – Краткая характеристика программных продуктов создания цифровых двойников

Название	Характеристика
Ansys Twin Builder	Ansys Twin Builder позволяет быстро и просто разрабатывать системы, сочетая возможности создания многодоменных моделей систем с обширными библиотеками 0D - элементов для разных областей применения, 3D - решениями и моделями пониженного порядка (ROM). Данное решение включает в себя широкий набор функций для создания виртуальных прототипов реальных крупномасштабных систем, позволяя сочетать предустановленные базовые и отраслевые библиотеки компонентов с пользовательскими. Модели можно создавать как на общепринятых языках программирования (например, C/C++), так и на стандартных языках моделирования [11].
COMSOL Multiphysics	С помощью платформы COMSOL Multiphysics можно анализировать как отдельные, так и взаимосвязанные физические процессы. Среда разработки моделей, которая позволяет пройти все этапы от построения геометрической модели, задания свойств материалов и описания физики задачи до решения и визуализации результатов моделирования. Имеется возможность дополнять возможности программного пакета COMSOL Multiphysics модулями.[12].
Altair: One Total Twin platform	На этапе подготовки к производству Altair предлагает наборы инструментов для цифровых двойников, которые охватывают системные требования, разработку, проверку, прогнозирование производительности в реальных условиях, оптимизированную разработку продуктов и многое другое. Это позволяет проектировать, анализировать и оптимизировать системы без дорогостоящих натурных прототипов. В единой среде возможно объединить мультифизическое моделирование с передовыми возможностями высокопроизводительных вычислений, искусственного интеллекта и анализа данных [13].

Продолжение таблицы 1

Microsoft: Azure Digital Twins	Azure Digital Twins – это предложение "платформа как услуга" (PaaS), которое позволяет создавать графы двойников на основе цифровых моделей комплексных сред, которыми могут быть здания, фабрики, фермы, энергосистемы, железные дороги, стадионы и даже целые города. Эти цифровые модели можно использовать для получения аналитических сведений, позволяющих улучшать продукты, оптимизировать операции, сокращать расходы и повышать эффективность работы с клиентами [14].
PlantSight	Объединяет 1D, 2D и 3D-данные в единую визуальную копию актива. Кодовые метки являются связующим звеном между различными типами данных и обеспечивают их целостность. Этот просмотр предоставляется пользователям через веб-браузер, что упрощает доступ из офиса или с планшета, подключенного к сети, на самом предприятии. Доступ к облаку означает, что проверенные данные доступны всем авторизованным инженерам и операторам, независимо от их физического местонахождения. Это оптимизирует совместную работу как на местах, так и с внешними экспертами для оценки и выработки стратегии решения проблем. Цифровой двойник может обновляться в режиме реального времени данными с датчиков, находящихся на предприятии, через промышленный Интернет вещей (IIoT). Все это позволяет принимать взвешенные решения, основанные на фактах [15].

Проведённая краткая характеристика программных продуктов создания цифровых двойников изделий, объектов, систем позволила сформулировать основные задачи, стоящие перед разработчиками программного обеспечения:

— визуализация инженерных решений на основе 3D-данных;

— реализация простоты понимания полученных данных, с выделением оценки их влияния на технические системы, производственные процессы как в контексте технологии, так и на уровне предприятия;

— выстраивание профессиональной коммуникации разработчиков, пользователей с инструментарием выработки консенсусных решений;

— обеспечение высокого уровня обоснованности проектных решений.

В ходе исследования были выявлены некоторые технические ограничения, трудности создания современного программного обеспечения разработки и функционирования цифровых двойников:

— отсутствие достоверных больших данных;

— низкий уровень структурированности больших данных;

большие размеры файлов данных.

Рассматривая применение цифровых двойников и их программное обеспечение, следует отметить, что в России существуют отечественные цифровые платформы для разработки и функционирования цифровых двойников. Примерами российских компаний, занимающимися разработкой программного обеспечения в сфере создания цифровых двойников, могут служить:

— СКБ Контур. Компания СКБ Контур предлагает решения для создания цифровых двойников в различных отраслях, включая городское планирование и управление инфраструктурой. Эти решения могут помочь организациям и государственным учреждениям эффективнее управлять своей инфраструктурой и ресурсами.

— ЭЛОКОН. Компания ЭЛОКОН специализируется на разработке систем для мониторинга и управления оборудованием в промышленности. Их продукты включают решения для создания цифровых двойников и оптимизации производственных процессов, что помогает компаниям повышать эффективность и снижать издержки.

1.3 Перспективы применения цифровых двойников в рамках корпоративных киберфизических сред

Современные цифровые двойники переживают радикальную трансформацию, эволюционируя от статических моделей, используемых для симуляции отдельных параметров, до интеллектуальных систем, способных к адаптивному управлению, прогнозированию и автономному принятию решений. Эта эволюция обусловлена конвергенцией нескольких технологических прорывов, включая искусственный интеллект (ИИ), машинное обучение (МО), квантовые вычисления и когнитивные архитектуры.

Цифровые двойники всё чаще базируются на алгоритмах глубокого обучения, которые позволяют не только имитировать поведение физических объектов, но и прогнозировать их состояние в условиях неопределённости. Например, в аэрокосмической отрасли компания Airbus внедрила двойники самолётных двигателей, где рекуррентные нейросети (RNN) анализируют данные с 10 000+ датчиков в реальном времени. Эти модели предсказывают остаточный ресурс лопаток турбин с точностью 99.5%, учитывая такие факторы, как температурные перепады, вибрации и коррозию [16]. Алгоритмы обучаются на исторических данных о поломках, что позволяет системе рекомендовать оптимальные интервалы техобслуживания, снижая простои.

Новым этапом развития стала концепция «цифровых триплетов», где к физическому объекту и его виртуальной копии добавляется третий компонент — когнитивная система, способная генерировать сценарии оптимизации. Например, в автомобилестроении компания BMW создала триплеты для электромобилей, где когнитивный модуль на базе ИИ анализирует данные о стиле вождения, состоянии батареи и дорожных условиях, предлагая персонализированные рекомендации по энергопотреблению. Это позволило увеличить запас хода. В энергетике

описаны кейсы цифровых двойников ветрогенераторов, которые сокращают затраты на инспекции за счет анализа данных [17].

Квантовые технологии открывают новые горизонты для моделирования сложных систем, которые ранее были недоступны классическим компьютерам. Например, IBM Quantum разработала гибридные платформы, где квантовые алгоритмы решают задачи оптимизации для цифровых двойников нефтеперерабатывающих заводов. Такие алгоритмы, как Quantum Approximate Optimization Algorithm (QAOA), позволяют находить оптимальные маршруты транспортировки сырья за минуты вместо дней, снижая логистические.

Цифровые двойники немыслимы без интеграции с IoT-устройствами, которые обеспечивают непрерывный поток данных. Например, в умных городах двойники транспортных систем Siemens Aggregrate используют данные с 500 000+ датчиков дорожного движения, светофоров и общественного транспорта. Периферийные вычисления (Edge Computing) позволяют обрабатывать эти данные локально, сокращая задержки до 5 мс. Это критически важно для систем, требующих мгновенного реагирования, таких как управление беспилотными автомобилями [18].

Несмотря на прогресс, внедрение интеллектуальных двойников сталкивается с проблемами. Например, «чёрный ящик» нейросетей затрудняет интерпретацию решений, что критично в медицине. Для решения этого MIT разработал фреймворк Explainable AI (XAI), который визуализирует логику принятия решений двойников диагностического оборудования. Другой вызов — энергопотребление: тренировка моделей ИИ для двойников ЦОДов Google требует до 1 МВт·ч, что стимулирует разработку энергоэффективных алгоритмов, таких как TinyML.

Будущее цифровых двойников связано с их переходом к полной автономии. Например, проект NASA «Cognitive Avatars» предполагает создание двойников космических аппаратов, способных самостоятельно корректировать траекторию полёта при столкновении с астероидами [16]. В

то же время, растёт тренд на симбиоз человека и двойников: в строительстве платформа Autodesk Tandem позволяет инженерам взаимодействовать с виртуальными моделями зданий через AR-очки, внося изменения в реальном времени [17].

Тенденции в развитии цифровых двойников демонстрируют их трансформацию в ключевые элементы киберфизических сред. Интеграция ИИ, квантовых вычислений и IoT не только повышает точность моделей, но и переопределяет их роль — от инструмента анализа до активного участника производственных и социальных процессов. Однако успех зависит от преодоления технических ограничений и формирования этических стандартов, обеспечивающих прозрачность и безопасность технологий.

Современные корпоративные киберфизические среды, несмотря на их потенциал, сталкиваются с рядом сложных технических проблем, требующих междисциплинарных решений. Одной из ключевых трудностей является несовместимость платформ и систем. Разнородное программное обеспечение, такое как Ansys, COMSOL и Azure Digital Twins, а также аппаратные решения от разных производителей, создают «информационные силосы», где данные не могут свободно интегрироваться. Это приводит к дублированию усилий, ошибкам синхронизации и росту затрат. Например, на заводах Volkswagen использование протокола OPC UA (Unified Architecture) позволило связать цифровые двойники станков с ERP-системами, сократив время настройки на 30%. Для унификации взаимодействия консорциум Digital Twin Consortium разработал язык DTDL (Digital Twin Definition Language), который стандартизирует описание моделей. Компания Bosch успешно применила DTDL для интеграции двойников промышленных роботов с облачными системами SAP, что повысило точность прогнозов износа оборудования на 40%. В дополнение к этому, промежуточное ПО, такое как PTC ThingWorx, играет критическую роль, синхронизируя данные между CAD-системами (например, SolidWorks) и облачными хранилищами,

как это реализовано в аэрокосмической отрасли для обновления двойников спутников в реальном времени.

Кибербезопасность остаётся ещё одним серьёзным вызовом. Цифровые двойники становятся мишенями для атак, включая подмену данных датчиков и утечку конфиденциальной информации. Например, Siemens Energy внедрила блокчейн-технологии для защиты двойников газовых турбин, где каждое показание датчика хешируется и записывается в распределённый реестр. Это сократило число киберинцидентов на 90%. В логистике платформа Maersk TradeLens на базе блокчейна IBM Hyperledger обеспечивает сквозное шифрование данных, защищая цепочки поставок от атак. Для работы с конфиденциальными данными, такими как медицинские диагнозы, используется гомоморфное шифрование. Библиотека Microsoft SEAL позволяет обрабатывать зашифрованные данные без их расшифровки, сохраняя приватность. В финансовом секторе JPMorgan Chase внедрила квантовую криптографию Toshiba Quantum Key Distribution (QKD), генерируя неуязвимые ключи шифрования для защиты транзакций. Регуляторные аспекты также играют роль: стандарт ISO/IEC 27001 требует внедрения систем управления информационной безопасностью, что позволило Schneider Electric сертифицировать свои платформы и выйти на рынок ЕС.

Управление большими данными представляет собой отдельный комплекс проблем. Цифровые двойники, такие как модель нефтеперерабатывающего завода ExxonMobil, генерируют до 1 ПБ данных в день, что создаёт задержки в обработке и высокое энергопотребление. Для решения этих задач активно применяется Edge Computing. Мини-компьютеры NVIDIA Jetson, установленные на беспилотных грузовиках TuSimple, обрабатывают 95% данных локально, снижая нагрузку на облачные серверы. В сельском хозяйстве AWS IoT Greengrass позволяет запускать ML-модели прямо на датчиках, анализируя состояние почвы в реальном времени. Квантовые технологии также вносят вклад: стартап

Cambridge Quantum разрабатывает СУБД на алгоритмах Гровера, ускоряющих поиск данных в 1000 раз. Тестирование в CERN показало сокращение времени обработки экспериментов с 3 дней до 1 часа. Оптимизация хранения достигается за счёт алгоритмов сжатия, таких как DeepZip, которые сокращают объём данных на 80% без потери качества, как это реализовано в двойниках авиадвигателей Pratt & Whitney. Холодное хранение в Amazon Glacier архивирует неактивные двойники, снижая затраты на 70%.

Энергоэффективность становится критическим фактором, учитывая, что обучение ИИ-моделей для двойников потребляет огромные ресурсы. Например, тренировка GPT-4 для двойника городской инфраструктуры требует 10 ГВт·ч — эквивалент годового потребления 1000 домохозяйств. Для снижения энергозатрат Google DeepMind использует ИИ для оптимизации работы дата-центров, сокращая потребление на 40%. Квантовые аннигиляторы D-Wave решают задачи оптимизации с энергопотреблением в 100 раз меньше, чем классические компьютеры. Технологии TinyML, такие как TensorFlow Lite, позволяют запускать миниатюрные ML-модели на микроконтроллерах. В умных домах это помогает анализировать данные двойников систем отопления, сокращая энергопотребление на 15%.

Будущие тенденции включают развитие автономных систем безопасности, таких как Darktrace Antigena, которые автоматически блокируют кибератаки без вмешательства человека. Фотонные вычисления, реализуемые компаниями Lightmatter и Intel, используют свет вместо электронов, ускоряя обработку данных в 10 раз. Децентрализованные хранилища на базе IPFS и Filecoin распределяют данные двойников, исключая единые точки отказа.

Современные корпоративные среды, включая университетские кампусы и офисные пространства, активно трансформируются под влиянием технологий цифровых двойников, которые становятся ключевым инструментом оптимизации управления ресурсами, повышения

энергоэффективности и создания адаптивных рабочих условий. В университетах цифровые двойники используются для моделирования и контроля инфраструктуры, учебных процессов и безопасности. Например, Массачусетский технологический институт (MIT) внедрил систему, объединяющую данные с 2000+ датчиков, установленных в учебных корпусах. Эти датчики отслеживают температуру, освещение и занятость аудиторий, что позволяет автоматически регулировать работу систем отопления, вентиляции и кондиционирования (HVAC). Результатом стало сокращение энергопотребления на 25%, что не только снизило эксплуатационные расходы, но и способствовало достижению целей устойчивого развития. Университет Калифорнии в Беркли пошёл ещё дальше, интегрировав цифровой двойник кампуса с солнечными панелями и системами хранения энергии. Такой подход позволил снизить углеродный след на 30% к 2025 году, демонстрируя потенциал двойников для создания «зелёных» кампусов.

Помимо энергосбережения, цифровые двойники оптимизируют учебное пространство. В Стэнфордском университете алгоритмы искусственного интеллекта анализируют исторические данные о занятости аудиторий и предпочтениях студентов, автоматически корректируя расписание. Это сократило конфликты в расписании на 40% и повысило эффективность использования помещений на 35%, освобождая ресурсы для дополнительных образовательных инициатив. Безопасность кампусов также выходит на новый уровень благодаря двойникам. В Швейцарской высшей технической школе Цюриха (ETH Zurich) система прогнозирует риски чрезвычайных ситуаций, такие как пожары или утечки газа, с точностью 92%. Данные с камер видеонаблюдения, датчиков задымления и систем контроля доступа обрабатываются в реальном времени, сокращая время реакции служб безопасности до 2 минут, что критически важно для предотвращения инцидентов.

Офисные помещения, в свою очередь, эволюционируют от «умных зданий» к адаптивным средам, где цифровые двойники персонализируют рабочие условия. Компания WeWork использует IoT-датчики и Wi-Fi-трекинг для анализа занятости рабочих мест. На основе этих данных система автоматически распределяет сотрудников по зонам, уменьшая плотность размещения на 20% и повышая комфорт. Это особенно актуально в условиях гибридной работы, когда часть команды работает удалённо. Климатический комфорт в офисах достигается за счёт интеграции двойников с системами микроклимата. В штаб-квартире Deloitte в Амстердаме платформа Microsoft Azure Digital Twins корректирует температуру, влажность и качество воздуха, основываясь на данных датчиков и отзывах сотрудников. Это снизило количество жалоб на дискомфорт на 50%, что напрямую влияет на продуктивность. Энергоэффективность также остаётся приоритетом: в офисах Google цифровой двойник, управляемый ИИ-алгоритмами, оптимизирует нагрузку на серверы и системы охлаждения, сокращая энергопотребление.

Однако внедрение цифровых двойников в корпоративных средах сопряжено с техническими и регуляторными вызовами. Одной из ключевых проблем является конфиденциальность данных. Сбор информации о перемещениях сотрудников или студентов требует строгого соблюдения нормативов, таких как GDPR в ЕС или 152-ФЗ в России. Например, использование Wi-Fi-трекинга для анализа занятости офисов может вызывать вопросы о приватности. Решением становится анонимизация данных через блокчейн-технологии, как в случае платформы IOTA, где информация шифруется и распределяется между узлами без привязки к личности пользователя. Другой вызов — интеграция устаревшего оборудования (legacy-систем) в современные IoT-сети. Здесь помогают адаптеры OPC UA, которые преобразуют сигналы старых устройств в формат, совместимый с облачными платформами, такими как Siemens MindSphere. Кибербезопасность также остаётся критической: атаки на двойники,

например, подмена данных климат-контроля, могут парализовать работу офиса. Для защиты применяется аппаратное шифрование (Intel SGX) и многофакторная аутентификация.

Экономическая эффективность цифровых двойников подтверждается снижением операционных затрат и повышением продуктивности. В университетах превентивное обслуживание инфраструктуры, основанное на прогнозах двойников, сокращает расходы на ремонт на 25%, как в случае ETH Zurich. В офисах Deloitte оптимизация микроклимата и пространства позволила снизить текучесть кадров на 15%, увеличив удовлетворённость сотрудников. Компания Accenture отмечает, что внедрение двойников повысило продуктивность на 20% за счёт сокращения времени поиска рабочих зон и улучшения условий труда.

Цифровые двойники становятся ключевым элементом трансформации корпоративных киберфизических сред, открывая новые горизонты для управления ресурсами, оптимизации процессов и повышения устойчивости. Их применение в университетах, офисных помещениях и других корпоративных пространствах демонстрирует значительные преимущества:

— энергоэффективность и экологичность. Интеграция цифровых двойников с системами энергоменеджмента позволяет сократить потребление ресурсов на 25–30%, как показали кейсы MIT и Google. Это не только снижает эксплуатационные затраты, но и способствует достижению целей углеродной нейтральности;

— адаптивность и персонализация. В офисных средах двойники создают умные рабочие пространства, где микроклимат, освещение и распределение зон автоматически подстраиваются под потребности сотрудников. Это повышает комфорт и продуктивность, как в случае Deloitte и WeWork;

— безопасность и прогнозирование рисков. Университетские кампусы, такие как ETH Zurich, используют двойники для мониторинга ЧС, сокращая

время реакции служб до 2 минут. Это обеспечивает защиту людей и инфраструктуры в режиме реального времени;

— экономическая эффективность. ROI внедрения цифровых двойников достигает 150–220% за счёт превентивного обслуживания, оптимизации пространства и сокращения простоев. Примеры Rolls-Royce и Shell подтверждают, что инвестиции окупаются за 2–4 года.

Технологические и регуляторные вызовы, такие как кибербезопасность, конфиденциальность данных и интеграция legacy-систем, остаются критическими. Решения включают блокчейн для анонимизации данных, аппаратное шифрование и использование открытых стандартов (OPC UA, DTDL).

Будущее корпоративных сред связано с углублением симбиоза физических и виртуальных миров. Гибридные модели работы, AR-визуализация (HoloLens) и интеграция в экосистемы «умных городов» (Smart Dubai 2030) становятся новыми векторами развития. Однако успех зависит от:

— междисциплинарного сотрудничества (инженеры, data scientists, юристы);

— обучения персонала работе с ai/ml-инструментами;

— гибкого регулирования, стимулирующего инновации без ущерба для этики и приватности.

Цифровые двойники являются фундаментом для создания устойчивых, интеллектуальных и человеко-ориентированных корпоративных сред. Их потенциал будет раскрываться по мере развития квантовых вычислений, нейроморфных чипов и децентрализованных систем, формируя новую парадигму взаимодействия между человеком, машиной и цифровым пространством.

ГЛАВА 2. РАЗРАБОТКА И ИССЛЕДОВАНИЕ МОДЕЛИ И ТЕХНОЛОГИИ ЦИФРОВЫХ ДВОЙНИКОВ ДЛЯ КОРПОРАТИВНЫХ КИБЕРФИЗИЧЕСКИХ СРЕД

Разработка цифровых двойников для корпоративных сред базируется на IoT-датчиках, облачных платформах и стандартах совместимости. IoT-устройства (датчики температуры, движения, камеры) собирают данные в реальном времени: например, в MIT 2000+ датчиков управляют энергопотреблением зданий, сокращая затраты на 25%. Edge-вычисления (NVIDIA Jetson) обрабатывают данные локально, уменьшая задержки — так дроны TuSimple анализируют 95% информации на борту. Облачные платформы (Siemens MindSphere, Azure Digital Twins) интегрируют данные с ERP-систем и BIM-моделей, создавая единые цифровые модели. Для совместимости применяются стандарты OPC UA (интеграция устаревшего оборудования на заводах Volkswagen) и DTDL (описание моделей в проектах Bosch).

Интеграция legacy-систем требует адаптеров OPC UA и промежуточного ПО (PTC ThingWorx), а безопасность обеспечивается блокчейном (Siemens Energy для защиты данных турбин) и гомоморфным шифрованием (Microsoft SEAL в медицине). В ETH Zurich облачная платформа объединяет данные с 500+ датчиков, прогнозируя ЧС с точностью 92%, а в Smart Dubai двойники офисов и университетов синхронизируются через IBM Watson IoT, снижая энергопотребление города на 20%. Перспективы включают квантовые вычисления (IBM Quantum для оптимизации нефтезаводов) и нейроморфные чипы (Intel Loihi для энергоэффективных дронов). Успешная интеграция цифровых двойников зависит от выбора технологий, стандартов и решения вопросов безопасности, создавая основу для умных, адаптивных и устойчивых корпоративных сред.

2.1 Методы и технологии создания цифровых двойников

Создание цифровых двойников для корпоративных сред требует уникального подхода, учитывающего специфику каждой отрасли. В отличие от универсальных решений, здесь ключевым становится адаптация технологий под конкретные бизнес-процессы. Например, в розничной торговле двойники используются для оптимизации цепочек поставок, а в здравоохранении — для моделирования работы операционных.

Разработка цифровых двойников для корпоративных сред — это многослойный процесс, требующий тщательного подбора инструментов, которые позволяют не только моделировать физические объекты, но и обеспечивать их интеграцию с реальными данными и бизнес-системами. Технологический стек варьируется в зависимости от задач: от быстрого прототипирования до сложных научных симуляций. Далее представляются ключевые компоненты, их роль и примеры применения.

Low-code платформы идеальны для проектов, где скорость внедрения и минимальные требования к программированию критически важны. Такие решения, как PTC ThingWorx или AWS IoT TwinMaker, предлагают готовые модули для сбора данных с IoT-устройств, аналитики и визуализации. Например, на заводах Bosch платформа ThingWorx используется для мониторинга станков: датчики вибрации и температуры передают данные в реальном времени, а алгоритмы машинного обучения прогнозируют износ деталей, сокращая простои на 20%. В умных зданиях Azure Digital Twins помогает управлять микроклиматом: система анализирует показатели влажности и освещения, автоматически корректируя работу HVAC-систем. Преимущество low-code в том, что даже малые предприятия могут создавать функциональные двойники без привлечения дорогостоящих разработчиков, экономя до 50% бюджета [19].

Для более сложных задач, таких как моделирование физических процессов или интеграция уникальных алгоритмов, требуются специализированные SDK и фреймворки. Python с библиотеками

NumPy и SciPy становится основой для математических расчетов. Например, в Массачусетском технологическом институте (MIT) эти инструменты используются для моделирования теплопотерь зданий: уравнения теплопередачи интегрируются с данными датчиков, что позволяет оптимизировать энергопотребление. В аэрокосмической отрасли MATLAB/Simulink незаменимы для симуляции динамических систем. Для робототехники ROS 2 (Robot Operating System) предоставляет гибкую среду разработки: Boston Dynamics использует его для тестирования алгоритмов управления роботами в виртуальных средах перед физическим внедрением.

Графические движки, такие как Unity и Unreal Engine, добавляют интерактивность и наглядность. Siemens, например, создает виртуальные фабрики в Unity, где инженеры могут тестировать изменения производственных линий — от перенастройки конвейеров до оптимизации логистики. Это сокращает время внедрения инноваций на 40%. Unreal Engine, благодаря фотореалистичному рендерингу, используется в аэрокосмической отрасли для симуляции полетов дронов и тренировки пилотов. Для веб-решений WebGL/Three.js позволяют визуализировать двойники прямо в браузере. Университетские кампусы используют Three.js для онлайн-мониторинга энергопотребления, предоставляя администраторам доступ к данным через интуитивные 3D-интерфейсы.

Инструменты IoT — это фундамент, связывающий физический и цифровой миры. Протокол MQTT обеспечивает легковесную передачу данных с датчиков, что особенно важно в умных домах, где тысячи устройств (лампы, термостаты, камеры) обмениваются информацией. В промышленности Apache Kafka обрабатывает потоки данных в реальном времени: на заводах Tesla Kafka агрегирует информацию с 10 000+ датчиков, чтобы двойники могли мгновенно реагировать на аномалии. Node-RED, платформа визуального программирования, упрощает создание IoT-

сценариев. В проектах умных городов она автоматизирует управление светофорами, анализируя данные о трафике с камер и датчиков [20].

Выбор технологического стека зависит от масштаба и специфики проекта. Для небольших задач, таких как мониторинг офисных зданий, low-code платформы вроде AWS IoT TwinMaker позволяют развернуть решение за недели с бюджетом до 50тыс. Гибридные подходы, например, комбинация PTC ThingWorx для сбора данных и Unity для визуализации, часто становятся оптимальным решением.

Примеры успешных реализаций иллюстрируют универсальность подхода. В медицине Mayo Clinic использует стек Python (TensorFlow) + ROS 2 + WebGL для создания двойников пациентов: алгоритмы анализируют данные МРТ и wearable-устройств, прогнозируя риски осложнений с точностью до 85%. В проекте Smart Dubai комбинация AWS IoT TwinMaker, Apache Kafka и Unreal Engine позволила синхронизировать энергопотребление городской инфраструктуры, сократив затраты на 18%.

Технологический стек для цифровых двойников — это всегда баланс между скоростью, точностью и бюджетом. Low-code платформы демократизируют доступ к технологиям, SDK обеспечивают глубину анализа, а графические движки делают данные осязаемыми. Будущее лежит в интеграции ИИ и квантовых вычислений: например, IBM уже применяет квантовые алгоритмы для оптимизации логистических моделей. Однако успех любого проекта зависит не только от инструментов, но и от понимания бизнес-задач — только тогда цифровые двойники становятся не только виртуальными копиями, но и стратегическими активами.

Цифровые двойники — это динамические системы, требующие постоянного обновления и адаптации к изменениям в реальном мире. Их жизненный цикл охватывает разработку, тестирование, внедрение, эксплуатацию и вывод из эксплуатации. Управление этим циклом основано на автоматизации процессов, контроле версий и непрерывном мониторинге.

Для поддержания актуальности цифровых двойников используются CI/CD-конвейеры (Continuous Integration/Continuous Delivery) (Рисунок 4). Эти системы автоматизируют тестирование, сборку и развертывание новых версий моделей, минимизируя риски ошибок. Например, на заводах Toyota конвейеры Jenkins ежедневно обновляют двойники роботов-сварщиков. После внесения изменений в алгоритмы происходит автоматическая проверка: архивные данные используются для тестирования, а обновление развертывается только при повышении точности прогнозов [21].

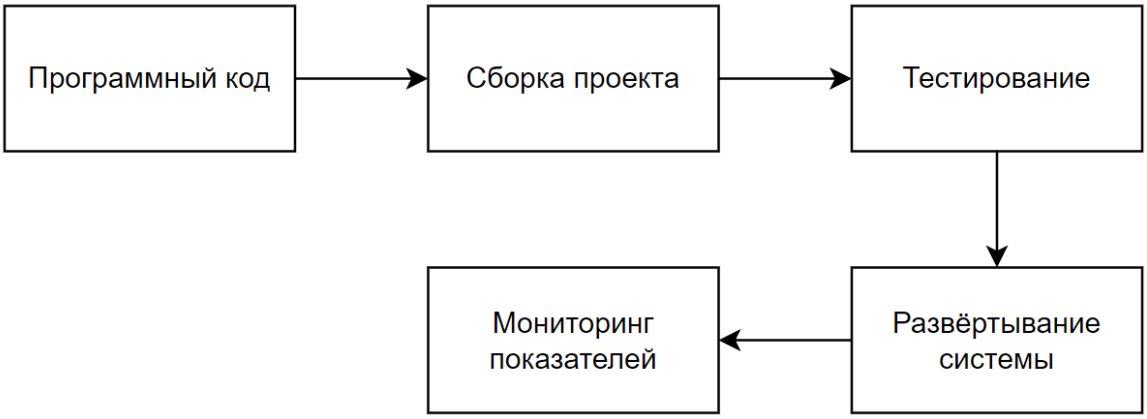

Рисунок 4 – Структурная схема CI/CD конвейера

Каждое изменение проходит через этапы автоматической проверки. Например, обновление модели прогноза износа станка сначала тестируется на данных прошлых месяцев, а затем внедряется в производство.

Немаловажной технологией, используемой при разработке ЦД является версионирование моделей и MLOps-платформы. MLOps (Machine Learning Operations) — это набор практик для управления версиями моделей, данных и экспериментов. Эти инструменты обеспечивают воспроизводимость результатов и контроль изменений [22].

Таблица 2- Инструменты MLOps

Инструмент	Роль	Пример применения
MLflow	Логирование экспериментов, управление моделями	General Electric хранит 50+ версий моделей прогноза отказов турбин.
Kubeflow	Оркестрация ML-пайплайнов в Kubernetes	BMW обучает модели для автономных роботов, распределяя вычислительные ресурсы между командами.
DVC	Контроль версий данных и моделей	В фармацевтике DVC отслеживает изменения в клинических данных для двойников пациентов.

MLflow фиксирует параметры каждой версии модели: точность, гиперпараметры, метрики. Это позволяет быстро вернуться к предыдущей версии при сбоях. Kubeflow управляет распределенными вычислениями, что критично для обучения моделей на больших массивах данных, например, в автономных транспортных системах. DVC синхронизирует данные и код, предотвращая конфликты при совместной работе над проектами (Рисунок 5).

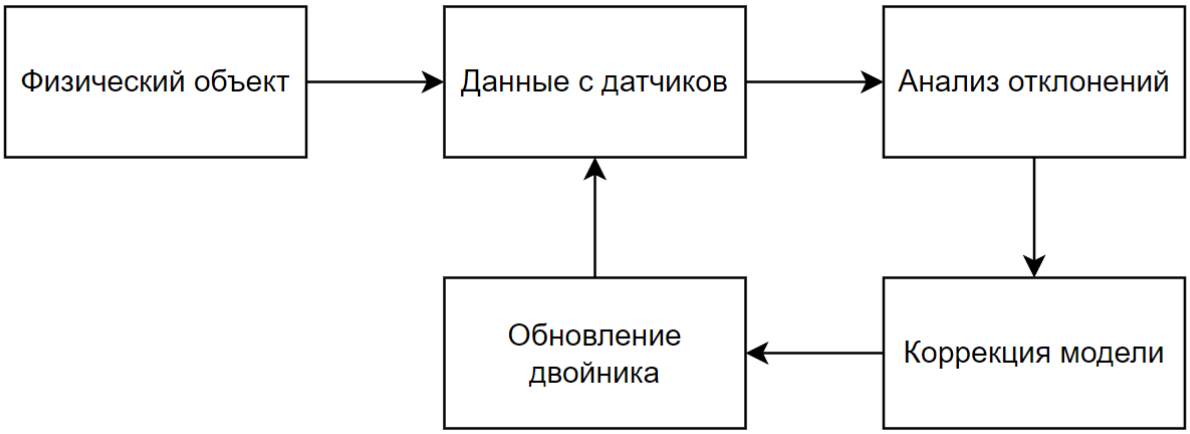

Рисунок 5 – Структурная схема цифрового двойника с адаптацией

Классический процесс управления включает этапы моделирования, анализа и проектирования MAD (modeling, analysis and design). Данный подход основан на двух ключевых вопросах о том какие требования предъявляются к системе и какие данные доступны для моделирования и проектирования регулятора.

На первом этапе создается математическая модель системы, описываемая передаточной функцией, пространством состояний или нелинейными зависимостями, полученными с помощью методов идентификации систем.

На втором этапе проводится анализ устойчивости системы, позволяющий выбрать оптимальную стратегию управления (PID, прогнозирующее управление, адаптивное или робастное управление и др.).

На третьем этапе разрабатывается математическая модель регулятора и производится настройка параметров. Затем проводится моделирование, проверяющее, соответствует ли отклик системы заданным условиям. После успешной симуляции алгоритм управления внедряется в физическое устройство.

Цифровой двойник позволяет обновлять модель системы в реальном времени, формируя замкнутый контур управления с обратной связью, что делает систему адаптивной [23].

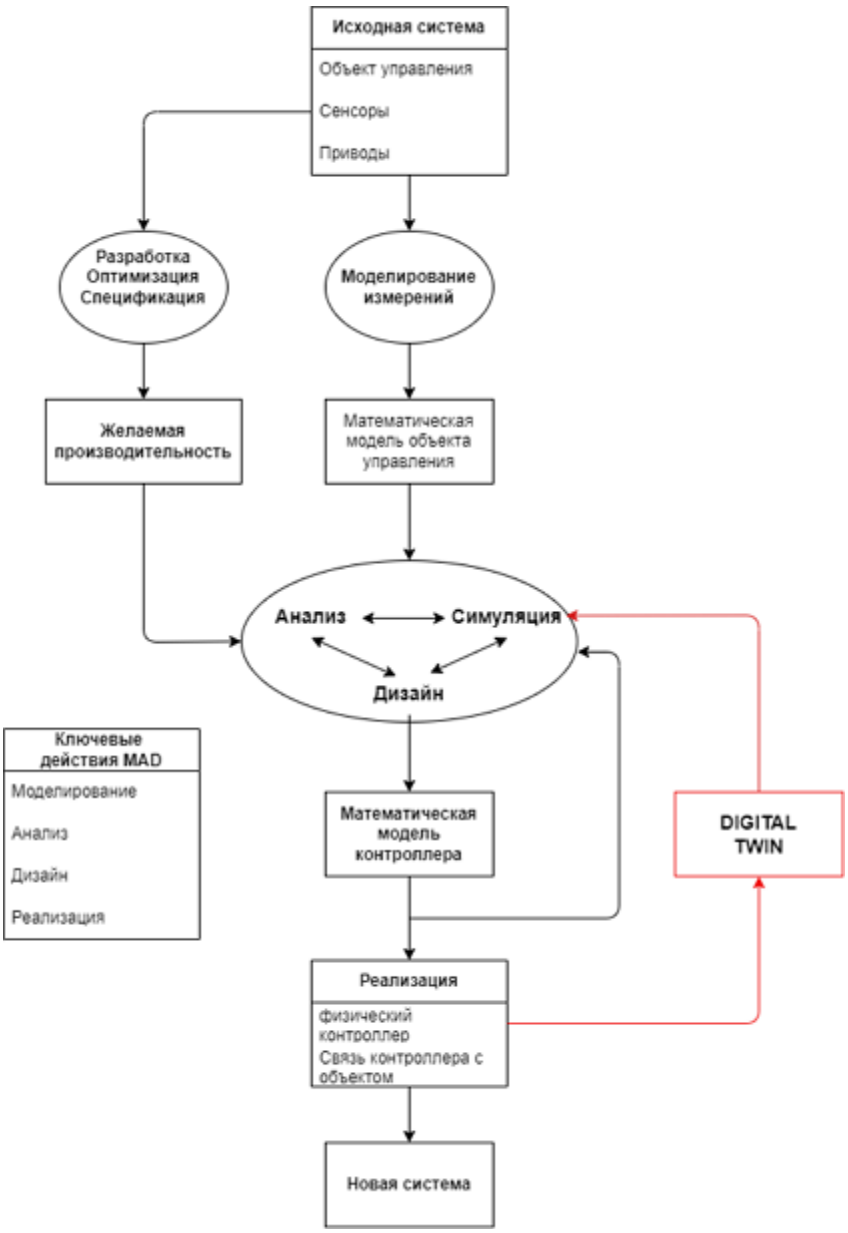

Рисунок 6 – Адаптивная система управления с DT

Двойники должны непрерывно адаптироваться к изменениям физических объектов. Системы мониторинга отслеживают отклонения между моделью и реальностью, а алгоритмы автоматически корректируют параметры (Рисунок 6). Если двойник прогнозирует отклонение, система отправляет предупреждение инженерам.

2.2 Структура цифрового двойника

Цифровые двойники, представляя собой динамические виртуальные копии физических объектов или процессов, требуют системного подхода к управлению их жизненным циклом. Этот процесс начинается с разработки модели, основанной на анализе бизнес-целей и сборе данных из разнородных источников, таких как IoT-датчики, BIM-модели и исторические архивы. На этом этапе выбор методологии моделирования играет ключевую роль: детерминированные модели, основанные на физических законах, применяются для прогнозирования нагрузки на сети, тогда как машинное обучение (LSTM-сети, нейросети) используется в логистике для оптимизации грузопотоков. Гибридные подходы, сочетающие физику и ИИ, находят применение в авиации, где Airbus моделирует работу двигателей, учитывая как аэродинамические уравнения, так и данные датчиков. Инструменты, такие как Ansys Twin Builder и TensorFlow, обеспечивают гибкость разработки, позволяя адаптировать модели под специфику задач.

После разработки следует этап тестирования, направленный на валидацию точности и устойчивости двойника. Валидация включает сравнение прогнозов модели с реальными данными. Симуляция экстремальных сценариев, таких как аварийные отключения энергосетей или экстремальные погодные условия, позволяет оценить устойчивость модели. Например, A/B-тестирование, как в офисах Deloitte, где сравнивается работа HVAC-систем с двойником и без него, помогает количественно оценить эффективность внедрения.

Интеграция цифрового двойника в корпоративную инфраструктуру требует решения задач совместимости и масштабируемости. Подключение к IoT-устройствам через протоколы OPC UA и MQTT обеспечивает передачу данных с датчиков, как на заводах Volkswagen, где устаревшее оборудование адаптируется с помощью шлюзов. Интеграция с ERP/CRM-системами, такими как SAP или Salesforce, автоматизирует бизнес-процессы: в логистических центрах Amazon двойники синхронизируются с системами

управления запасами, сокращая время обработки заказов. Настройка REST и GraphQL API, как в проекте Smart Dubai, позволяет связывать двойники транспорта с городскими службами. Проблемы несовместимости данных решаются инструментами вроде Apache NiFi, а задержки передачи минимизируются за счет Edge Computing (NVIDIA Jetson), обрабатывающего данные локально.

Эксплуатация цифрового двойника представляет собой непрерывный цикл мониторинга, обновления и адаптации. Системы визуализации, такие как Grafana и Tableau, отслеживают отклонения между моделью и физическим объектом в реальном времени. На нефтяных вышках Shell дашборды отображают давление и температуру, сигнализируя об аномалиях, в то время как CI/CD-пайплайны (Jenkins, GitLab CI) автоматически развертывают новые версии моделей. Например, Airbus обновляет двойники двигателей после каждого полета, используя данные о вибрациях и температуре. Алгоритмы обратной связи, как в роботах Boston Dynamics, корректируют параметры моделей в ответ на изменения внешней среды. MLOps-платформы (MLflow, Kubeflow) управляют версиями моделей, обеспечивая воспроизводимость результатов, а системы мониторинга (Prometheus, Datadog) собирают метрики производительности для оптимизации.

Архивация завершает жизненный цикл, снижая затраты на хранение неактивных моделей. Устаревшие двойники, такие как модели станков 1990-х годов, переносятся в «холодные» хранилища (Amazon Glacier), сохраняя метаданные для возможного восстановления. Siemens, например, архивирует двойники старых производственных линий после модернизации, обеспечивая доступ к историческим данным для аудита.

Системный подход, сочетающий автоматизацию, адаптивность и стратегическую архивацию, превращает двойники в ключевые активы для оптимизации ресурсов и стимулирования инноваций. Успешные кейсы подтверждают, что эффективное управление жизненным циклом не только

повышает операционную эффективность, но и обеспечивает долгосрочную конкурентоспособность в условиях цифровой трансформации.

Цифровой двойник представляет собой сложную многоуровневую систему, объединяющую физические и виртуальные элементы для моделирования, анализа и прогнозирования поведения объекта или процесса. Его архитектура формируется из взаимосвязанных компонентов, обеспечивающих сбор, обработку, хранение и передачу данных. Физический объект или процесс, являющийся основой цифрового двойника, служит источником исходных данных. Это может быть промышленное оборудование, здание, транспортное средство или биологическая система. Физический объект генерирует данные через сенсорную сеть, которая включает датчики, измеряющие механические, термические, электрические или химические параметры. Например, акселерометры фиксируют вибрации, термопары — температуру, а газоанализаторы определяют состав веществ. Сенсоры передают данные в аналоговом или цифровом формате, используя протоколы, такие как Modbus, OPC UA или MQTT, которые обеспечивают совместимость с другими компонентами системы.

Собранные данные поступают в модель данных, где структурируются и нормализуются. Модель данных определяет форматы хранения (JSON, XML, Protobuf) и включает метаданные, описывающие источники информации, единицы измерения и временные метки. Онтологии, или семантические модели, связывают данные с предметной областью, например, Industry 4.0 или Smart City, обеспечивая контекстную интерпретацию. На этом этапе выполняется валидация данных, проверяющая их корректность и полноту, а также интеграция с внешними системами, такими как ERP или SCADA.

Аналитический модуль обрабатывает структурированные данные, выполняя симуляции, прогнозирование и оптимизацию. Этот модуль может включать физическое моделирование, основанное на решении уравнений теплопередачи, гидродинамики или механики, а также алгоритмы машинного обучения, такие как LSTM или Random Forest, для классификации и

регрессии. Гибридные модели сочетают физические законы с искусственным интеллектом, например, используя подкрепляющее обучение для адаптации к изменяющимся условиям. Результаты анализа — прогнозы состояния объекта, рекомендации по оптимизации режимов работы или симуляционные сценарии — передаются через интерфейсы взаимодействия.

Интерфейсы обеспечивают связь между цифровым двойником, пользователями и внешними системами. API, такие как REST или GraphQL, интегрируют двойник с ERP, MES или CRM, автоматизируя бизнес-процессы. Веб-интерфейсы, включая дашборды Grafana или Tableau, визуализируют данные в реальном времени, отображая графики, отчеты и предупреждения. Для интерактивного взаимодействия могут использоваться AR/VR-интерфейсы, созданные в Unity или Unreal Engine, которые предоставляют 3D-визуализацию модели. Управляющие команды, такие как изменение параметров симуляции, передаются от интерфейсов к аналитическому модулю, замыкая цикл управления.

Системы хранения данных обеспечивают долгосрочное сохранение информации. Базы данных временных рядов, такие как InfluxDB или TimescaleDB, оптимизированы для работы с данными сенсоров, записываемыми с высокой частотой. Облачные хранилища (AWS S3, Azure Blob) архивируют большие объемы данных, включая резервные копии и исторические записи. Распределенные системы, такие как HDFS, используются для обработки больших данных в промышленных масштабах. Данные разделяются на сырые (необработанные сигналы), обработанные (структурированные наборы) и метаданные, описывающие версии моделей и источники информации.

Механизмы обратной связи позволяют корректировать модель на основе расхождений между прогнозами и реальными показателями. В замкнутом цикле коррекция параметров выполняется автоматически, например, через адаптивные алгоритмы машинного обучения, которые переобучаются на новых данных. В разомкнутом цикле изменения вносятся

вручную на основе экспертной оценки. Данные для коррекции включают выявленные аномалии, новые параметры объекта или изменения внешних условий, не учтенные в исходной модели.

Связи между компонентами организованы в виде прямых, обратных и циклических потоков данных. Прямой поток начинается с физического объекта, передающего данные через сенсоры в модель данных, которая структурирует информацию для аналитического модуля. Результаты анализа поступают в интерфейсы, где визуализируются или преобразуются в управляющие команды. Обратный поток направляет эти команды через аналитический модуль и модель данных к физическому объекту, корректируя его режимы работы. Циклический поток обеспечивает итеративную оптимизацию модели, когда данные из аналитического модуля возвращаются для уточнения параметров.

Технические стандарты и протоколы, такие как OPC UA для промышленной интеграции, MQTT для легковесной передачи данных и REST/GraphQL для API-взаимодействия, обеспечивают совместимость компонентов. Безопасность данных гарантируется шифрованием (TLS/SSL для передачи, AES-256 для хранения), аутентификацией (OAuth 2.0, JWT-токены) и контролем доступа (RBAC). Архитектура цифрового двойника, объединяющая эти элементы, позволяет создавать универсальные системы для моделирования и оптимизации объектов любой сложности, обеспечивая их адаптивность и точность в динамически изменяющихся условиях.

Цифровой двойник состоит из множества взаимосвязанных компонентов. Он получает данные с физических объектов через периферийные линии данных и IoT-устройства, а затем анализирует их, используя многодоменные модели и алгоритмы машинного обучения. Это позволяет создавать цифровые экземпляры объекта для решения различных задач — проектирования систем управления, диагностики состояния оборудования и прогнозирования его работы.

Цифровой двойник включает пять ключевых компонентов:

1. Сенсоры — собирают данные о работе системы и окружающей среде.

2. Исполнительные устройства — применяют управляющие воздействия на систему.

3. Данные — формируют основу для анализа и прогнозирования.

4. Моделирование — воспроизводит поведение системы с учетом физических и статистических моделей.

5. Аналитика — обрабатывает данные для визуализации, прогнозирования и управления.

Существует две основные архитектуры цифровых двойников:

— трехуровневая модель (grieves, 2003) — включает физический объект, его виртуальную копию и механизм связи между ними. Такая структура позволяет создавать цифровой двойник, но не учитывает сложные аспекты его интеграции и эксплуатации.

— пятиуровневая модель — включает физический объект, виртуальную модель, систему сервисов, модель данных и систему коммуникаций. Эта архитектура обеспечивает более точное моделирование, взаимодействие с реальным миром и аналитику на основе больших данных [24].

Пятиуровневая модель объединяет физические и виртуальные данные для более полного и точного сбора информации. Она может производить анализ и прогнозирование в моделях сервисов для унифицированного управления и использования по требованию.

Согласно предложенной архитектуре цифровой двойник для сложного оборудования можно представить следующим образом:

$$M_{DT} = (PE, VE, S_S, DD, CN) \quad (1)$$

где PE обозначает физическую сущность, VE — виртуальное оборудование, Ss — сервисы для PE и VE, DD — данные цифрового двойника, а CN — соединение между PE, VE, Ss и DD.

Модель физической сущности (PE) состоит из различных функциональных подсистем и сенсорных устройств. Подсистемы выполняют заранее определенные задачи во время работы, а сенсоры собирают данные о состоянии подсистем и рабочих условиях. Неисправность любого компонента может привести к сбою PE. Функциональные подсистемы ветрогенератора включают: лопасть, генератор, редуктор, систему поворота и т. д. Они преобразуют энергию ветра в механическую и электрическую энергию. Сенсоры используются для сбора данных о температуре генератора, вибрации редуктора, выходной мощности и других параметрах.

Модель виртуального оборудования (VE) представляет собой цифровой аналог PE, который интегрирует множество переменных и характеристик PE для воспроизведения его геометрии, физических свойств, поведения и правил в виртуальной среде. Для ветрогенератора можно смоделировать, например, деформацию лопасти, напряжение в зубьях шестерен и температуру подшипников.

Модель сервисов (Ss) включает сервисы для PE и VE. Она оптимизирует работу PE и поддерживает высокую точность VE путем калибровки параметров во время работы для обеспечения синхронизации с PE. Ss состоит из следующих элементов: функция, вход, выход, качество, состояние. В качестве примера можно выделить сервис мониторинга выходной мощности для физического ветрогенератора. Его можно представить как Ss_monitor = (Мониторинг выходной мощности, (скорость ветра, выходная мощность физического WT, выходная мощность виртуального TW), состояние мощности, (время, стоимость, надежность), (работа, простои, отказ).

Модель данных (DD) включает пять частей, как показано ниже:

$$DD = (D_p, D_v, D_S, D_k, Df) \tag{2}$$

Где D_p — данные от PE, D_v — данные от VE, D_S — данные от Ss, D_k — отраслевые знания, а Df — объединенные данные. Модель данных

объединяет данные из физических и виртуальных источников, а также их комбинации, что значительно расширяет объем информации.

Модель соединения (CN) включает шесть частей, представленных в следующем выражении

$$CN = (CN_S D, D_v, D_S, D_k, Df) \quad (3)$$

Где CN_SD, CN_PD, CN_VD, CN_PS, CN_VS и CN_PV обозначают соединения между Ss и DD, PE и DD, VE и DD, PE и Ss, VE и Ss, PE и VE соответственно. Каждое соединение (обозначаемое как CN_XX) является двунаправленным, а передаваемые данные моделируются следующим образом:

CN_XX = (Источник данных, Единица измерения, Значение, Диапазон, Интервал выборки).

Цифровые двойники классифицируются по уровню развития:

Уровень I (пред ЦД): Создание виртуального прототипа системы без физического объекта.

Уровень II (моделируемый ЦД): Наличие физической системы и ее цифрового двойника без прямой связи в реальном времени.

Уровень III (операционный ЦД): Полноценный цифровой двойник, обновляющийся в реальном времени на основе данных физической системы.

Уровень IV (интеллектуальный ЦД): Использование машинного обучения и искусственного интеллекта для автоматизированного управления физической системой.

Несмотря на перспективность технологии, существуют определенные сложности:

— моделирование: Разработка многодисциплинарных моделей требует высокоточной симуляции и унификации стандартов данных;

— интеграция сенсоров: Реализация надежной передачи данных требует высокой вычислительной мощности и устойчивых коммуникационных протоколов;

— работа с большими данными: Современные DT-системы используют не только физические модели, но и экспертные знания, что затрудняет автоматизированную настройку и оптимизацию.

В отличие от классического моделирования, DT использует поток реальных данных, обновляющих виртуальную модель, что позволяет не только предсказывать поведение системы, но и принимать управленческие решения в реальном времени. Цифровой двойник — это не просто симуляция, а комплексная технология, объединяющая моделирование, сенсорику, аналитику и управление, что делает ее одним из ключевых инструментов цифровой трансформации промышленности.

В аналитическом модуле цифровых двойников часто применяются алгоритмы LSTM (Long Short-Term Memory) — разновидность рекуррентных нейронных сетей, предназначенная для обработки временных рядов. LSTM-сети способны запоминать долгосрочные зависимости в данных благодаря своей архитектуре, включающей ворота (input, forget, output), которые регулируют поток информации. Это делает их идеальными для прогнозирования отказов оборудования, анализа трендов энергопотребления или моделирования поведения объекта в динамически изменяющихся условиях. Для решения задач, требующих точного описания физических процессов, используются детерминированные модели, основанные на дифференциальных уравнениях. Например, уравнения Навье-Стокса применяются для гидродинамических симуляций, а законы термодинамики — для прогнозирования тепловых режимов. Инструменты вроде Ansys Twin Builder или COMSOL Multiphysics позволяют интегрировать физические модели с данными IoT-устройств, создавая гибридные цифровые двойники.

Хранение данных в цифровых двойниках организовано с использованием специализированных систем. InfluxDB, как база данных временных рядов, оптимизирована для работы с высокочастотными данными сенсоров, предоставляя функции агрегации (среднее, максимум, минимум) и язык запросов Flux для анализа. Для долгосрочного хранения больших

объемов данных применяется AWS S3 (Simple Storage Service) — облачное хранилище, обеспечивающее масштабируемость и отказоустойчивость. В промышленных средах, где объемы данных достигают петабайтов, задействуется HDFS (Hadoop Distributed File System), распределенная файловая система, которая разбивает данные на блоки и распределяет их между узлами кластера, обеспечивая параллельную обработку.

Визуализация данных в цифровых двойниках реализуется через платформы вроде Grafana, которая создает интерактивные дашборды для мониторинга метрик в реальном времени, таких как температура, давление или энергоэффективность. Для сложной 3D-визуализации и создания иммерсивных интерфейсов используются игровые движки Unity и Unreal Engine, позволяющие строить точные цифровые копии объектов (заводов, зданий) и моделировать их поведение в виртуальной среде.

Безопасность данных в цифровых двойниках обеспечивается на нескольких уровнях. TLS/SSL (Transport Layer Security/Secure Sockets Layer) шифрует передаваемую информацию между датчиками, облаком и аналитическими модулями, предотвращая перехват данных. Для защиты данных при хранении применяется алгоритм AES-256 (Advanced Encryption Standard), обеспечивающий военную степень шифрования. Аутентификация и авторизация пользователей реализуются через OAuth 2.0 и JWT (JSON Web Tokens), которые управляют доступом к API и дашбордам, проверяя права пользователей на основе ролей (RBAC — Role-Based Access Control).

2.3 Модель интеграции цифрового двойника в киберфизическую среду

Интеграция цифрового двойника в киберфизическую среду представляет собой многоуровневый процесс, основанный на взаимодействии аппаратных и программных компонентов. Первым этапом является сбор данных, который обеспечивается сетью IoT-устройств и специализированных датчиков. Эти устройства фиксируют физические параметры объекта, такие как температура, давление, вибрация или биометрические показатели, и передают их через протоколы связи, адаптированные под конкретные условия. Например, в сценариях с большими расстояниями и низким энергопотреблением применяются LPWAN-технологии, включая LoRaWAN и NB-IoT, тогда как в промышленных системах с жесткими требованиями к синхронизации используется TSN (Time-Sensitive Networking). Для интеграции разнородного оборудования критически важны универсальные стандарты, такие как OPC UA, обеспечивающий семантическую совместимость, и MQTT, оптимизированный для обмена сообщениями в режиме реального времени. Предварительная обработка данных осуществляется на уровне edge-устройств, что позволяет снизить задержки и уменьшить объем передаваемой информации. Это включает фильтрацию шумов, агрегацию показателей и локальную аналитику, что особенно актуально в системах, где мгновенная реакция критична, таких как автономные транспортные средства или роботизированные производственные линии.

Следующий этап — обработка данных — предполагает их трансформацию в цифровую модель, которая отражает состояние и поведение физического объекта. Облачные платформы, такие как AWS IoT Core или Azure Digital Twins, предоставляют инфраструктуру для масштабируемого хранения, анализа и симуляции. Здесь применяются алгоритмы машинного обучения, включая нейросетевые модели (TensorFlow, PyTorch) и методы анализа временных рядов, чтобы выявлять аномалии,

прогнозировать износ оборудования или оптимизировать энергопотребление. Отдельное внимание уделяется концепции «цифровых теней» — упрощенных реплик физических объектов, которые позволяют тестировать сценарии и корректировать параметры без вмешательства в реальные процессы. Например, цифровая тень может имитировать нагрузку на энергосеть при изменении спроса, что помогает заранее адаптировать систему к пиковым условиям.

Механизмы обратной связи обеспечивают замкнутый цикл управления, где данные из цифрового двойника преобразуются в действия, влияющие на физический объект. Для этого используются программируемые логические контроллеры (PLC), роботизированные системы и автономные устройства, которые получают команды через API или промышленные шлюзы. REST-интерфейсы обеспечивают интеграцию с ERP-системами и облачными сервисами, а специализированные протоколы, такие как Kepware, позволяют подключать устаревшее оборудование к современным платформам. Динамическая адаптация в реальном времени достигается за счет непрерывного обмена данными между цифровым двойником и физическим объектом, что позволяет оперативно корректировать параметры работы. Например, в энергетических системах это может выражаться в автоматической регулировке мощности генераторов на основе прогнозов нагрузки, сформированных моделью.

Стандартизация играет ключевую роль в обеспечении совместимости и безопасности компонентов киберфизической среды. Международные стандарты, такие как ISO/IEC 30141, определяют архитектурные принципы IoT-систем, а IEC 62443 устанавливает требования к защите от кибератак. Открытые платформы, включая FIWARE и Eclipse Ditto, предоставляют инструменты для создания унифицированных интерфейсов и управления цифровыми двойниками через гибкие API. Это позволяет избежать фрагментации экосистемы и упрощает масштабирование решений [25].

Архитектура интеграции представляет собой последовательную цепочку: физический объект, оснащенный датчиками, передает данные через edge-устройства в облако, где они анализируются и преобразуются в прогнозы или рекомендации. Результаты передаются обратно через системы управления, замыкая цикл и обеспечивая непрерывную оптимизацию. Таким образом, интеграция цифрового двойника требует не только технологической связности компонентов, но и глубокой синхронизации между физическими и цифровыми слоями системы, что формирует основу для интеллектуального управления в рамках Industry 4.0.

Интегрирование технологии цифровых двойников с интернетом вещей позволяет получать данные, необходимые для понимания того, как физический близнец (например, производственная сборочная линия, сеть автономных транспортных средств) ведет себя и работает в условиях эксплуатации. Кроме того, совместное применение Интернета вещей и Цифровых двойников поможет оптимизировать профилактическое обслуживание для совершенствования физического объекта и бизнес-процессов. Действуя как мост между физическим и виртуальным миром, Интернет вещей может передавать данные о производительности, обслуживании и работоспособности от физического близнеца к цифровому. Объединение аналитики реальных данных с прогнозным моделированием может улучшить способность принимать обоснованные решения, которые потенциально могут привести к созданию более эффективных систем, разработке оптимизированных производственных операций и новых бизнес-моделей. Кроме того, Интернет вещей обеспечивает гибкость, столь необходимую, когда речь идет о мобильности системы, ее расположении и вариантах монетизации. Подобная гибкость способствует созданию новых вариантов бизнеса, таких, например, как продажа возможностей (то есть продукта как услуги).

Важно отметить, что комбинация Цифрового двойника и Интернета вещей позволяет организации получить представление о том, как система/продукт используется клиентами. Такое понимание позволит клиентам оптимизировать график технического обслуживания и применение ресурсов, заблаговременно прогнозировать возможные сбои продукта и избегать/сокращать простои системы

Потенциальные преимущества интернета вещей лежат в сфере сервисов. Например, служба, которая непрерывно получает данные о рабочем состоянии системы, может быть эффективной в обеспечении экономии затрат и высокой отказоустойчивости. Аналогично можно, например, использовать прогнозную аналитику для предварительного подбора и быстрой доставки требуемой детали обслуживающей бригаде.

Для реализации цифровых двойников в системах управления используются устройства периферийных (edge) вычислений. Edge-компьютинг — это вычислительная парадигма, в которой обработка данных выполняется ближе к их источнику, снижая нагрузку на облачные серверы. Такие устройства способны: запускать сложные вычислительные модели, подключаться к датчикам и исполнительным устройствам, обрабатывать данные в реальном времени, а также поддерживать промышленные протоколы связи.

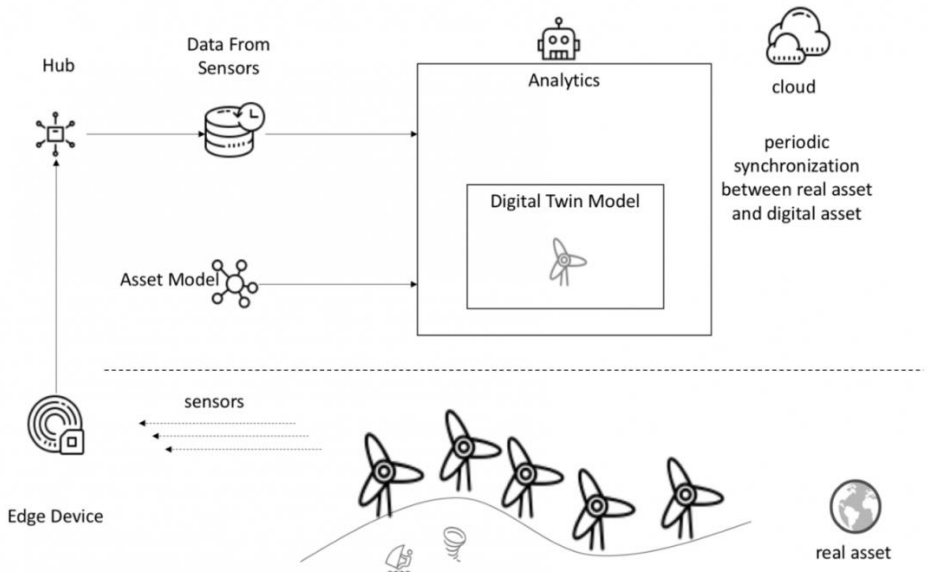

Рисунок 7 – Edge-платформа

На рынке представлено множество решений для edge-вычислений. Выбор конкретного устройства и платформы зависит от требований системы. Цифровые двойники в сочетании с edge-вычислениями открывают новые возможности для предсказания отказов, адаптивного управления и оптимизации промышленных процессов (Рисунок 7).

Интеллектуальная система обладает возможностями сенсорики, управления исполнительными механизмами, способная принимать решения на основе текущих и исторических данных для обеспечения заданных характеристик в условиях неопределенности и внешних возмущений.

Интеллектуальное управление базируется на трех ключевых технологиях:

— промышленный искусственный интеллект (IAI);

— цифровой двойник (DT);

— самонастраивающееся управление (SOC).

Промышленный AI (IAI) ориентирован на мониторинг, оптимизацию и управление физическими процессами в промышленности. Он широко применяется в Industry 4.0, включая обнаружение отказов, кибератак, машинное обучение и другие области. IAI обеспечивает аналитику в реальном времени, позволяя оценивать текущее состояние системы и прогнозировать её поведение.

SOC — это стратегия управления, первоначально разработанная для химических производств с большим числом управляющих переменных. Её цель — выбор оптимальных параметров, минимизирующих экономические издержки, с использованием многоуровневой оптимизации на различных временных масштабах (от минут до месяцев).

Существует несколько подходов к реализации SOC:

— методы поиска экстремума (extremum seeking) и отслеживание точки максимальной мощности (MPPT), применяемые при отсутствии точной информации о динамике системы;

— градиентные и безградиентные алгоритмы оптимизации в реальном времени для настройки контуров управления;

— повторяющаяся стратегия управления и итеративное обучение (repetitive control strategies, iterative learning control).

Для обеспечения стабильности системы при поиске оптимального решения важно тестировать предложенные параметры на цифровом двойнике. Это особенно актуально для процессов с медленным откликом и высокими рисками (например, термические или нефтехимические производства). DT позволяет моделировать различные сценарии, проверяя, сохраняется ли заданная производительность системы. Структура системы SOC-управления может быть представлена следующим образом (Рисунок 8).

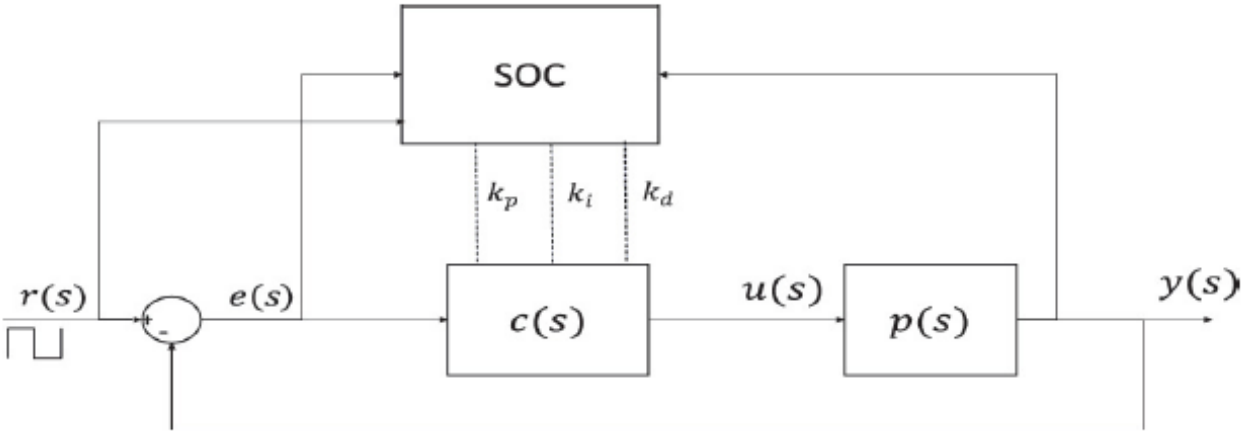

Рисунок 8 – Структурная схема SOC-управления

Верхний уровень управления отвечает за оптимизацию, принимая в качестве входных данных эталонное значение r, сигнал ошибки e и выходную переменную y. На основе этих данных вычисляются оптимальные параметры контроллера c(s), который представляет собой PID-регулятор:

$$C(s) = K_p + \frac{K_i}{s} + K_d s \quad (4)$$

Модель системы с передаточной функцией фильтра первого порядка с задержкой имеет следующий вид:

$$P(s) = \frac{K}{T_s+1} * e^{-Ls} \quad (5)$$

где K – коэффициент усиления системы, T – постоянная времени, L – запаздывание.

Основная цель SOC-контроллера определяется следующим образом:

$$min(J) = \omega_1 OV(\mu) + \omega_2 T_s(\mu) + \omega_3 \int_0^t e^2(t,\mu)dt \quad (6)$$

при ограничениях:

$$OV(\mu) < A$$

$$T_s(\mu) < B$$

$$K_{p,min} \leq K_p \leq K_{p,max}$$

$$K_{i,min} \leq K_i \leq K_{i,max}$$

$$K_{d,min} \leq K_d \leq K_{d,max}$$

где OV – процент перерегулирования, T_s – время выборки системы, $\omega_1, \omega_2, \omega_3$ – весовые коэффициенты, $\mu = K_p, K_i, K_d$ - пропорциональный, интегральный и дифференциальный коэффициенты усиления PID-регулятора, A и B соответствуют максимальному перерегулированию и времени установления. Оптимизация выполняется после каждого периода входного сигнала.

ГЛАВА 3. РАЗРАБОТКА АРХИТЕКТУРЫ ПРОГРАММНО-АППАРАТНОГО КОМПЛЕКСА ЦИФРОВЫХ ДВОЙНИКОВ

Цифровые двойники опираются на набор специализированных технологий, обеспечивающих их функциональность, начиная от сбора данных и заканчивая безопасностью.

3.1 Аппаратно-программный комплекс цифровых двойников

Микропроцессорные платформы, такие как Raspberry Pi, NVIDIA Jetson и Intel NUC, играют ключевую роль в создании цифровых двойников благодаря их многоядерной архитектуре, поддержке операционных систем общего назначения и возможности интеграции с облачными сервисами. Эти устройства представляют собой системы на кристалле (SoC), объединяющие CPU, GPU, модули памяти и интерфейсы периферии. Например, Raspberry Pi 5 оснащен 4-ядерным ARM Cortex-A76 с тактовой частотой до 2.4 ГГц и видеоядером VideoCore VII, а NVIDIA Jetson Orin предлагает 12 ядер ARM Cortex-A78AE и 2048 ядер CUDA для задач искусственного интеллекта.

В цифровых двойниках микропроцессоры применяются для обработки сигналов с датчиков, управления исполнительными устройствами и реализации алгоритмов машинного обучения на периферии (edge AI). Для фильтрации шумов и частотного анализа применяются библиотеки SciPy (Python) или FFTW (C++), а управление двигателями реализуется через ШИМ-сигналы с использованием GPIO.

Микропроцессоры представляют собой многоядерные системы на кристалле (SoC), объединяющие CPU, GPU, модули памяти и периферийные интерфейсы. Главными элементами микопроцессорных устройств является:

— CPU: Многоядерные архитектуры (ARM Cortex-A, x86) для многопоточной обработки.

— GPU: Ускорители для параллельных вычислений (NVIDIA CUDA, Mali).

Периферия:

— Интерфейсы: USB, Ethernet, HDMI, GPIO, PCIe.

— Беспроводная связь: Wi-Fi 6, Bluetooth 5.0, 5G/LTE (в модульных решениях).

Программная часть микропроцессорных систем включает операционные системы (Linux, Windows IoT), фреймворки для edge-вычислений (AWS IoT Greengrass, EdgeX Foundry) и инструменты моделирования (MATLAB Coder). Например, Linux-дистрибутивы на базе Ubuntu Core позволяют развертывать Docker-контейнеры для изолированных задач, а ROS2 обеспечивает связь между узлами цифрового двойника.

Микропроцессорные платформы играют ключевую роль в создании цифровых двойников (ЦД), обеспечивая баланс между вычислительной мощностью, гибкостью и доступностью. В отличие от микроконтроллеров (МК) и программируемых логических схем (FPGA), микропроцессоры (МП) ориентированы на сложные алгоритмы, интеграцию с облачными сервисами и работу с операционными системами общего назначения. Рассмотрим их особенности, применение в ЦД и актуальные решения (Таблица 3).

Таблица 3 – Сравнение микропроцессоров для цифровых двойников

Параметр	Raspberry Pi 5	NVIDIA Jetson Orin	Intel NUC 13
Производительность	До 24 GFLOPS (CPU)	200 TOPS (AI)	До 1.3 TFLOPS (GPU)
Энергопотребление	10–15 Вт	15–50 Вт	25–65 Вт
Архитектура	ARM Cortex-A76	ARM Cortex-A78AE	x86 (Core i7)
ОЗУ	До 8 Гб	До 16Гб	До 32 Гб
Цена	6 800–8 500 ₽	42 500–170 000 ₽	51 000–127 500 ₽
Применение	Прототипирование, IoT	Автономные роботы, AI	Промышленные ЦД, HMI

Ключевым преимуществом микропроцессоров является их гибкость: они поддерживают как низкоуровневое программирование на C/C++, так и высокоуровневые языки (Python, Java). Однако выбор платформы зависит от требований проекта. Raspberry Pi подходит для бюджетных решений, Jetson

Orin — для задач с ИИ, а Intel NUC — для интеграции с промышленными контроллерами через Modbus TCP или OPC UA.

Микроконтроллеры (МК) являются базовыми компонентами сенсорного уровня цифрового двойника (ЦД), обеспечивающими сбор и первичную обработку данных. В зависимости от требований к производительности, энергоэффективности и стоимости выделяют следующие варианты реализации, представленные в таблице 4.

Следует отметить, что различный аппаратный состав и характеристики имеют не только микроконтроллеры разных семейств, но и модели (версии) одного семейства. Однако общим для большинства моделей контроллеров является:

- RISC – архитектура (с двухступенчатым конвейером), обеспечивающая выполнение большинства команд процессора за один машинный цикл,
- гарвардская архитектура (с раздельными шинами данных и программ), обеспечивающая одновременный доступ к памяти данных и программ.

Микроконтроллеры служат основным звеном нижнего уровня в цифровых двойниках, обеспечивая сбор данных, управление исполнительными устройствами и выполнение алгоритмов реального времени. Их ключевые преимущества — низкая стоимость, энергоэффективность и компактность. Однако для сложных задач (например, машинное обучение) требуется интеграция с микропроцессорами. Примером микроконтроллера может служить микросхема ATMega на рисунке 9.

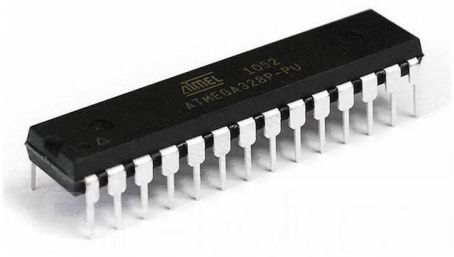

Рисунок 9 – Микроконтроллер ATMega

Таблица 4 - Основные различия микроконтроллеров AVR, ARM и PIC

	PIC	AVR	ARM
Разрядность	8/16/32 бит	8/32 бит	32 бит, 64 бит
Интерфейсы	PIC, UART, USART, LIN, CAN, Ethernet, SPI, I2S	UART, USART, SPI, I2C, иногда CAN, USB, Ethernet	UART, USART, LIN, I2C, SPI, CAN, USB, Ethernet, I2S, DSP, SAI, IrDA
Скорость	4 такта на инструкцию	1 такт на инструкцию	1 такт на инструкцию
Память	SRAM, FLASH	Flash, SRAM, EEPROM	Flash, SDRAM, EEPROM
Шинная архитектура	Частично RISC	RISC	RISC
Архитектура памяти	Гарвардская	Модифицированная	Модифицированная гарвардская
Энергопотребление	Низкое	Низкое	Низкое
Семейства	PIC16,PIC17, PIC18, PIC24, PIC32	Tiny, Atmega, Xmega, спец. AVR	ARMv4,5,6,7
Производители	Microchip	Atmel (Microchip)	Apple, Nvidia, Qualcomm, Samsung Electronics, TI ...
Стоимость	Средняя	Средняя	Низкая
Популярные микроконтроллеры	PIC18fXX8, PIC16f88X, PIC32MXX	Atmega8, 16, 32; вариации для Arduino	LPC2148, ARM Cortex-M0, ARM Cortex-M3, ARM Cortex-M7

Беспроводная передача данных между микроконтроллерами может быть организована несколькими способами, рассмотрим каждый из них и выделим их особенности:

а) Bluetooth/BLE - беспроводной стандарт связи используемый для обмена данными на коротком расстоянии. Bluetooth может применяется во множестве ситуаций, где требуется беспроводная передача данных, можно выделить его основные функции:

— коммуникация между периферийными устройствами и ПК,

— передача данных между Bluetooth-устройствами.

Другими словами, технология Bluetooth используется в ситуациях, когда для передачи данных между устройствами требуется непрерывное сквозное («точка-точка») подключение. Структурная схема передачи данных при помощи технологии Bluetooth представлена на рисунке 10.

Рисунок 10 – Связь устройств при использовании технологии Bluetooth

В отличие от Bluetooth, который включён постоянно, BLE-устройство постоянно находится в спящем режиме, кроме ситуаций, когда оно подключено к другим устройствам. Благодаря этому BLE-устройства потребляют меньше электроэнергии.

Из-за этого стандарт BLE подходит для проектов, где требуется периодический обмен небольшими порциями данных – например, в медицине, фитнесе, отслеживании объектов, навигации, безопасности и домашней автоматизации.

В стандарте Bluetooth Low Energy предусмотрено два типа устройств: сервер и клиент. Сервер оповещает о своём существовании, чтобы его могли найти другие устройства, а также содержит данные для считывания клиентом. Клиент сканирует близлежащие устройства и, найдя искомый сервер, подключается к нему и начинает передавать ему данные.

Стандарт BLE предусматривает и другие типы подключения:

— режим вещания, когда сервер передаёт данные нескольким клиентам, подключённым к нему.

— ячеистая сеть, где все устройства подключены друг к другу.

Сравнение технических особенностей Bluetooth и BLE представлено в таблице 5.

Таблица 5 – Различия между Bluetooth и BLE

	Bluetooh Low Energy (BLE)	Bluetooh: Базовый режим (Base Rate, BR)/ Дополнительный режим (Enhanced Date Rate, EDR)
Оптимизирован для:	Передача данных “короткими очередями”	Постоянный поток данных
Диапазон частот	ISM-диапазон 2.4 ГГц	ISM-диапазон 2.4 ГГц
Каналы	40 каналов с диапазоном 2 МГц (3 канала для оповещений, 37 каналов для данных)	79 каналов с диапазоном 1 МГц
Скорость передачи данных	- LE2M PHY: 2 Мбит/с - LE1M PHY: 1 Мбит/с - LE Coded PHY (S=2): 500 Кбит/с - LE Coded PHY (s=8): 125 Кбит/с	- EDR PHY (8DPSK): 3 Мбит/с - EDR PHY (п/4 DQPSK): 2 Мбит/с - BR PHY (GFSK): 1 Мбит/с
Макс. мощность	- Класс 1: 100 мВТ (+20 дБм) - Класс 1.5: 10 мВт (+10 дБм) - Класс 2: 2.5 мВт (+4 дБм) - Класс 3: 1 мВт (0 дБм)	- Класс 1: 100 мВт (+20 дБм) - Класс 2: 2.5 мВт (+4 дм) - Класс 3: 1 мВт (0 дБм)
Сетевая топология	- “Точка-точка” - Вещание - Ячеистая сеть	“Точка-точка”

b) Wi-fi соединение на основе модуля ESP8266. Параметры модуля ESP8266:

— напряжение питания 3,3 В,
— ток до 210 мА в режиме передачи,
— ток до 60 мА во время приема,
— 802.11 b/g/n протокол,
— SDIO,
— режимы сохранения энергии и сна для экономии энергии,
— встроенный микроконтроллер,
— управление через AT-команды,
— температура функционирования от -40 до +125 градусов Цельсия,
— максимальная дистанция связи 100 метров.

c) Использование радиочастотных модулей без поддержки протоколов. Приемник XY-MK-5V и передатчик XY-FST. Данные модули работают на частоте 430 МГц. Важно отметить, что оба модуля должны быть настроены на одну и ту же частоту, в противном случае они не будут работать друг с другом.

Данные модули представляют простое схемное решение суперрегенеративных приемников заданной частоты, предназначенные для приема (передачи) цифрового сигнала. Передатчик имеет три вывода - два по питанию и один для данных. Приемник также имеет два вывода по питанию и два вывода для приема данных микроконтроллером, эти два вывода данных фактически являются одним выводом, и впаяны параллельно друг другу. Таким образом, если на вывод данных передатчика подать логическую единицу, на выводе данных приемника также появится логическая единица. Такие модули являются радиочастотными удлинителями одного вывода микроконтроллера, заменяющими провод.

Рассмотрим параметры модулей приемника и передатчика:

Приемник:

— напряжение питания 5 В,

— потребляемый ток 4 мА,

— частота 430 МГц,

— чувствительность 110 дБ,

Передатчик:

— расстояние передачи от 20 до 200 метров в зависимости от напряжения питания и условий окружающей среды,

— напряжение питания от 3,5 до 12 В,

— скорость передачи до 4 кб/с,

— мощность передатчика 10 мВт,

— частота 430 МГц,

Протокол MQTT (Message Queuing Telemetry Transport) играет ключевую роль в передаче данных между физическими объектами и цифровыми моделями. Этот легковесный протокол, работающий по модели «издатель-подписчик», оптимизирован для IoT-устройств с ограниченными ресурсами и низкой пропускной способностью каналов. В рамках цифрового двойника MQTT обеспечивает доставку данных с датчиков (температура, давление, вибрация) через брокера, который распределяет сообщения между подписчиками, такими как аналитические модули или облачные платформы. Важной особенностью MQTT является поддержка уровней качества обслуживания (QoS 0, 1, 2), гарантирующих надежность доставки сообщений даже в условиях нестабильной сети. Система связи, построенная на MQTT, состоит из сервера-издателя, сервера-брокера и одного или нескольких клиентов. Издатель не требует каких-либо настроек по количеству или расположению подписчиков, получающих сообщения. Кроме того, подписчикам не требуется настройка на конкретного издателя. В системе может быть несколько брокеров, распространяющих сообщения.

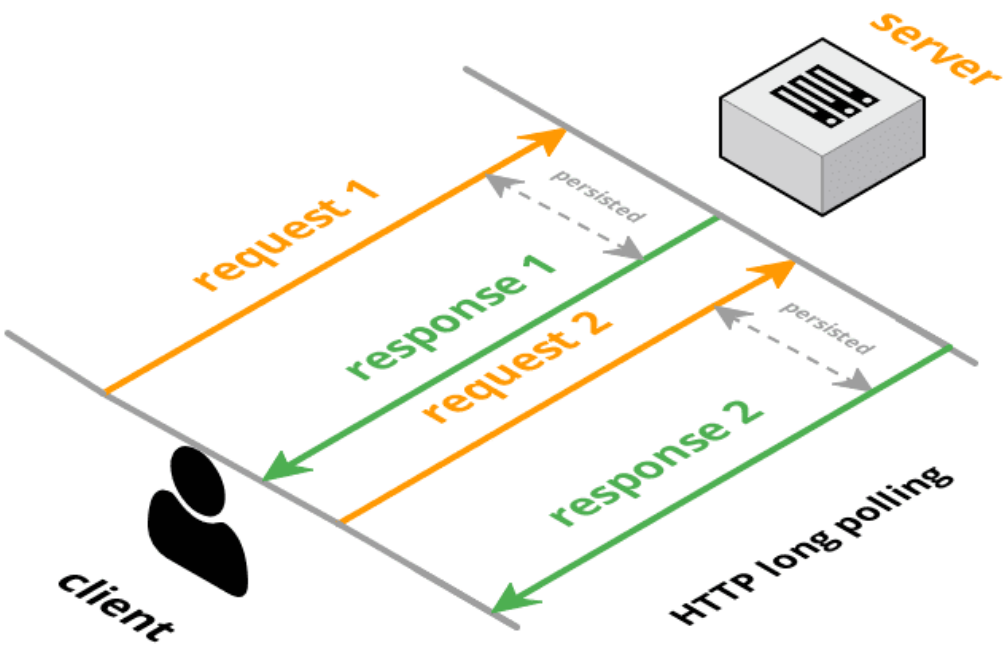

Рисунок 11 – Пример связи на основе MQTT

MQTT предоставляет способ создания иерархии каналов связи — своего рода ветвь с листьями. Всякий раз, когда у издателя есть новые данные для распространения среди клиентов, сообщение сопровождается примечанием контроля доставки. Клиенты более высокого уровня могут получать каждое сообщение, в то время как клиенты более низкого уровня могут получать сообщения, относящиеся только к одному или двум базовым каналам, «ответвляющимся» в нижней части иерархии (Рисунок 11). Это облегчает обмен информацией размером от двух байт до 256 мегабайт.

MQTT обладает следующими свойствами:

— нейтрален к содержимому сообщения;

— идеально подходит для распределённых коммуникаций «один ко многим» и разъединённых приложений;

— оснащён функцией lwt (last will and testament, «последняя воля и завещание») для уведомления сторон об аномальном отключении клиента;

— полагается на tcp/ip для базовых задач связи;

— разработан для доставки сообщений по шаблонам «максимум один раз», «минимум один раз» и «ровно один раз».

Для интеграции цифровых двойников с внешними системами и бизнес-приложениями используется REST API (Representational State Transfer. Клиент-серверная архитектура, лежащая в основе REST API, представляет собой модель взаимодействия между компонентами распределённой системы, где клиент и сервер функционируют как независимые сущности, разделённые по принципу ответственности. Клиент инициирует запросы к серверу, который обрабатывает их и возвращает соответствующие ответы, обеспечивая тем самым декомпозицию функций и повышение модульности системы. Данная архитектура базируется на строгом разделении обязанностей: клиентская часть отвечает за пользовательский интерфейс и логику представления данных, тогда как серверная сосредоточена на обработке запросов, управлении ресурсами и обеспечении бизнес-логики. Взаимодействие между клиентом и сервером осуществляется через стандартизированные протоколы, преимущественно HTTP, что гарантирует совместимость и независимость от конкретных технологических стеков.

REST (Representational State Transfer), введённый Роем Филдингом в 2000 году как архитектурный стиль для распределённых гипермедиа-систем, формализует набор ограничений, направленных на создание масштабируемых, производительных и простых в поддержке веб-сервисов. Основополагающим принципом REST является единообразие интерфейса, которое достигается за счёт стандартизации методов взаимодействия, использования уникальных идентификаторов ресурсов (URI) и представления данных в форматах, отделённых от их внутренней структуры (JSON, XML). Сервер предоставляет ресурсы в виде репрезентаций, а клиент манипулирует этими ресурсами через операции, соответствующие семантике HTTP-методов: GET (получение), POST (создание), PUT (полное обновление), PATCH (частичное обновление), DELETE (удаление). Каждый запрос клиента содержит всю необходимую информацию для его обработки, что соответствует требованию отсутствия состояния (statelessness) на сервере — ключевому аспекту масштабируемости и отказоустойчивости. Примером

реализации данной архитектуре может служить представленная на рисунке 12 структурная схема системы получения информации о температуре с термометра. В данном случае клиент посылает запрос серверу о получении данных с термометра, сервер в свою очередь, используя собственные алгоритмы, получает требуемые данные и отправляет ответ клиенту, при этом вся процедура запрос-ответ кэшируется в базу данных для возможного анализа и визуализации работы системы.

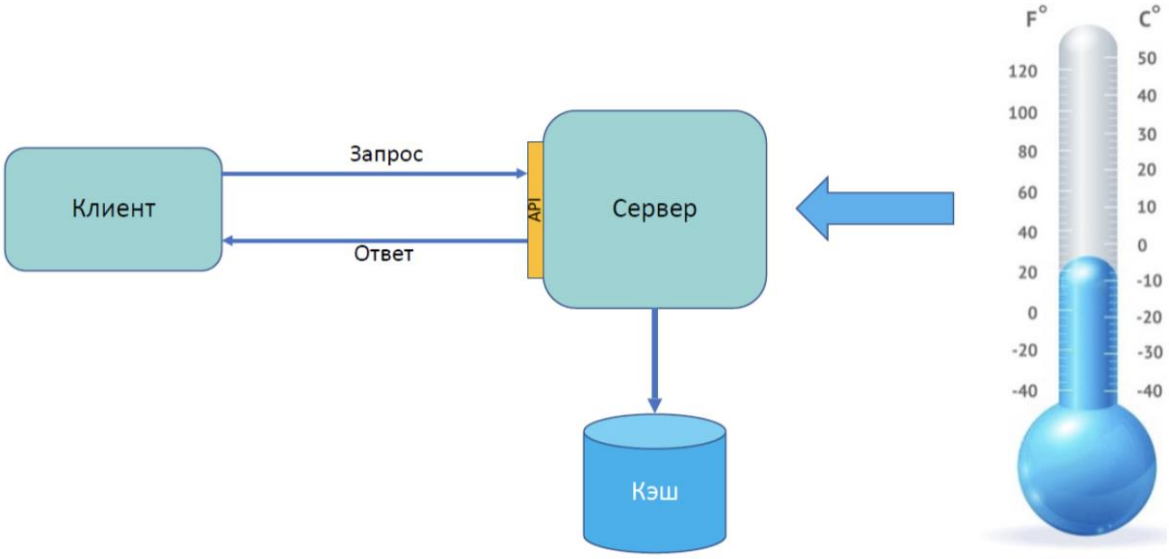

Рисунок 12 – Пример реализации архитектуры REST API

Технология BLE-Beacon, основанная на стандарте Bluetooth Low Energy (BLE), представляет собой решение для реализации протоколов ближней локализации и контекстно-зависимых сервисов, функционирующее через периодическую трансляцию идентификационных пакетов в радиодиапазоне. BLE-Beacon выступает в роли пассивного передатчика, излучающего сигналы с определённой периодичностью, которые принимаются совместимыми устройствами (смартфонами, датчиками) для определения относительного положения или активации предопределённых действий. Архитектура системы строится на взаимодействии трёх компонентов: маяка (beacon) — аппаратного устройства с BLE-модулем, клиентского приложения, интерпретирующего сигналы, и серверной

платформы, управляющей логикой взаимодействия и данными. Основной функциональной задачей BLE-Beacon является передача уникального идентификатора (UUID), дополненного значениями major и minor, позволяющими сегментировать устройства по группам и подкатегориям, что обеспечивает гибкость в сценариях развёртывания.

Технология маяков BLE имеет ряд преимуществ по сравнению с другими RF (радиочастота) технологии, такие как маяки Wi-Fi, Зигби-маяки, RFID-маяки, Маяки LoRa, NFC-маяки, и маяки LTE. В таблице 6 сравниваются эти радиочастотные технологии на основе различных параметров.

Таблица 6 – Радиочастотные технологии

	Точность	Диапазон	Задержка	Энергопотребление
BLE-маяк	Высокая	10-100м	Низкий(100мс+)	Низкий
Wi-Fi маяк	Высокая	30-100м	Низкий(100мс+)	Высокая
Зигби-маяк	Средний	10-100м	Низкий(100мс+)	Низкий
RFID маяк	Низкий	До 100 м	Низкий(PC)	Низкий
Маяк лора	Высокая	До 15 км	Высокая(секунд)	Очень низкий
NFC-маяк	Средняя	<20 см	Низкий(PC)	Очень низкий
Маяк LTE	Высокая	До 30 км	Средний(PC)	Высокая

Принцип работы BLE-Beacon базируется на механизме advertising — непрерывной рассылке пакетов данных в эфир без установления соединения. Это отличает его от классического Bluetooth, требующего pairing, и минимизирует энергопотребление, позволяя устройствам функционировать на батарейном питании до нескольких лет. Структура рекламного пакета включает служебные данные (header), идентификатор производителя (Manufacturer Specific Data) и полезную нагрузку (payload), содержание которой варьируется в зависимости от протокола (iBeacon, Eddystone, AltBeacon). Например, протокол iBeacon, разработанный Apple, использует

UUID длиной 128 бит, тогда как Eddystone от Google поддерживает передачу URL-адресов (Eddystone-URL) и телеметрии (Eddystone-TLM).

Примером применения данной технологии может служить структурная схема подключения BLE-маяков и передача данных на облачный сервер представленная на рисунке 13.

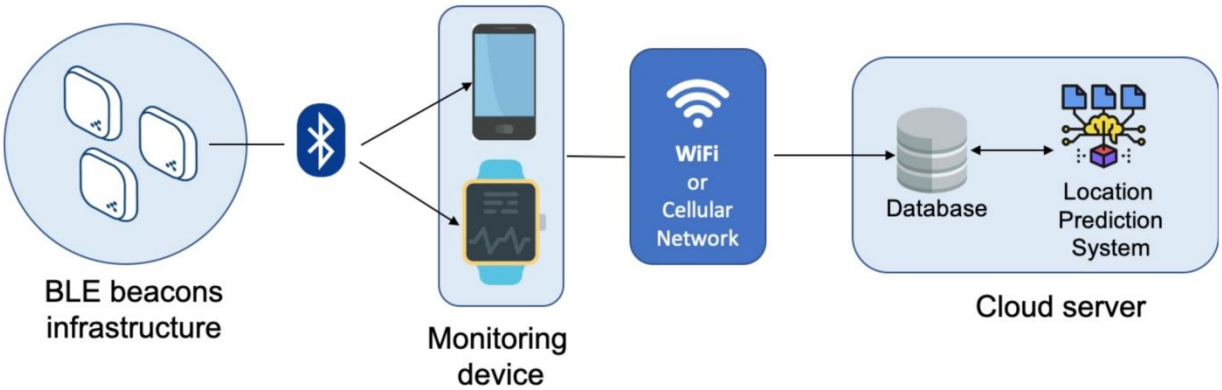

Рисунок 13 – Пример реализации передачи данных на сервер с BLE-маяков

Применение BLE-Beacon в умных городах и розничной торговле иллюстрирует их адаптивность к разнородным сценариям. В музеях маяки используются для автоматической активации аудиогидов при приближении к экспонатам, в логистике — для отслеживания перемещения грузов в режиме реального времени, а в ритейле — для персонализированных уведомлений о скидках. В здравоохранении технология внедряется для мониторинга пациентов и контроля доступа к оборудованию, однако нормативные требования к конфиденциальности данных (например, HIPAA в США) требуют усиления мер защиты.

Перспективы развития BLE-Beacon связаны с интеграцией в гибридные системы позиционирования, сочетающие BLE с ультраширокополосной связью (UWB), датчиками LiDAR или компьютерным зрением. Это позволит повысить точность локализации до сантиметрового уровня и нивелировать ограничения, вызванные многолучевым распространением радиоволн. Кроме того, внедрение энергоавтономных маяков с питанием от энергии окружающей среды (energy harvesting) устранит зависимость от батарей, расширив область применения до труднодоступных мест.

3.2 Модель предметной области на основе онтологий и семантических сетей

Модель предметной области цифрового двойника учебной лаборатории, основанная на онтологиях и семантических сетях, формализует структуру, взаимодействие и семантику компонентов системы через декомпозицию на уровни абстракции и установление отношений между сущностями. Онтология системы строится на иерархии классов, свойств и аксиом, описывающих функциональные модули, потоки данных и правила их интеграции, что обеспечивает машиночитаемую интерпретацию предметной области для автоматизированного анализа и логического вывода.

Основные звенья модели (Рисунок 14) включают IoT, клиентское приложение на Unity, микросервис на REST API и сервер с базой данных. Каждое из звеньев выполняет свою уникальную функцию, обеспечивая взаимодействие между физическим пространством и цифровой моделью.

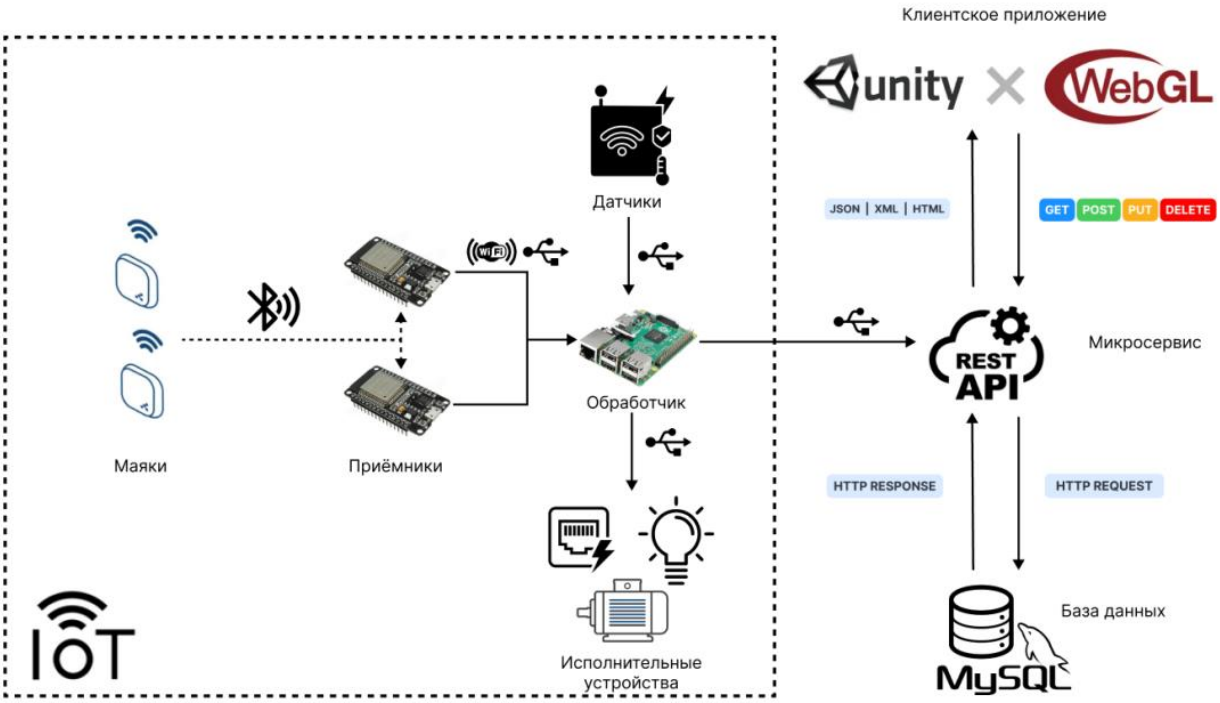

Рисунок 14 - Концептуальная модель цифрового двойника

Слой IoT репрезентируется как совокупность физических и виртуальных объектов, включая датчики (класс Sensor), исполнительные устройства (Actuator), микроконтроллеры (Microcontroller) и микропроцессорное устройство (MicroprocessorUnit). Каждый экземпляр класса Sensor характеризуется свойствами hasMeasurementType (температура, влажность, движение), hasAccuracy, hasSamplingRate и отношением connectedTo, указывающим на микроконтроллер, агрегирующий данные. Исполнительные устройства (Actuator) определяются через hasActionType (включение/выключение, регулировка мощности) и hasActuationRange, связанные с управляющими командами от микропроцессорного устройства. Микроконтроллеры и микропроцессорное устройство образуют класс Controller с подклассами, дифференцированными по вычислительной мощности (hasComputeCapacity) и роли в архитектуре: микроконтроллеры (Microcontroller) выступают как периферийные узлы обработки, а MicroprocessorUnit — как центральный координатор, реализующий локальную логику (правила реакции на события, фильтрацию шума).

Слой микросервисов моделируется как набор независимых сервисов (Microservice), взаимодействующих через REST API. Каждый микросервис инкапсулирует доменную функцию: DeviceManagementService (регистрация устройств, обновление прошивок), DataIngestionService (приём и валидация данных с IoT), AnalyticsService (агрегация, прогнозная аналитика). Семантика API описывается через класс APIEndpoint с свойствами hasHTTPMethod (GET, POST), hasPath (например, /api/devices/{id}/status), expectsPayloadSchema (JSON Schema) и returnsResponseType (application/json). Отношение exposes связывает микросервис с его эндпоинтами, а dependsOn отражает зависимость между сервисами (например, AnalyticsService зависит от DataIngestionService). Аутентификация и авторизация формализуются классом SecurityPolicy с подклассами OAuth2Scope, RateLimitingRule, связанными с микросервисами через отношение enforces.

Слой базы данных онтологически разделяется на подсистемы хранения: TimeSeriesDatabase для данных с датчиков (метки времени, значения), RelationalDatabase для метаданных устройств и пользователей (конфигурации, роли), CacheDatabase для временных данных (сессии, результаты запросов). Класс DatabaseTable связывается с микросервисами через отношение managedBy, определяя, какой сервис отвечает за CRUD-операции. Свойства hasIndexingStrategy (B-деревья, хэш-индексы) и hasReplicationFactor (уровень отказоустойчивости) уточняют архитектурные параметры. Для поддержки семантической совместимости вводится класс DataMappingRule, описывающий трансформации между форматами данных IoT-устройств (SensorDataFormat) и структурами таблиц (TableSchema).

Слой визуализации на Unity онтологически представлен как совокупность 3D-моделей (Lab3DModel), интерфейсов (DashboardUI) и механизмов рендеринга (RenderingEngine). Класс Lab3DModel агрегирует компоненты RoomLayout (пространственная структура лаборатории), DeviceRepresentation (3D-объекты, соответствующие физическим устройствам) и AnimationController (анимация состояний устройств). Отношение bindsTo связывает 3D-модели устройств с экземплярами классов Sensor и Actuator из слоя IoT, обеспечивая синхронизацию состояний. Интерфейсы (DashboardUI) включают RealTimeMetricsPanel (графики данных), ControlPanel (кнопки управления) и AlertSystem (оповещения), связанные с микросервисами через отношение consumesDataFrom. Механизмы рендеринга (RenderingEngine) описываются свойствами hasFrameRate, supportsVR (интеграция с VR-шлемами) и optimizesForPlatform (десктоп, мобильные устройства).

Семантическая сеть системы устанавливает связи между слоями через предикаты, отражающие поток данных и управления:

— Sensor → sendsDataTo → DataIngestionService → storesIn → TimeSeriesDatabase;

— AnalyticsService → generatesInsightsFor → DashboardUI;

— ControlPanel → triggersCommand → Actuator через DeviceManagementService;

— MicroprocessorUnit → validates → SensorData перед отправкой в DataIngestionService.

Таксономия событий (SystemEvent) включает DeviceFailureEvent (авария датчика), DataAnomalyEvent (отклонение показаний), UserInteractionEvent (действие пользователя в интерфейсе). Каждое событие связывается с правилами обработки (EventProcessingRule), определяющими, какой микросервис (Microservice) инициирует реакцию (например, отправку уведомления в AlertSystem или перезапуск устройства через DeviceManagementService).

Онтология расширяется аксиомами, обеспечивающими логическую целостность:

— если устройство (device) имеет статус inactive, то связанные с ним 3d-модели (devicerepresentation) должны отображаться с анимацией disabledstate;

— данные от sensor со значением accuracy < threshold должны маркироваться как unreliable и игнорироваться в analyticsservice;

— пользователь (user) с ролью student не может отправлять команды (controlcommand) к actuator без одобрения user с ролью professor.

Динамическая семантика системы формализуется через машины состояний (StateMachine), где переходы между состояниями (например, calibration → operational для датчика) инициируются событиями и обрабатываются микросервисами. Визуализация в Unity отражает эти

состояния через изменения в Shader (цветовая кодировка) и AnimationController, обеспечивая интуитивную обратную связь.

Примером упрощённой модели предметной области данной разработки может служить модель предметной области микросервиса представленная на рисунке 15.

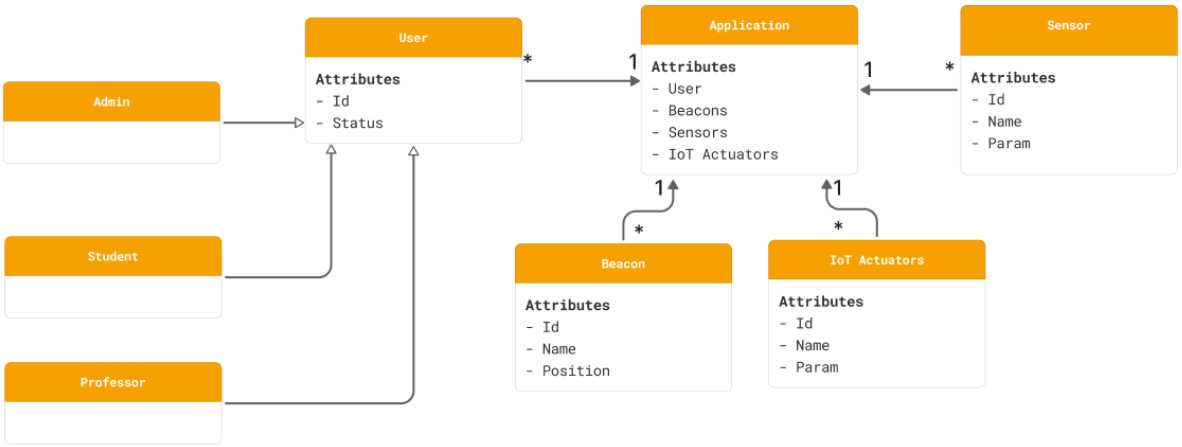

Рисунок 15 – Упрощённая модель предметной области микросервиса

Интеграция семантических сетей и онтологий позволяет реализовать контекстно-зависимую логику: например, автоматическую корректировку визуализации при изменении конфигурации лаборатории или адаптацию алгоритмов аналитики к типу подключаемых датчиков. Использование стандартов вроде OWL (Web Ontology Language) и RDF (Resource Description Framework) обеспечивает интероперабельность модели с внешними системами управления образовательным процессом (расписания, системы оценки), образуя связный цифровой экосистемный контур.

Онтологическая модель цифрового двойника учебной лаборатории обеспечивает не только структурное представление компонентов, но и когнитивную основу для автоматизированного принятия решений, семантического поиска и адаптации системы к изменениям в физической инфраструктуре или педагогических требованиях.

3.3 Структура клиентского программного обеспечения

Структура клиентского программного обеспечения, ответственного за визуализацию процессов и данных цифрового двойника учебной лаборатории, проектируется как многоуровневая система, интегрирующая механизмы рендеринга, управления состоянием, взаимодействия с API и семантической интерпретации данных. Архитектура клиента разделяется на модули, обеспечивающие синхронизацию между физическими сущностями IoT, бизнес-логикой микросервисов и динамическим представлением в Unity, с опорой на онтологическую модель для контекстно-зависимого отображения информации.

Модуль взаимодействия с данными (Data Acquisition & Processing) выступает промежуточным звеном между REST API микросервисов и визуализацией. Он реализует подписку на эндпоинты, предоставляющие метрики датчиков (/api/sensors/telemetry), состояния устройств (/api/devices/status) и события системы (/api/events). Для обработки потоковых данных используется WebSocket или технология Server-Sent Events (SSE), обеспечивающая низкую задержку при обновлении интерфейса. Полученные данные трансформируются в формат, совместимый с онтологией: значения датчиков сопоставляются с экземплярами классов Sensor и Actuator, а метаданные (например, геолокация устройств) — с 3D-моделями в сцене Unity. Механизм нормализации данных применяет правила из DataMappingRule для конвертации единиц измерения, фильтрации шума и агрегации показателей (средние значения, пиковые нагрузки).

Модуль 3D-визуализации, реализованный на движке Unity, структурирован вокруг сценарной логики, объединяющей статические и динамические элементы. Сцена лаборатории строится на основе класса Lab3DModel, включающего:

— Пространственную модель помещения (RoomLayout), импортируемую из CAD-файлов или созданную procedural generation с учётом параметров реальной лаборатории (размеры, расположение оборудования);

— Интерактивные 3D-объекты устройств (DeviceRepresentation), связанные через отношение bindsTo с экземплярами IoT-устройств. Каждый объект содержит компоненты для визуализации состояния: материалы с динамическими шейдерами (изменение цвета при перегреве, аварии), анимационные контроллеры (AnimationController) для имитации движения (вращение вентиляторов, открытие/закрытие замков);

— Систему частиц и эффектов (Partical System), отражающую физические процессы: дым при перегрузке устройства, свечение индикаторов, динамические тени от перемещающихся объектов.

Модуль пользовательского интерфейса (UI) разделяется на два контура. Иммерсивный интерфейс, интегрированный в 3D-сцену: контекстные панели, появляющиеся при наведении на устройства; голографические подсказки, проецируемые в пространстве; элементы управления (ползунки, кнопки), активируемые через raycasting (взаимодействие курсора или VR-контроллера с объектами). 2D-дашборды (DashboardUI), отображаемые поверх 3D-сцены или на внешних устройствах: графики временных рядов (RealTimeMetricsPanel), тепловые карты загрузки оборудования, таблицы журнала событий (EventLog). Данные дашбордов обновляются через подписку на микросервис AnalyticsService, с возможностью фильтрации по времени, типу устройства или критичности событий.

Модуль обработки событий (Event Handling) обеспечивает реакцию на изменения в системе. Пользовательские события (клики, голосовые команды, жесты в VR) транслируются в REST-запросы к DeviceManagementService или ControlService, с предварительной валидацией прав доступа через SecurityPolicy. Например, команда на отключение датчика требует подтверждения роли professor. Системные события (DataAnomalyEvent,

DeviceFailureEvent) активируют сценарии визуализации: мигание аварийных индикаторов, запуск анимации устранения неисправности, автоматический фокус камеры на проблемной зоне. Правила обработки (EventProcessingRule) импортируются из онтологии, гарантируя соответствие логике цифрового двойника.

Модуль конфигурации и персонализации позволяет адаптировать интерфейс под требования пользователей. Ролевые профили (Student, Professor, Administrator) определяют доступные функции: студенты видят только показатели датчиков, преподаватели — элементы управления, администраторы — системные метрики (нагрузка серверов, журналы аудита). Настройки визуализации: выбор схемы цветокодировки (например, температура от синего к красному), уровень детализации 3D-моделей (Low-Poly для слабых устройств), режимы отображения (упрощённый, расширенный).

Клиентское приложение, представленное на рисунке 16, предоставляет визуализацию цифрового двойника учебной аудитории. Основные функции приложения:

— визуализация данных: отображение информации о состоянии аудитории в реальном времени (температура, присутствие студентов и т.д.);

— интерактивность: пользователи могут взаимодействовать с моделью, получая доступ к различным функциям управления (например, регулировка освещения).

— симуляция сценариев: приложение может проводить симуляцию различных сценариев процессов обучения, что помогает в принятии управленческих решений.

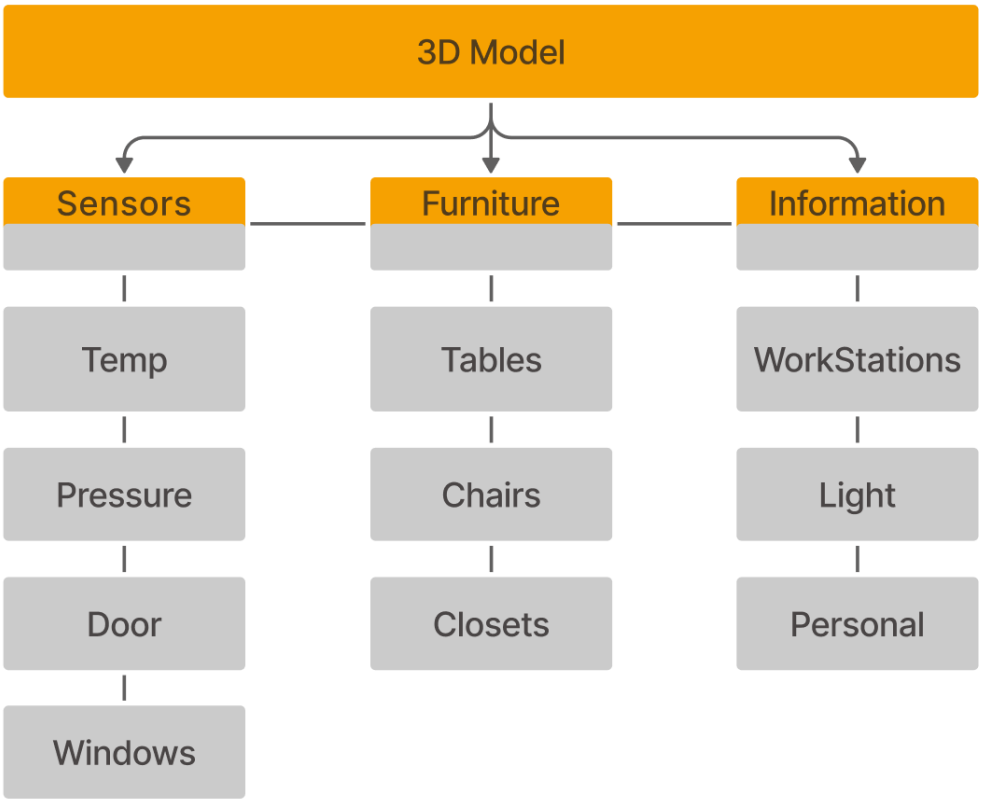

Рисунок 16 – Структура клиентского программного обеспечения

Интеграция с внешними системами расширяет функциональность:

— экспорт данных в образовательные lms (moodle, canvas) через стандарты xapi (tin can api), фиксируя действия студентов в цифровом двойнике как элементы учебной аналитики.

— синхронизация с расписанием занятий: автоматическая активация сценариев визуализации, соответствующих текущей лабораторной работе, на основе данных из календарных сервисов (google calendar, microsoft outlook).

Структура клиентского ПО проектируется с учётом принципов реактивности и декомпозиции, где каждый модуль функционирует как независимый агент, обменивающийся сообщениями через шину событий (Event Bus). Это позволяет масштабировать систему, добавляя новые типы визуализаций (например, AR-оверлеи через ARCore/ARKit) или интеграции с AI-сервисами для прогнозирования аномалий без переписывания базовой логики. Семантическая связность с онтологической моделью обеспечивает консистентность данных и возможность автоматического рефакторинга интерфейса при изменении требований к цифровому двойнику.

ГЛАВА 4. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ЦИФРОВОГО ДВОЙНИКА

Архитектура цифрового двойника учебной лаборатории, основанная на распределённых технологиях и семантической интеграции компонентов, требует применения специализированных программных инструментов, обеспечивающих разработку, развёртывание и поддержку многоуровневой системы. Каждый слой инфраструктуры — IoT, микросервисы, база данных, визуализация — взаимодействует через стандартизированные интерфейсы, что диктует необходимость выбора средств, поддерживающих кросс-платформенность и масштабируемость.

В контексте учебных лабораторий университетов цифровой двойник выступает как связующее звено между материальной инфраструктурой (датчики, оборудование) и цифровыми сервисами, предоставляя студентам и преподавателям инструменты для удалённого взаимодействия, анализа данных и визуализации сложных физических явлений.

Актуальность разработки обусловлена ростом спроса на гибридные формы обучения, где сочетание очных и дистанционных методик требует создания универсальных платформ, способных отражать состояние лабораторных установок в режиме 24/7. Пандемия COVID-19 ускорила этот процесс, продемонстрировав критическую важность систем, обеспечивающих непрерывность образовательного процесса вне зависимости от географических и временных ограничений. Цифровой двойник учебной лаборатории не только решает эти задачи, но и открывает возможности для:

— персонализации обучения через адаптацию сценариев работы под уровень подготовки студентов;

— повышения безопасности за счёт симуляции критических режимов оборудования без риска повреждения физических устройств;

— оптимизации ресурсов путём анализа данных об использовании приборов и прогнозирования их износа.

Архитектурная сложность системы заключается в необходимости интеграции разнородных компонентов:

1. IoT-устройства (датчики, микроконтроллеры), собирающие данные с физических объектов;
2. Распределённые микросервисы, обрабатывающие информацию и реализующие бизнес-логику;
3. Гетерогенные базы данных, хранящие временные ряды, метаданные и семантические описания;
4. Интерактивный слой визуализации, обеспечивающий иммерсивное взаимодействие с цифровым двойником.

Онтологический подход к проектированию такой системы позволяет формализовать знания о предметной области через машиночитаемые модели, описывающие:

— иерархию сущностей (оборудование, пользователи, события);

— семантические связи между ними (влияние датчика на исполнительный механизм, ролевой доступ к данным);

— правила логического вывода (автоматическая генерация уведомлений при аномалиях, адаптация интерфейса под контекст задачи).

Методология разработки опирается на принципы DevOps и MLOps, интегрируя непрерывную интеграцию (CI/CD), автоматизированное тестирование и мониторинг в жизненный цикл системы. Это позволяет оперативно адаптировать цифровой двойник к изменениям в физической инфраструктуре лаборатории или педагогических требованиях.

Последующие разделы детализируют архитектурные решения, инструментальную базу и практические аспекты реализации каждого компонента системы, раскрывая взаимосвязи между онтологическими моделями, программными сервисами и пользовательским опытом.

4.1 Технология программной реализации цифрового двойника

Инструменты разработки встроенного ПО для IoT-устройств включают платформы, оптимизированные для работы с микроконтроллерами и микропроцессорами. PlatformIO, интегрируемый в VS Code, предоставляет унифицированную среду для написания, отладки и прошивки кода на языках C/C++ и Python, с поддержкой библиотек для работы с периферийными устройствами (датчики температуры, гироскопы, реле). Его ключевым преимуществом является совместимость с широким спектром аппаратных платформ (ESP32, STM32, Arduino), а также встроенные инструменты статического анализа кода и профилирования энергопотребления. Arduino IDE, несмотря на ограниченную функциональность для сложных проектов, остаётся актуальной для быстрого прототипирования благодаря упрощённому интерфейсу и обширному сообществу готовых скетчей.

PlatformIO представляет собой кросс-платформенную экосистему, ориентированную на профессиональную разработку встроенного ПО. Её основное назначение — обеспечение унифицированной среды для написания, отладки и развёртывания кода на широком спектре микроконтроллеров и микропроцессоров, включая популярные платформы ESP32, STM32, Arduino и Raspberry Pi. PlatformIO интегрируется в редактор кода Visual Studio Code, предоставляя доступ к расширенным функциям, таким как статический анализ кода, автоматическое управление зависимостями и профилирование энергопотребления. Инструмент поддерживает более 50 платформ и фреймворков, включая Arduino, ESP-IDF, FreeRTOS и Zephyr, что позволяет разработчикам гибко выбирать стек технологий под конкретные требования проекта. Для цифрового двойника учебной лаборатории PlatformIO используется для реализации сложной логики обработки данных с датчиков (например, фильтрации шума, агрегации показаний температуры или влажности), управления исполнительными устройствами (реле, сервоприводы) и интеграции с облачными IoT-платформами, такими как AWS IoT Core или Azure IoT Hub,

через протоколы MQTT и HTTP. Преимущества PlatformIO включают кросс-платформенность, поддержку CI/CD-пайплайнов через GitHub Actions, а также возможность расширения функционала за счёт плагинов, таких как PlatformIO Home для удалённого управления устройствами (Рисунки 17-18). Однако инструмент имеет высокий порог входа из-за необходимости настройки для нестандартных аппаратных конфигураций и требует глубокого понимания архитектуры целевых устройств.

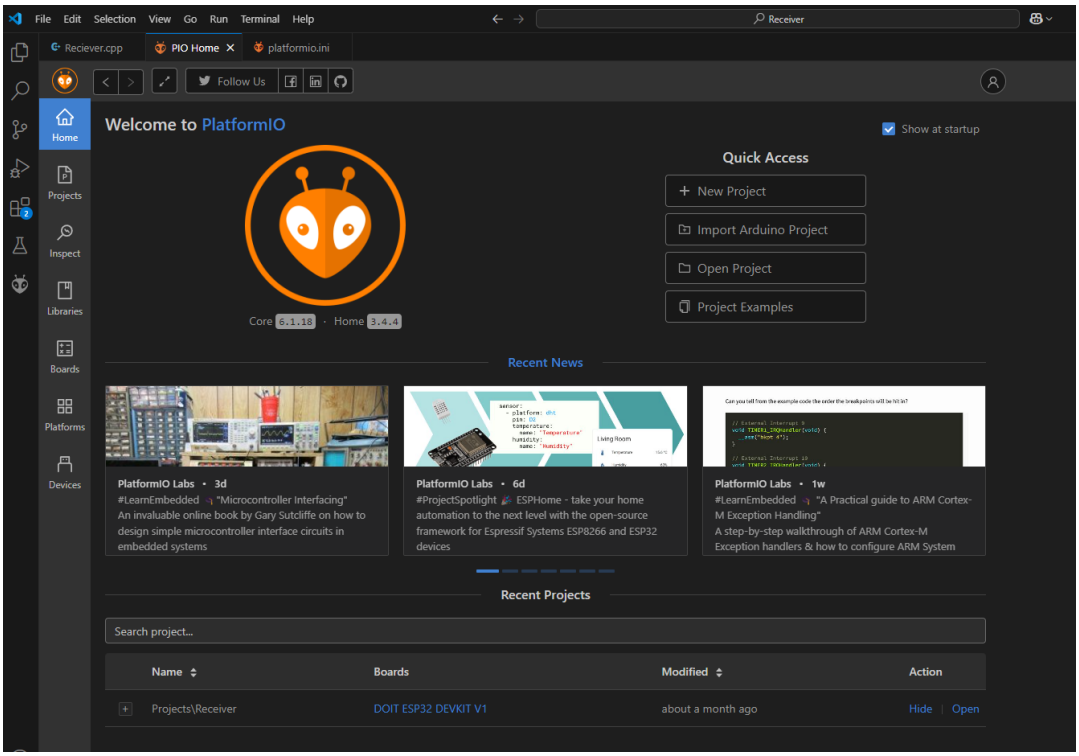

Рисунок 17 – Пример внешнего вида плагина Platform IO в среде VSCode

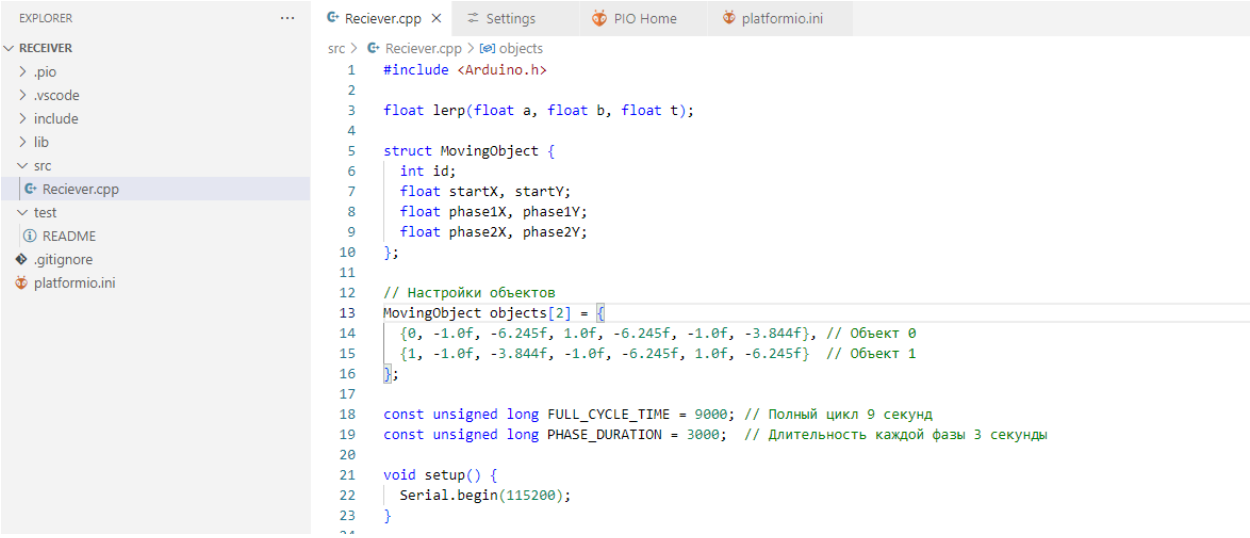

Рисунок 18 – Пример внешнего вида среды разработки VSCode

Arduino — торговая марка аппаратно - программных средств, для построения простых систем автоматики и робототехники, ориентированная на непрофессиональных пользователей. Программная часть состоит из бесплатной программной оболочки (IDE) для написания программ, их компиляции и программирования аппаратуры. Аппаратная часть представляет собой набор смонтированных печатных плат, продающихся как официальным производителем, так и сторонними производителями. Полностью открытая архитектура системы позволяет свободно копировать или дополнять линейку продукции Arduino.

Под торговой маркой Arduino выпускается несколько плат с микроконтроллером и платы расширения (так называемые «шилды»). Большинство плат с микроконтроллером снабжены минимально необходимым набором обвязки для нормальной работы микроконтроллера (стабилизатор питания, кварцевый резонатор, цепочки сброса и т. п.).

Микроконтроллеры для Arduino отличаются наличием предварительно прошитого в них загрузчика (bootloader). С помощью этого загрузчика пользователь загружает свою программу в микроконтроллер без использования традиционных отдельных аппаратных программаторов.

Программирование ведется целиком через собственную программную оболочку (IDE), бесплатно доступную на сайте Arduino. В этой оболочке имеется текстовый редактор, менеджер проектов, препроцессор, компилятор и инструменты для загрузки программы в микроконтроллер. Оболочка написана на Java на основе проекта Processing, разработана под такие операционный системы, как Windows, Mac OS X и Linux. Оболочка программы изображена на рисунке 19.

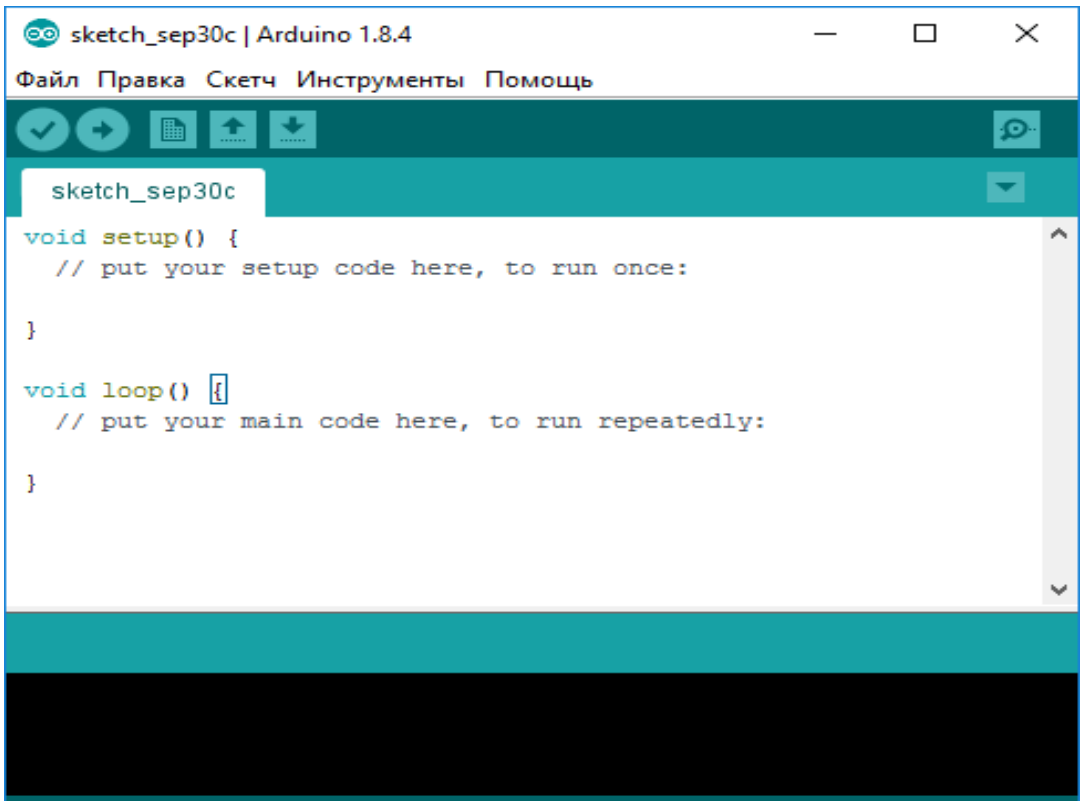

Рисунок 19 – Пример внешнего вида среды разработки Arduino IDE

Интеграция Arduino IDE с PlatformIO позволяет преодолеть часть этих ограничений. Проекты, созданные в Arduino IDE, могут быть импортированы в PlatformIO для подключения профессиональных инструментов отладки (например, JTAG/SWD-адаптеров) и настройки CI/CD-процессов. Это сочетание обеспечивает плавный переход от учебных прототипов к промышленным решениям, что особенно важно в контексте цифрового двойника, где требуется масштабируемость и надёжность. Например, студенты могут начать с разработки простого датчика температуры на Arduino, отладить его в Arduino IDE, а затем перенести код в PlatformIO для оптимизации энергопотребления, добавления шифрования данных и интеграции с REST API микросервисов.

Оба инструмента играют критическую роль в создании IoT-слоя цифрового двойника. PlatformIO обеспечивает промышленную надёжность и гибкость, необходимые для работы с гетерогенными устройствами и облачными сервисами, в то время как Arduino IDE упрощает образовательный процесс, позволяя быстро визуализировать результаты

экспериментов. Совместное использование этих сред позволяет балансировать между требованиями производительности, безопасности и доступности, формируя основу для устойчивой IoT-инфраструктуры учебной лаборатории.

В рамках разработки микросервисов для цифрового двойника учебной лаборатории ключевыми инструментами являются Visual Studio, C#, .NET (включая .NET Framework и .NET Core) и Postman. Их интеграция обеспечивает создание надёжных, масштабируемых REST API, взаимодействующих с IoT-устройствами, базами данных и слоем визуализации.

Visual Studio выступает основной интегрированной средой разработки (IDE) для проектирования, отладки и развёртывания микросервисов на C# и .NET. Она предоставляет шаблоны проектов для ASP.NET Core, включая Web API, gRPC и Blazor, а также интеграцию с Entity Framework Core для объектно-реляционного отображения (ORM). Инструменты отладки, такие как точки останова, пошаговое выполнение и профилирование производительности (анализ использования CPU и HTTP-трафика), упрощают выявление узких мест. Интеграция с облачной платформой Azure позволяет публиковать микросервисы напрямую в Azure App Service или Kubernetes, управляя ресурсами вроде Cosmos DB или Service Bus через Azure Explorer. Расширения, такие как Swagger UI для интерактивной документации API и ReSharper для рефакторинга кода, расширяют функциональность среды. Например, в проекте цифрового двойника Visual Studio используется для создания REST API, управляющего IoT-устройствами, и сервиса аналитики, обрабатывающего данные с датчиков в реальном времени.

Язык C# с его строгой типизацией и поддержкой асинхронности (через async/await) идеально подходит для разработки высоконагруженных микросервисов. Использование LINQ (Language Integrated Query) упрощает работу с коллекциями и базами данных, а паттерны проектирования, такие

как внедрение зависимостей (Dependency Injection) через IServiceCollection, обеспечивают модульность и тестируемость кода. Например, DTO (Data Transfer Objects) применяются для сериализации данных API, что гарантирует согласованность форматов.

Платформа .NET предлагает два основных направления: .NET Framework для legacy-приложений Windows с ограниченной кроссплатформенностью и .NET 6+/7+ (ранее .NET Core), который поддерживает Linux, macOS и Windows, обеспечивает высокую производительность благодаря нативной AOT-компиляции и оптимизирован для контейнеризации. Пример внешнего вида среды представлен на рисунке 20.

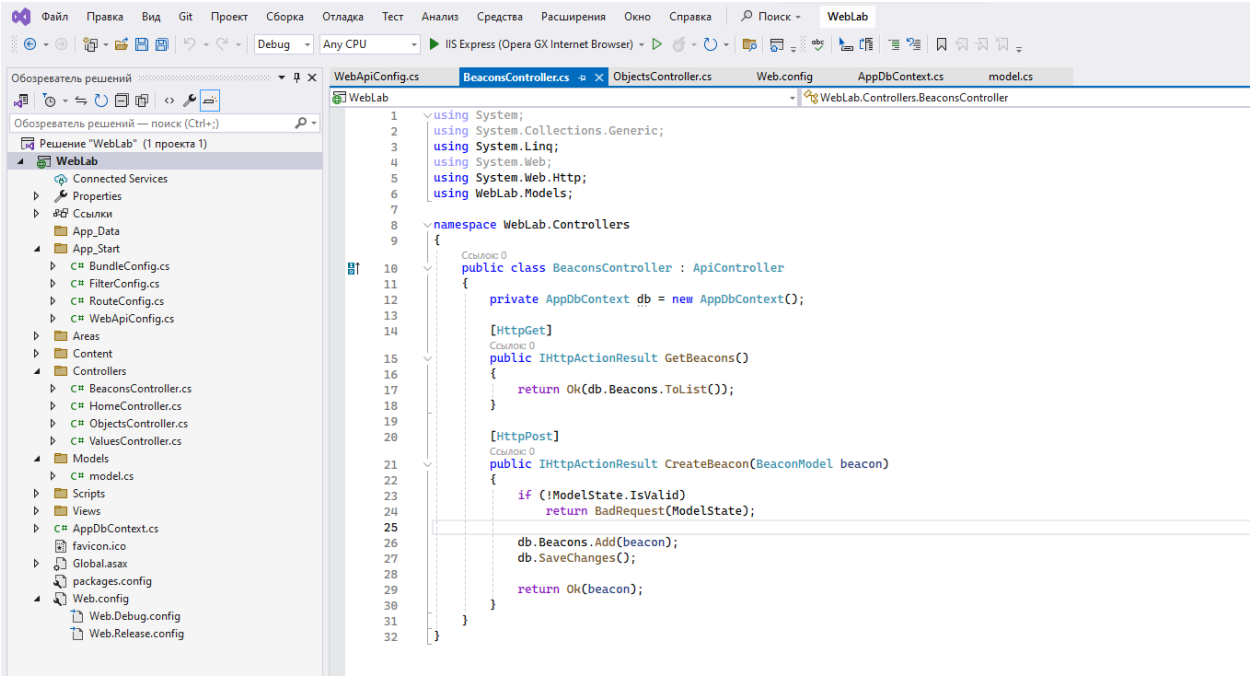

Рисунок 20 - Пример внешнего вида среды Visual Studio

Здесь контроллер обрабатывает HTTP-запросы, используя атрибуты маршрутизации ([HttpGet], [HttpPost]), а внедрение зависимостей обеспечивает связь с сервисом бизнес-логики. Middleware, такой как аутентификация через JWT Bearer Tokens (UseAuthentication) и глобальная обработка ошибок (UseExceptionHandler), добавляется в конвейер запросов. Сериализация данных в JSON выполняется через System.Text.Json с настройкой camelCase для имён полей.

Интеграция с IoT-слоем реализуется через получение данных от шлюзов по HTTP или через брокеры сообщений (например, RabbitMQ). Фоновые задачи, такие как агрегация данных каждые 5 минут, выполняются с помощью BackgroundService. Документирование API осуществляется через Swagger/OpenAPI с использованием библиотеки Swashbuckle.AspNetCore, которая автоматически генерирует спецификации и интерактивную документацию.

Postman играет ключевую роль в тестировании и автоматизации REST API. С его помощью выполняются ручные проверки эндпоинтов: отправка GET-запросов к /api/devices/{id} для получения информации об устройстве или POST-запросов для добавления новых устройств.

Коллекции Postman (Рисунок 21) параметризуются переменными окружения (URL, токены). Синхронизация с Swagger упрощает поддержку актуальной документации: OpenAPI-спецификации импортируются в Postman для создания коллекций, а изменения в коллекциях экспортируются обратно в Swagger.

Взаимодействие инструментов в проекте начинается с разработки в Visual Studio: создание проекта ASP.NET Core, настройка DI-контейнера и интеграция с Entity Framework Core для работы с MySQL. После локального тестирования API через Postman выполняется деплой в Docker-контейнерах с настройкой обратного прокси через Nginx. Мониторинг метрик производительности (Prometheus) и визуализация в Grafana завершают цикл, обеспечивая надёжность системы.

Лучшие практики включают версионирование API через URL (/api/v1/devices) или заголовки (Accept: application/vnd.lab.v1+json), аутентификацию через JWT с ограниченным временем жизни и ролевой моделью доступа (RBAC), а также оптимизацию через кэширование ответов (ResponseCacheAttribute) и пагинацию данных. Глобальный обработчик

исключений (IExceptionHandler) и кастомные коды ошибок (например, 450 для ошибок калибровки датчиков) улучшают обработку сбоев.

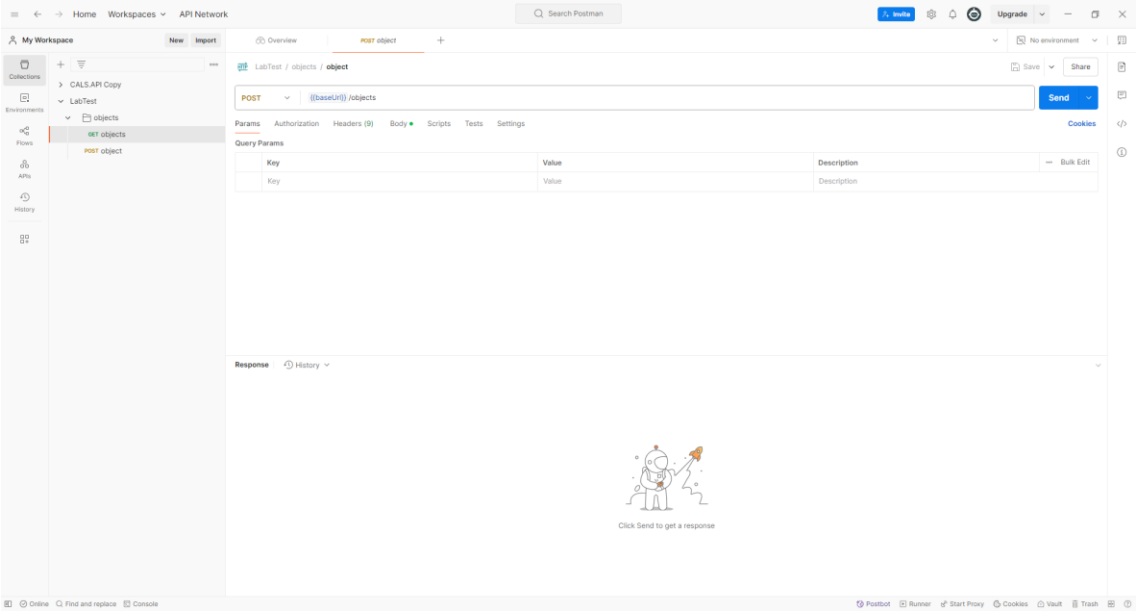

Рисунок 21 - Пример внешнего вида среды Postman

MySQL, как реляционная система управления базами данных (СУБД) с открытым исходным кодом, играет ключевую роль в хранении структурированных данных, обеспечивая целостность, масштабируемость и эффективный доступ к информации. В контексте цифрового двойника учебной лаборатории MySQL используется для управления метаданными IoT-устройств, пользовательскими профилями, журналами событий и конфигурационными параметрами системы, что делает её центральным компонентом слоя данных.

Архитектурные особенности MySQL, такие как поддержка ACID-транзакций (Atomicity, Consistency, Isolation, Durability), механизмы репликации (master-slave, group replication) и шардинга, обеспечивают надёжность и отказоустойчивость. Репликация позволяет синхронизировать данные между основным и резервными серверами, минимизируя downtime при сбоях, а шардинг (горизонтальное разделение данных) повышает производительность за счёт распределения нагрузки между узлами. Для учебной лаборатории, где количество устройств и пользователей может

динамически расти, это гарантирует стабильную работу даже при пиковых нагрузках.

Интеграция MySQL с микросервисами на .NET реализуется через Entity Framework Core (EF Core) — объектно-реляционный маппер (ORM), который абстрагирует взаимодействие с базой данных, преобразуя операции на C# в SQL-запросы. Модели данных описываются классами, а миграции автоматически генерируют схему БД.

Мониторинг и анализ производительности осуществляются через инструменты вроде MySQL Workbench (Рисунок 22) и Prometheus с экспортером MySQL. Метрики (количество подключений, скорость выполнения запросов) визуализируются в Grafana, что позволяет выявлять аномалии, такие как блокировки из-за долгих транзакций.

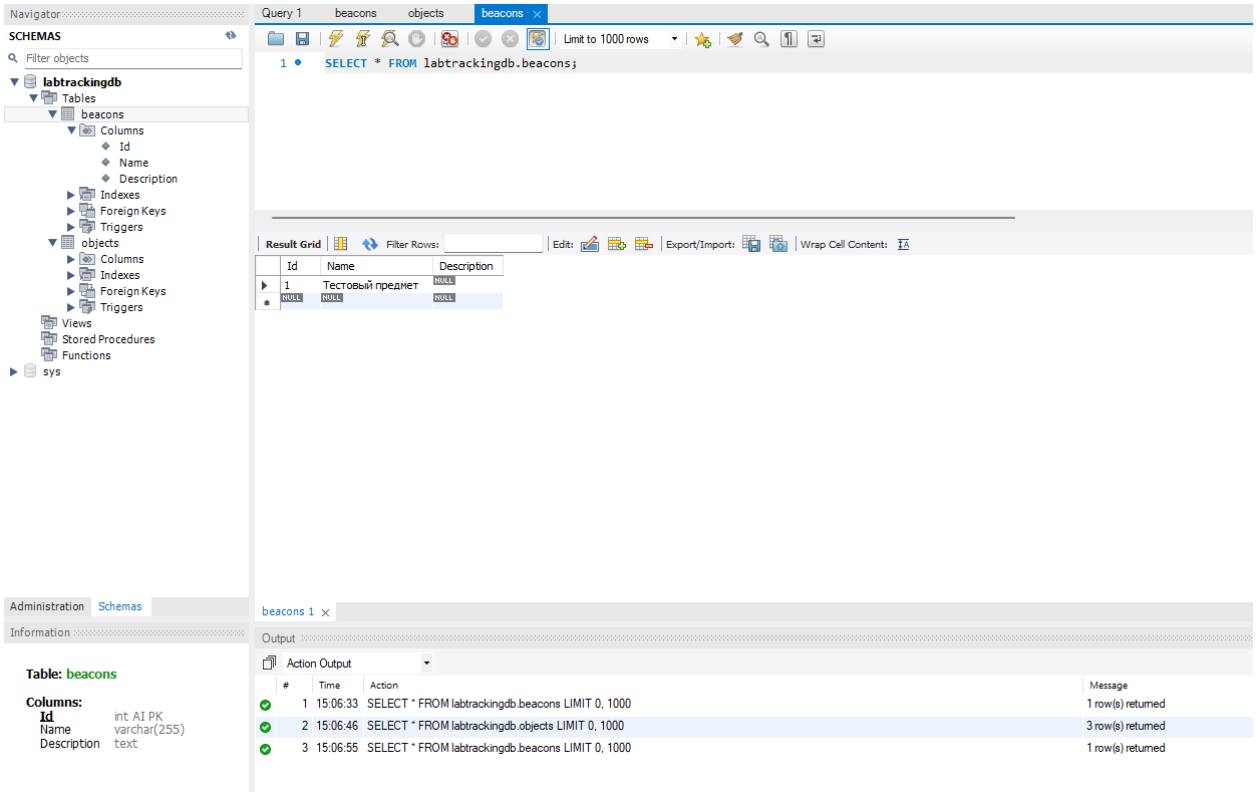

Рисунок 22 - Пример внешнего вида среды MySQL Workbench

Unity как инструмент разработки цифрового двойника учебной лаборатории представляет собой многофункциональный движок, объединяющий возможности 3D-визуализации, физической симуляции и интеграции с внешними системами. Его архитектура строится на компонентно-ориентированном подходе: каждый объект в сцене (например, модель микроскопа или датчика) состоит из компонентов, определяющих его поведение. Например, скрипт на C# может управлять анимацией прибора при получении данных о перегреве через REST API, изменяя цвет материала с синего на красный через Color.Lerp, в то время как система частиц (Particle System) визуализирует дым, имитируя аварию. Для рендеринга используются различные пайплайны: High-Definition Render Pipeline (HDRP) обеспечивает фотореалистичное освещение и тени, что важно для детализированных демонстраций, а Universal Render Pipeline (URP) оптимизирован под мобильные устройства и WebGL, сохраняя баланс между качеством и производительностью.

Интеграция с внешними системами реализуется через REST API с использованием класса UnityWebRequest, который отправляет асинхронные HTTP-запросы к микросервисам, получая данные о состоянии устройств. Например, корутина UpdateTemperatureFromAPI периодически запрашивает температуру датчика и обновляет цвет модели через UpdateMaterialColor. Для работы в реальном времени подключаются WebSocket-библиотеки (например, WebSocket-Sharp), позволяющие получать мгновенные уведомления о событиях, таких как срабатывание пожарной сигнализации.

Взаимодействие с IoT-устройствами осуществляется через MQTT-клиенты, подписывающиеся на топики вроде lab/devices/temperature, что обеспечивает синхронизацию между физическим оборудованием и его цифровым двойником.

Интеграция Unity (Рисунок 23) с WebGL представляет собой процесс адаптации 3D-проектов под запуск в браузерах через технологии WebAssembly и WebGL, что обеспечивает кросс-платформенность без необходимости установки плагинов. При компиляции Unity преобразует C#-код в WebAssembly с помощью IL2CPP, что позволяет достичь высокой производительности, но накладывает ряд ограничений: например, отсутствие поддержки многопоточности из-за особенностей JavaScript, что критично для асинхронных операций и параллельных вычислений. Рендеринг графики в WebGL базируется на OpenGL ES 2.0/3.0, что ограничивает использование расширенных шейдеров, тесселяции или постобработки, требуя упрощения материалов и оптимизации полигональных сеток. Для аудио используется Web Audio API, поддерживающий базовые функции воспроизведения, но не сложные эффекты в реальном времени, что вынуждает разработчиков минимизировать аудиокомпоненты или использовать предрассчитанные семплы. Сетевое взаимодействие в WebGL-сборках также имеет особенности: браузеры блокируют прямой доступ к сокетам, поэтому для обмена данными применяются WebSocket или HTTP-запросы, а безопасность соединений обеспечивается через CORS-политики, требующие корректной настройки серверов.

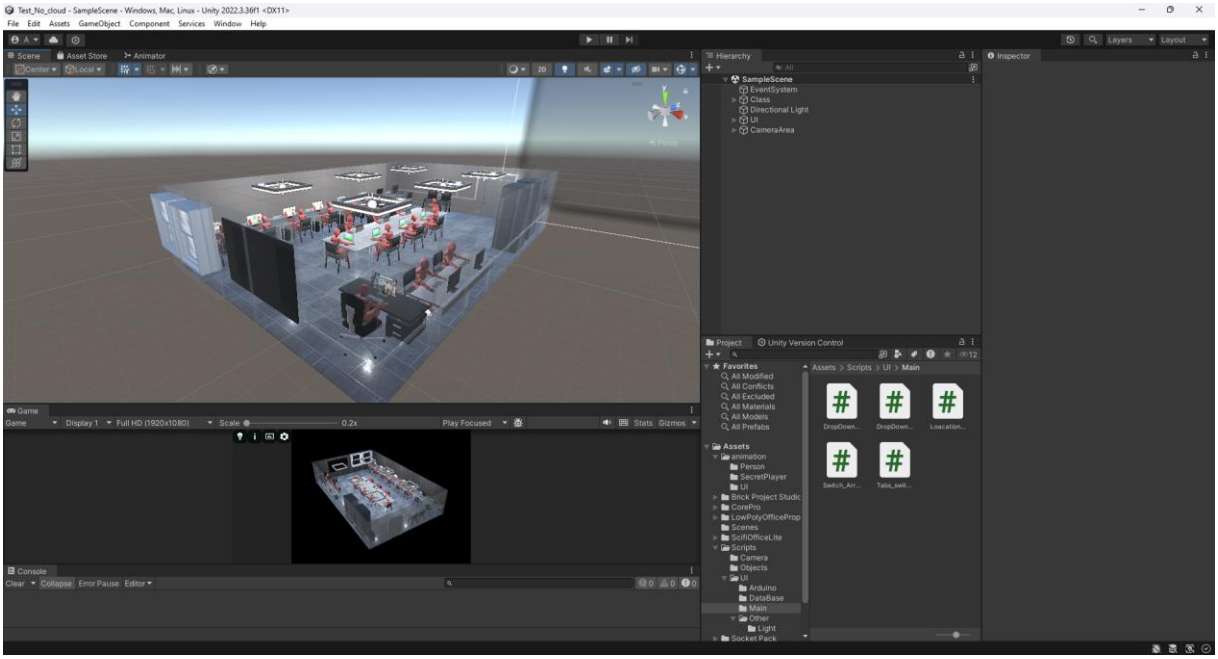

Рисунок 23 - Пример внешнего вида среды Unity 3D

Для работы с пользовательским вводом (мышь, клавиатура, сенсорные экраны) Unity предоставляет абстрактный Input System (Рисунок 24), который необходимо настраивать под специфику браузеров, учитывая задержки и ограничения мобильных устройств.

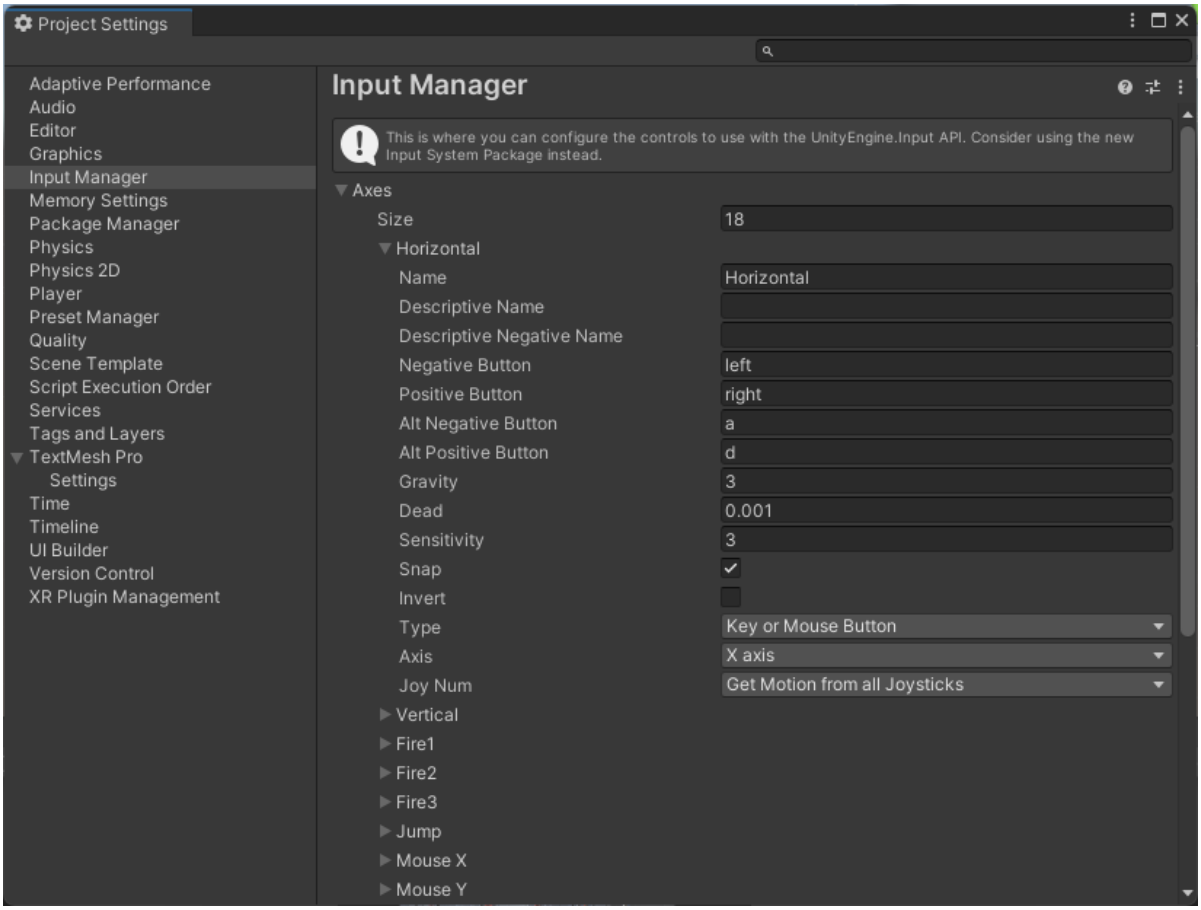

Рисунок 24 - Пример Input System в Unity 3D

Отладка WebGL-сборок выполняется через консоль браузера, где выводятся логи из Debug.Log, но обработка исключений усложнена из-за особенностей WebAssembly — ошибки часто оборачиваются в общие сообщения, что требует тщательного тестирования и использования кастомных обработчиков. На рисунке 25 представлена возможность интеграции приложения разработанного в среде Unity в целевую среду, которыми выступают как операционные системы для ПК, так и для мобильных устройств или браузерное приложение.

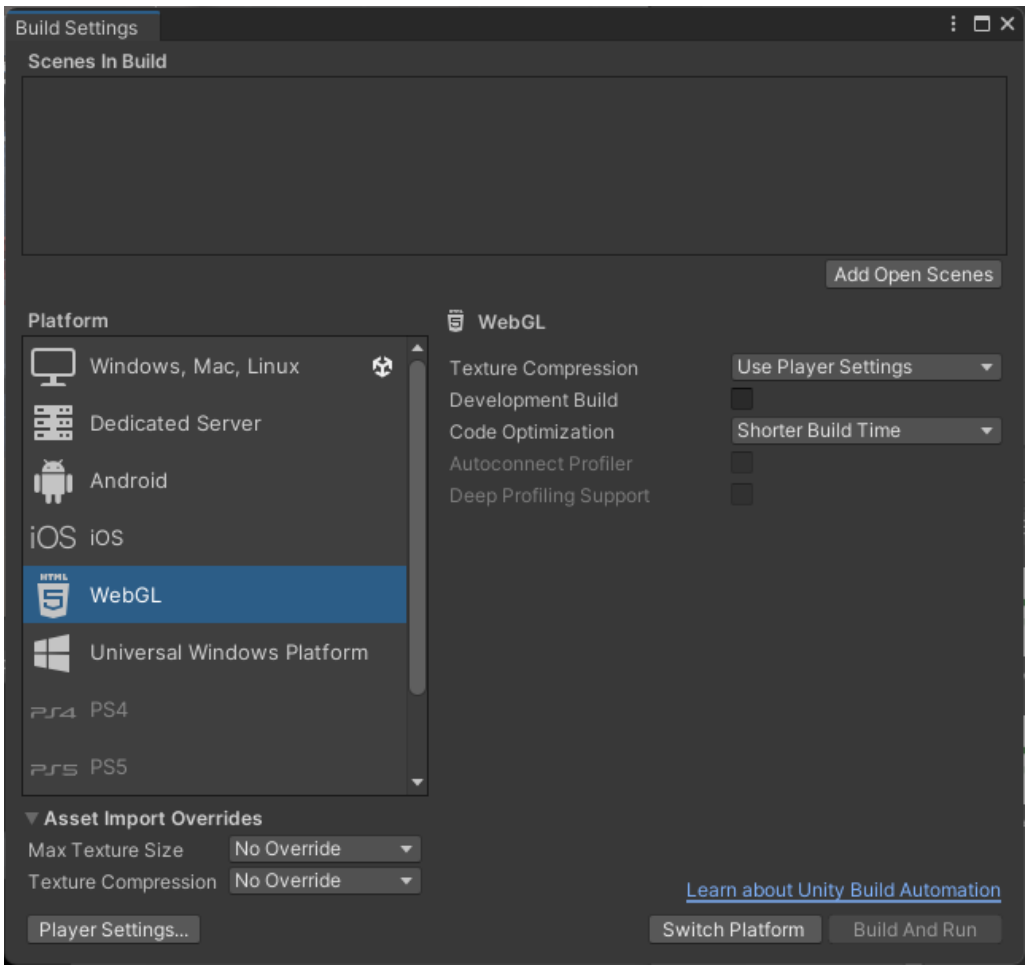

Рисунок 25 - Пример экспорта проекта в приложение под целевую среду

Для образовательных сценариев, где важна доступность, можно реализовать адаптивные интерфейсы с переключением между 3D- и 2D-режимами в зависимости от устройства. Безопасность WebGL-сборок также требует внимания: минификация кода, обфускация критических алгоритмов и вынос чувствительной логики на серверную сторону снижают риски обратной инженерии. В итоге, интеграция Unity с WebGL — это баланс между функциональностью и ограничениями веб-платформы, где ключевыми факторами успеха становятся оптимизация ресурсов, адаптация к специфике браузеров и тщательное тестирование на всех этапах разработки.

4.2 Вычислительный эксперимент по оценке производительности цифрового двойника

Поэтапная реализация разработки цифрового двойника учебной лаборатории начинается с проектирования системы позиционирования на базе BLE-маяков и ESP32. На первом этапе создаётся сеть из четырёх приёмников на ESP32, размещённых в углах лаборатории для обеспечения максимального покрытия. Каждый приёмник настраивается в среде PlatformIO с использованием стека Bluetooth Low Energy (BLE), где основная задача — сканирование эфира и захват рекламных пакетов от BLE-маяков. Программа на ESP32 включает фильтрацию RSSI (Received Signal Strength Indicator) для устранения шумов, вызванных отражениями сигнала от стен и оборудования, а также временную синхронизацию приёмников через NTP-сервер для минимизации расхождений в данных. Собранные показания (MAC-адрес маяка, RSSI, временная метка) передаются по Wi-Fi на центральный ESP32-обработчик, выступающий координационным узлом. Пример программы для ESP32 и вывод данных на экран представлен на рисунке 26.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -65, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -65, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -65, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -65, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -76, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -68, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -58, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -46, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -43, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -45, Status: OK
[2] MAC: d0:b1:f7:50:e6:85, RSSI: -41, Status: OK
```

Рисунок 26 - Пример вывода данных beacon

На втором этапе ESP32-обработчик агрегирует данные со всех четырёх приёмников, применяя алгоритм триангуляции для вычисления координат BLE-маяка в 3D-пространстве лаборатории. Для этого используется модифицированный метод наименьших квадратов, учитывающий калибровочные коэффициенты, полученные в ходе предварительного этапа картографирования помещения. Калибровка предполагает измерение эталонных значений RSSI на известных расстояниях от приёмников, что позволяет построить модель затухания сигнала с учётом специфики среды (наличие металлических конструкций, Wi-Fi-помех).

```
=== Данные приёмников ===
ID | MAC Beacon        | RSSI  | Актуальность
-----------------------------------------------
 1 | d0:b1:f7:50:e6:85 |   -42 | OK
 2 | d0:b1:f7:50:e6:85 |   -57 | OK
 3 | d0:b1:f7:50:e6:85 |   -29 | OK
 4 | d0:b1:f7:50:e6:85 |   -56 | OK
```

Рисунок 27 - Пример программы и вывода данных beacon

На этапе агрегации данных центральный ESP32 объединяет показания от всех четырёх приёмников, группируя их по идентификаторам BLE-маяков. Для преобразования RSSI в расстояние используется логарифмическая модель затухания сигнала:

$$d = 10^{\frac{(A-RSSI)}{10*n}} \quad (7)$$

где A — RSSI на расстоянии 1 метр (калибруется экспериментально), а n — коэффициент затухания (зависит от среды, например, 2–4 для открытого пространства). Предварительная калибровка системы проводится путём замера RSSI на известных дистанциях от каждого приёмника, что позволяет скорректировать A и n с учётом помех от металлических конструкций и Wi-Fi-сетей.

Для определения координат объекта применяется трилатерация — метод, использующий геометрию окружностей. Каждый приёмник задаёт окружность с радиусом, равным расчётному расстоянию до маяка, а точка пересечения этих окружностей соответствует позиции объекта. Однако из-за погрешностей RSSI окружности редко пересекаются в одной точке, поэтому используется метод наименьших квадратов (МНК), минимизирующий сумму квадратов отклонений между расчётными и измеренными расстояниями. Система уравнений для четырёх приёмников выглядит как:

$$\begin{cases} (x - x_1)^2 + (y - y_1)^2 = d_1^2 \\ (x - x_2)^2 + (y - y_2)^2 = d_2^2 \\ (x - x_3)^2 + (y - y_3)^2 = d_3^2 \end{cases} \quad (8)$$

где (xi,yi)(xi,yi) — координаты приёмников, di — расстояния до объекта. Линеаризация уравнений через вычитание пар уравнений позволяет свести задачу к решению матричного уравнения $A \cdot X = B$, где $X=[x,y]^T$, а матрицы A и B формируются из разности координат приёмников и расстояний. Решение через псевдообратную матрицу $X=(A^T*A)^{-1}*A^T*B$ даёт оценку координат объекта.

Для сглаживания шумов, вызванных нестабильностью RSSI и многолучевым распространением, применяется фильтр Калмана. Он работает в два этапа: предсказание, где текущее состояние (координаты и скорость) обновляется на основе модели движения (например, постоянной скорости), и коррекция, где предсказание корректируется с учётом новых измерений. Матрицы шумов процесса (Q) и измерений (R) настраиваются экспериментально: Q определяет "доверие" к модели движения, а R — к точности трилатерации. Например, если объект статичен, Q устанавливается малой, чтобы фильтр сильнее полагался на предыдущие состояния, а при движении — увеличивается для быстрой реакции на изменения. Пример вывода данных о местоположении объекта в пространстве представлен на рисунке 28.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

0:0.533,-5.684;1:-0.533,-6.245;
0:0.499,-5.644;1:-0.499,-6.245;
0:0.466,-5.604;1:-0.466,-6.245;
0:0.433,-5.564;1:-0.433,-6.245;
0:0.399,-5.524;1:-0.399,-6.245;
0:0.366,-5.484;1:-0.366,-6.245;
0:0.333,-5.444;1:-0.333,-6.245;
0:0.299,-5.404;1:-0.299,-6.245;
0:0.266,-5.364;1:-0.266,-6.245;
0:0.233,-5.324;1:-0.233,-6.245;
0:0.199,-5.284;1:-0.199,-6.245;
0:0.166,-5.244;1:-0.166,-6.245;
0:0.133,-5.204;1:-0.133,-6.245;
0:0.099,-5.164;1:-0.099,-6.245;
0:0.066,-5.124;1:-0.066,-6.245;
0:0.033,-5.084;1:-0.033,-6.245;
```

Рисунок 28 - Пример вывода данных

Итоговая точность системы в лабораторных условиях составляет 1–2 метра, что достаточно для образовательных задач: отслеживания перемещения студентов, мониторинга оборудования или интерактивных сценариев (например, автоматического включения приборов при приближении). Дальнейшее улучшение может быть достигнуто за счёт интеграции дополненной реальности (AR) для визуальной корректировки позиций или использования гибридных алгоритмов, сочетающих BLE с данными акселерометров.

Создание микросервиса на .NET Framework (C#) с интеграцией в MySQL включает следующие этапы:

1. Инициализация проекта и структуры. Проект создаётся как ASP.NET Web Application с шаблоном Web API, что генерирует базовую структуру с папками Controllers, Models, App_Start и конфигурацией маршрутизации. В папке Models определяются классы данных: ObjectModel и BeaconModel, аннотированные атрибутами для маппинга на таблицы MySQL. Атрибут [Table("objects")] указывает название таблицы, [Key] помечает первичный ключ, а [Required] обеспечивает валидацию обязательных полей.

Например, модель BeaconModel включает свойства Id, Name и Description, где Name обязательно для заполнения (Рисунок 29).

```
namespace WebLab.Models
{
    [Table("objects")] //название таблицы в БД
    Ссылок: 2
    public class ObjectModel
    {
        Ссылок: 0
        public int Id { get; set; }
        Ссылок: 0
        public string Name { get; set; }
        Ссылок: 0
        public string Description { get; set; }
    }

    [Table("beacons")] // название таблицы в БД
    Ссылок: 2
    public class BeaconModel
    {
        Ссылок: 0
        public int Id { get; set; }
        Ссылок: 0
        public string Name { get; set; }
        Ссылок: 0
        public string Description { get; set; }
    }
}
```

Рисунок 29 - Пример класса Model

2. Настройка Entity Framework. Для работы с MySQL (Рисунок 30) через Entity Framework устанавливаются пакеты NuGet: EntityFramework и MySql.Data.Entity. Контекст данных AppDbContext (Рисунок 31) наследуется от DbContext и содержит DbSet<BeaconModel> и DbSet<ObjectModel>, которые представляют коллекции сущностей в базе. Конструктор контекста вызывает базовый конструктор с именем строки подключения DefaultConnection, что позволяет гибко управлять параметрами БД через конфигурационный файл.

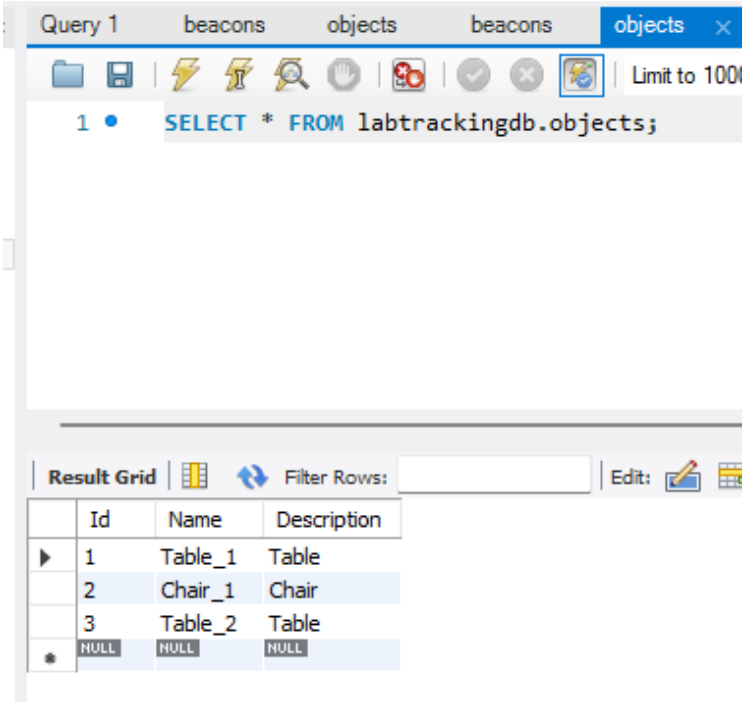

Рисунок 30 - Пример данных в MySQL

```
namespace WebLab
{
    Ссылок: 5
    public class AppDbContext : DbContext
    {
        Ссылок: 2
        public AppDbContext() : base("DefaultConnection") { }

        Ссылок: 2
        public DbSet<ObjectModel> Objects { get; set; }
        Ссылок: 2
        public DbSet<BeaconModel> Beacons { get; set; }
    }
}
```

Рисунок 31 - Пример класса AppDbContext

3. Реализация контроллеров. Контроллеры BeaconsController (Рисунок 32) и ObjectsController наследуются от ApiController и предоставляют методы для обработки HTTP-запросов. В BeaconsController:

— GetBeacons() обрабатывает GET-запросы, возвращая список всех маяков через db.Beacons.ToList().

— CreateBeacon() принимает POST-запросы, проверяет валидность модели через ModelState.IsValid, добавляет данные в БД через db.Beacons.Add(beacon) и сохраняет изменения через db.SaveChanges().

Аналогичная логика реализована в ObjectsController для работы с объектами. Для обработки ошибок методы обёрнуты в try-catch, возвращая статусы Ok(), BadRequest() или InternalServerError().

```
namespace WebLab.Controllers
{
    Ссылок: 0
    public class BeaconsController : ApiController
    {
        private AppDbContext db = new AppDbContext();

        [HttpGet]
        Ссылок: 0
        public IHttpActionResult GetBeacons()
        {
            return Ok(db.Beacons.ToList());
        }

        [HttpPost]
        Ссылок: 0
        public IHttpActionResult CreateBeacon(BeaconModel beacon)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            db.Beacons.Add(beacon);
            db.SaveChanges();

            return Ok(beacon);
        }
    }
}
```

Рисунок 32 - Пример класса BeaconsController

ASP.NET Web API использует конвенцию маршрутов вида api/{controller}/{id}, где:

— GET api/beacons вызывает GetBeacons().

— POST api/beacons вызывает CreateBeacon().

Атрибуты [HttpGet] и [HttpPost] явно указывают, какие методы обрабатывают определённые типы запросов. Пример запроса в Postman представлен на рисунке 33.

LabTest / objects / **objects**

GET https://localhost:44375/api/objects

Params Authorization Headers (7) Body Scripts Tests Settings

Query Params

	Key	Value
	Key	Value

Body Cookies Headers (10) Test Results

{} JSON Preview Visualize


```
1  [
2      {
3          "Id": 1,
4          "Name": "Table_1",
5          "Description": "Table"
6      },
7      {
8          "Id": 2,
9          "Name": "Chair_1",
10         "Description": "Chair"
11     },
12     {
13         "Id": 3,
14         "Name": "Table_2",
15         "Description": "Table"
16     }
17 ]
```


Рисунок 33 - Пример ответа на GET запрос

Микросервис предоставляет REST API для управления объектами и маяками, интегрируясь с MySQL через Entity Framework. Основные этапы разработки включают создание моделей, настройку контекста, реализацию контроллеров и валидацию данных.

4.3 Реализация интерфейсов взаимодействия цифрового двойника с киберфизической средой

Клиентское приложение (Рисунок 34), разработанное на платформе Unity, предоставляет визуализацию цифрового двойника учебной аудитории. Оно интегрируется с WebGL для обеспечения доступности через веб-браузеры. Основные функции приложения:

— визуализация данных: отображение информации о состоянии аудитории в реальном времени (температура, присутствие студентов и т.д.);

— интерактивность: пользователи могут взаимодействовать с моделью, получая доступ к различным функциям управления (например, регулировка освещения).

симуляция сценариев: приложение может проводить симуляцию различных сценариев процессов обучения, что помогает в принятии управленческих решений.

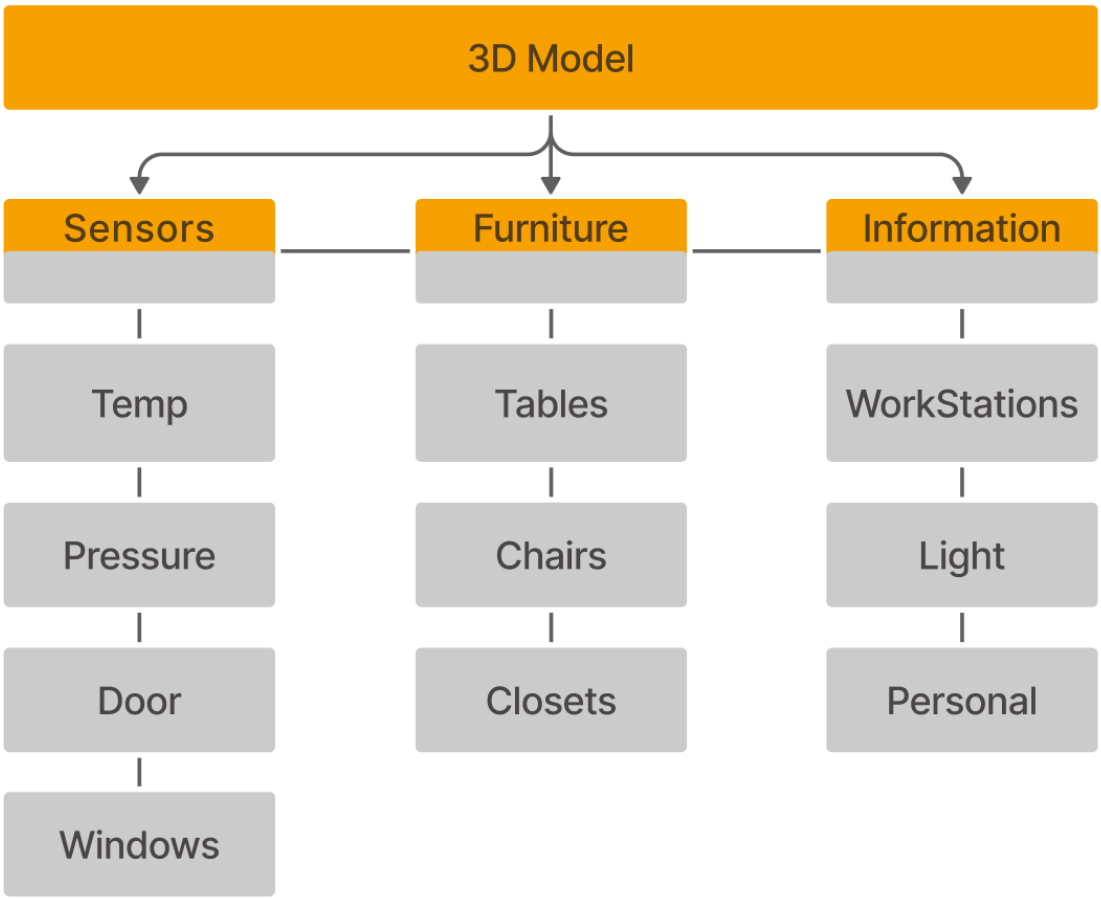

Рисунок 34 – Структура клиентского программного обеспечения

Цифровая копия учебной лаборатории, разработанная в Unity (Рисунок 35), включает в себя: интерьер помещения, компьютеры и их периферийные устройства, систему освещения, состоящую из трёх зон. Целью создания цифрового двойника является не только визуализация лаборатории, но и реализация системы мониторинга и управления объектами в лаборатории.

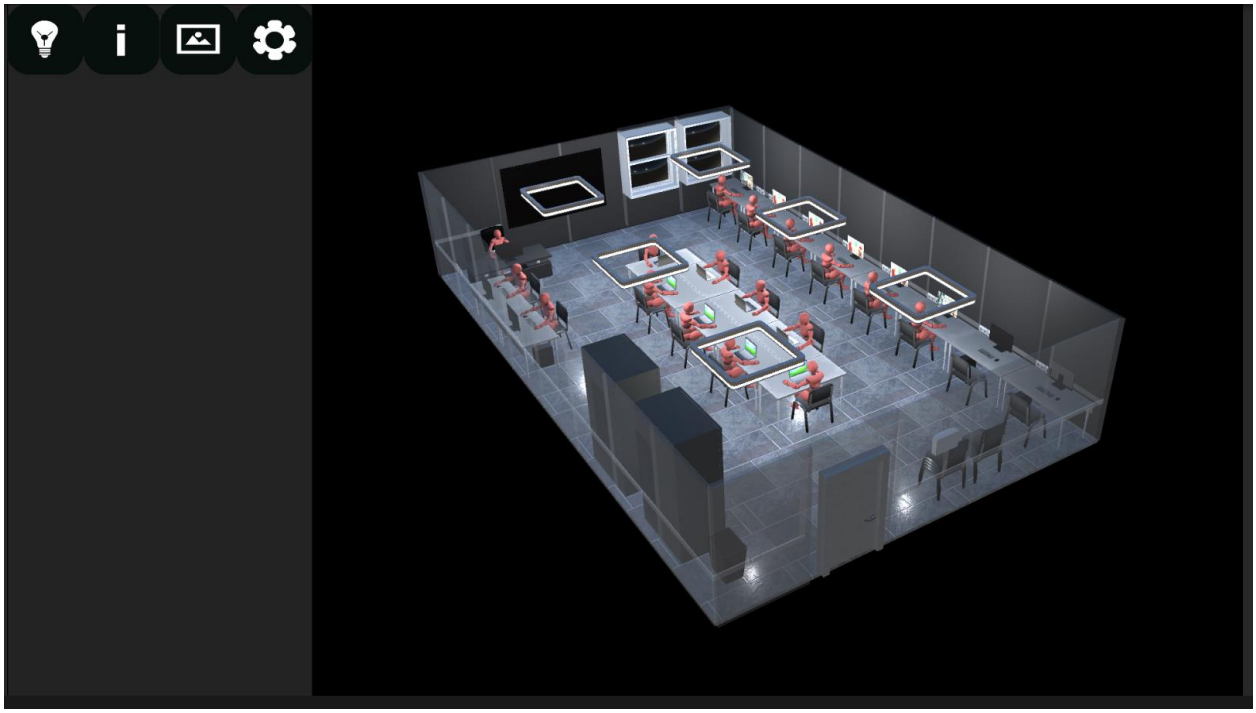

Рисунок 35 – Пример интерфейса виртуальной модели цифрового двойника учебной лаборатории

Цифровой двойник лаборатории должен управлять системой освещения. Лаборатория разделена на три зоны освещения: начало, середина и конец помещения. Виртуальная модель Unity позволяет контролировать состояние света в каждой зоне (Рисунок 36). Датчики света передают информацию о текущем состоянии в цифровую модель. В зависимости от времени суток или активности студентов, система автоматически может регулировать яркость освещения. В интерфейсе лаборатории администратор может вручную включать или выключать освещение в нужной зоне.

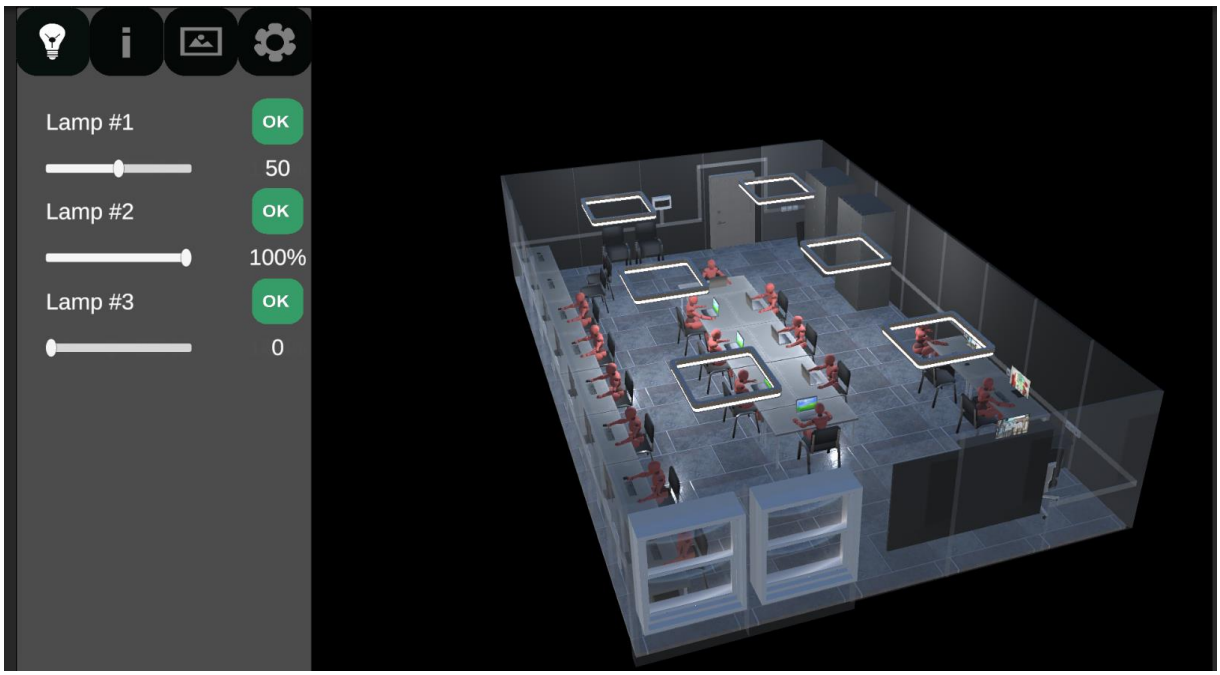

Рисунок 36 – Пример общего интерфейса управления освещением учебной лаборатории

Для системы освещения реализована плавная регулировка яркости: слайдеры в UI отправляют запросы на сервер, которые преобразуются в сигналы для диммеров, а затем визуализируются через изменение ламп (Рисунок 37).

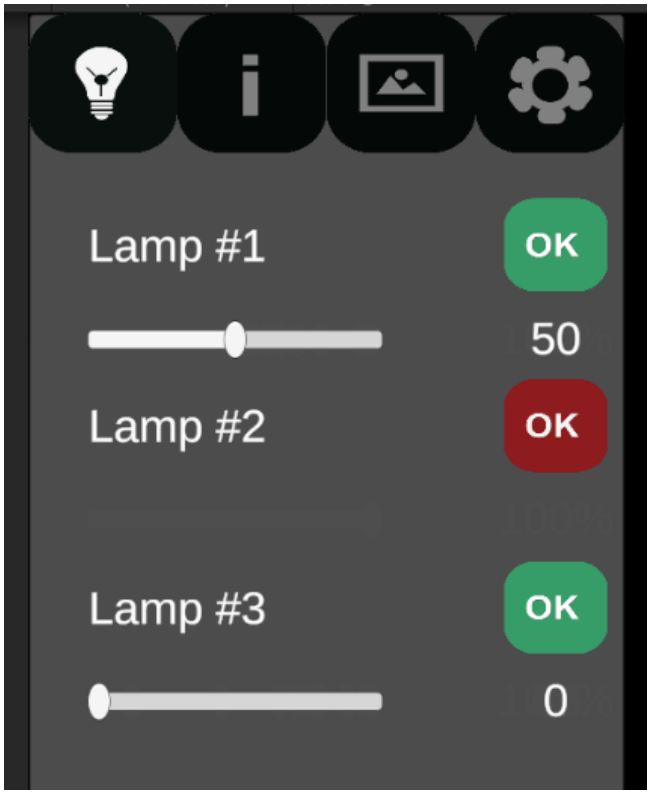

Рисунок 37 – Пример элементов интерфейса управления освещением

Визуализация состояний в Unity включает не только изменение графических элементов, но и обратную связь с пользователем. Например, при изменении состояния оборудования в помещении будет отображено соответсвующее уведомление. Также общее состояние учебной аудитории отображается в виде данных о присутствие персонала, состоянии открыт-закрыт, количества открытых окон и активных компьютерных устройств (Рисунок 38).

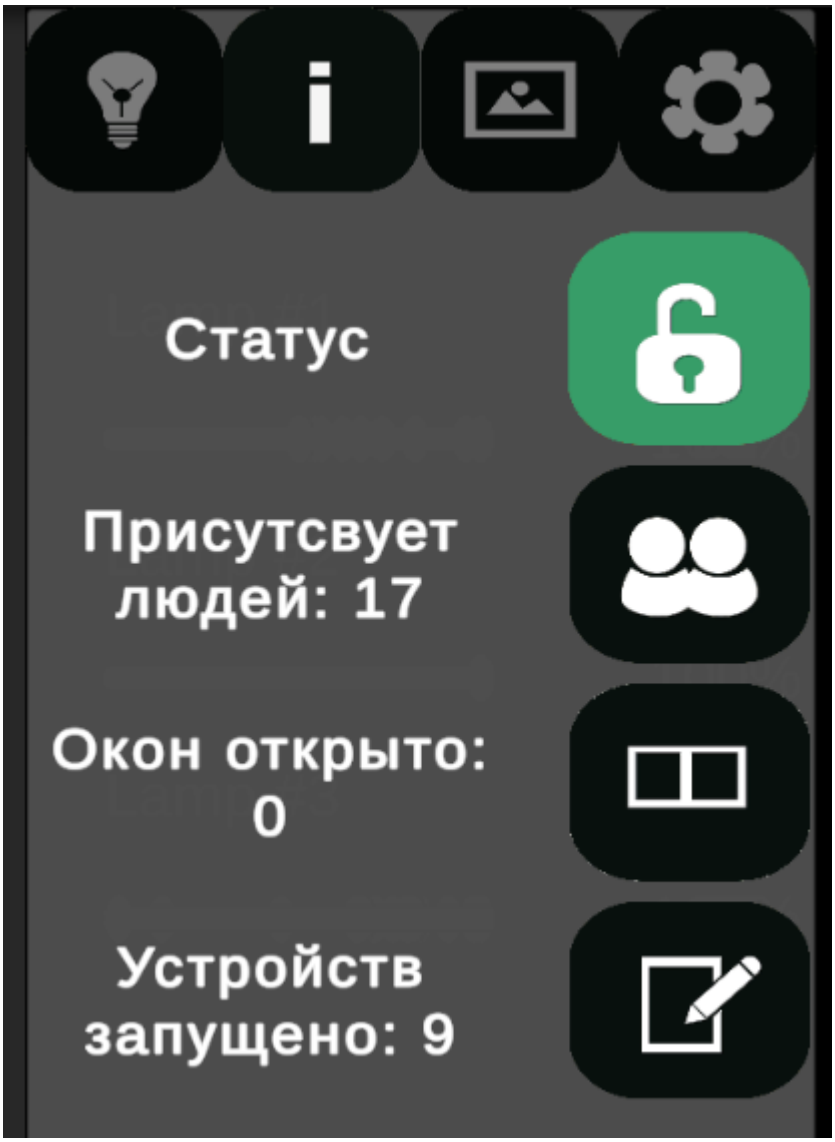

Рисунок 38 – Пример элементов интерфейса состояния учебной аудитории

Важной частью системы является возможность отслеживания наличия предметов внутри лаборатории. Для этого каждый предмет в цифровой модели связан с физическим объектом через уникальный идентификатор. В случае если предмет отсутствует в помещении (например, студент переместил стул или оборудование), система выдает уведомление об этом. Программное обеспечение Unity регулярно получает данные от датчиков и обновляет модель. Если объект не найден в течение определённого времени, отображается уведомление в интерфейсе. На рисунке 39 представлены элементы интерфейса для отображения данных о типе предмета интерьера.

Рисунок 39 – Пример элементов интерфейса информации о предметах интерьера

На рисунках 40 - 41 представлен пример отображения информации о выбранном типе предмета интерьера, для удобства отслеживания над каждым предметом выбранного типа появляется стрелочка.

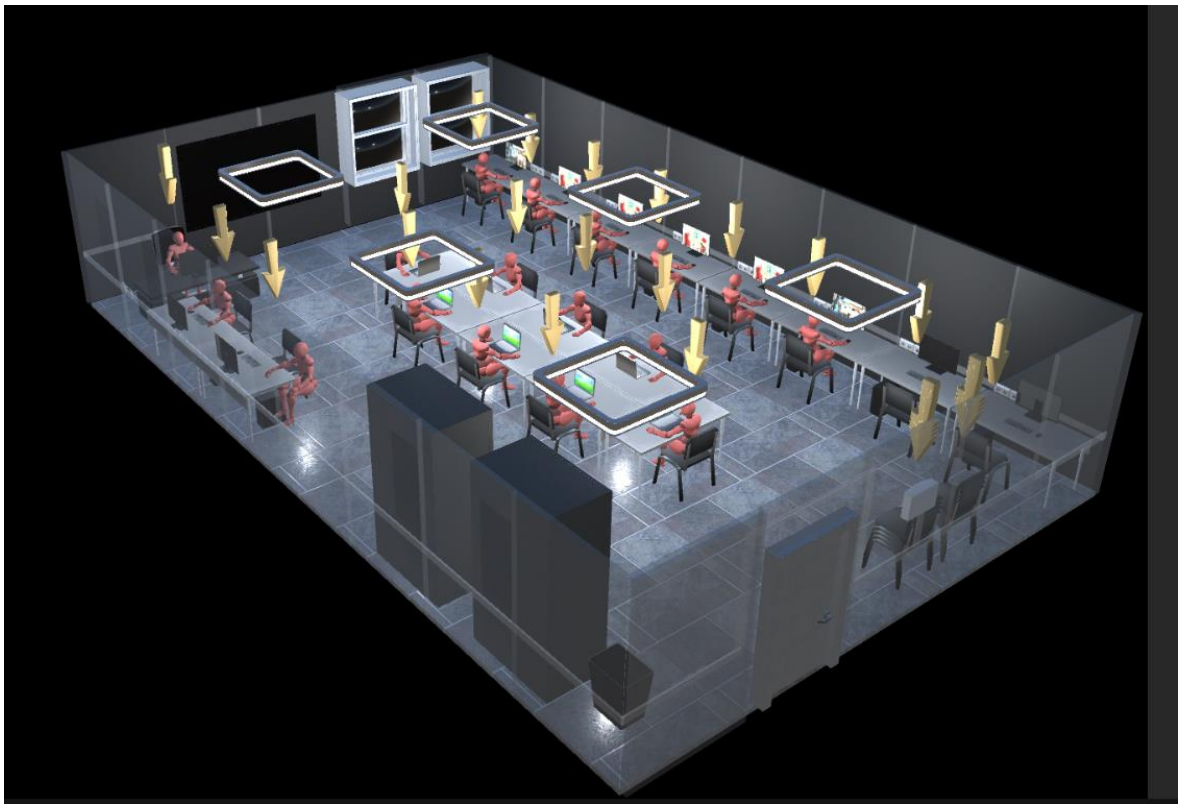

Рисунок 40 – Пример отслеживания выбранного типа интерьера в помещении

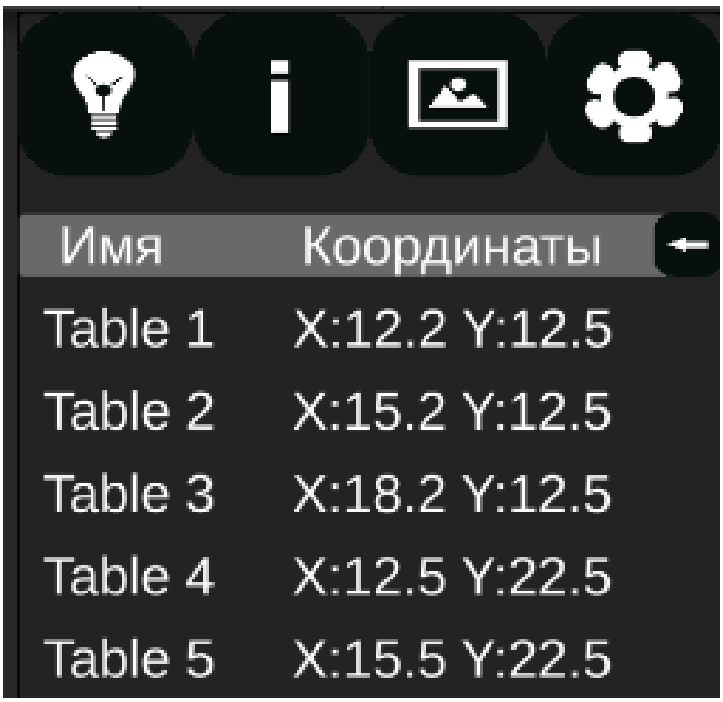

Рисунок 41 – Пример вывода данных о выбранном типе предмета интерьера

Важным элементом взаимодействия пользователя с моделью цифрового двойника выступает такой элемент как динамическая камера. На рисунке 42 представлен пример изменения положения камеры внутри пространства модели цифрового двойника учебной аудитории.

Рисунок 42 – Пример 3D модели учебной лаборатории в среде Unity

Для контроля работы рабочего места студента может использоваться интеграция с компьютерами через API. Система Unity проверяет состояние компьютеров и периферийных устройств (например, включение/выключение экрана или клавиатуры). В случае отключения компьютера на экране администратора лаборатории отображается информация о статусе рабочего места.

Интеграция всех компонентов обеспечивает непрерывный цикл взаимодействия: пользователь управляет устройствами через интерфейс Unity, команды передаются в реальную среду, обратная связь через датчики обновляет цифровой двойник, создавая замкнутую киберфизическую систему, пригодную для обучения, мониторинга и удалённого управления.

Система управления заказами на предприятии по производству корпусной мебели

ЗАКЛЮЧЕНИЕ

В процессе выполнения научно-исследовательской работы была разработана технология создания цифровых двойников учебных аудиторий, интегрируемых в единую киберфизическую среду цифрового университета.

Научная новизна заключается в использовании при создании цифровых двойников учебных аудиторий многоаспектных параметрических моделей, обеспечивающих представление с единых методологических позиций информации о состоянии моделируемых объектов, что обеспечивает возможность расширения контролируемых параметров с минимальными затратами на модификацию цифровых двойников.

Научная значимость состоит в развитии методов многоаспектного моделирования путем ее адаптации к технологии создания цифровых двойников учебных аудиторий, интегрируемых в единую корпоративную киберфизическую среду.

Практическая применимость определяется возможностью использования разработанной технологии создания цифровых двойников учебных аудиторий при формирования единой киберфизической среды цифрового университета.

Разработанная платформа для учебных аудиторий легко адаптируется к любым типам помещений (офисы, цеха, больницы) и интегрируется с BIM-моделями и цифровыми двойниками зданий. Датчики и алгоритмы, отслеживающие климат, энергопотребление и загрузку, могут быть перенастроены для мониторинга промышленных объектов, оптимизации логистики в торговых центрах или контроля условий хранения в лабораториях. В рамках BIM технология позволяет визуализировать данные сенсоров в 3D-моделях, автоматически обновлять цифровые двойники при изменении планировки и прогнозировать эксплуатационные риски .Ключевое преимущество — единая среда для IoT-сенсоров, аналитики и управления, которая служит основой для «умных» зданий, где решения принимаются на стыке BIM, AI и данных реального времени.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Громаков. Е.И., Сидорова А.А., Бабчинецкий С.Г. СОВРЕМЕННЫЕ ТЕХНОЛОГИИ. КИБЕРФИЗИЧЕСКИЕ СИСТЕМЫ. Томский политехнический университет. – Томск: Изд-во Томского политехнического университета, 2021. – 166 с.

2. Акимов С. В., Верхова Г. В., Меткин Н. П. Теоретические основы CALS : монография. СПб. : СПбГУТ. 2018. - 187 с.

3. Киберфизические системы. Cyber-Physical System, CPS. К чему приведет слияние интернета людей, вещей и сервисов [сайт] – Текст: электронный – URL: https://www.tadviser.ru/index.php/Статья: Киберфизические_системы_(CyberPhysicalSystem, _CPS) (Дата обращения 12.04.25).

4. ГОСТ Р 55768-2013. Информационная технология. МОДЕЛЬ ОТКРЫТОЙ ГРИД-СИСТЕМЫ. Основные положения.: Изд-во стандартов, 2015., 32 с.

5. ГОСТ Р 71576-2024. СИСТЕМЫ КИБЕРФИЗИЧЕСКИЕ. Основные положения.: Изд-во стандартов, 2024., 5 с.

6. Зырянов. Д.А., Бадика М.А., Бабчинецкий С.Г. Цифровые двойники в управление жизненным циклом изделий и технологий [сайт] – Текст: электронный – URL: https://www.researchgate.net/publication/364250604_DIGITAL_TWINS_IN_PRODUCT_AND_TECHNOLOGY_LIFECYCLE_MANAGEMENT) (Дата обращения 15.04.25).

7. Что такое цифровые двойники и где их используют [сайт] – Текст: электронный – URL: https://trends.rbc.ru/trends/industry/6107e5339a79478125166eeb (Дата обращения 15.04.25).

8. Суримова В.А., Скородумова Е.А. Создание и интеграция цифрового двойника [сайт] – Текст: электронный – URL: https://cyberleninka.ru/article/n/sozdanie-i-integratsiya-tsifrovogo-dvoynika/ viewer (Дата обращения 16.04.25).

9. Гапанович Д.А., Тарасова В.А., Сухомлин В.А., Куприяновский В.П. Анализ подходов архитектурного проектирования цифровых двойников

[сайт] – Текст: электронный – URL: https://cyberleninka.ru/article/n/analiz-podhodov-arhitekturnogo-proektirovaniya-tsifrovyh-dvoynikov/viewer (Дата обращения 17.04.25).

10. Лихтциндер Б.Я., Ольберг П.А. Моделирование и цифровые двойники [сайт] – Текст: электронный – URL: https://cyberleninka.ru/article/n/ modelirovanie-i-tsifrovye-dvoyniki/viewer (Дата обращения 18.04.25).

11. Ansys twin builder / Официальный сайт компании Ansys engineering modelin [сайт] – Текст: электронный – URL: https://www.ansys.com/content/ dam/ product/digital-twin/twin-builder/ansys-twin-builder-technical-datasheet.pdf (дата обращения: 20.04.25).

12. Официальный сайт производителя программного обеспечения «COMSOL» [сайт] – Текст: электронный – URL: https://www.comsol.ru/ comsol-multiphysics (дата обращения: 20.04.25).

13. Официальный сайт компании «Cision» [сайт] – Текст: электронный – URL: https://www.prnewswire.com/news-releases/altair-presents-open-flexible-andscalable-total-digital-twin-solution-301639251.html (дата обращения: 20.04.25).

14. What is Azure Digital Twins? / Microsoft Learn official site [сайт] – Текст: электронный – URL: https://learn.microsoft.com/en-us/azure/digital-twins/overview (дата обращения: 20.04.25).

15. Цифровые двойники PlantSight — ключ к контролю над данными промышленных объектов [сайт] – Текст: электронный – URL: https://sapr.ru/article/26264 (дата обращения: 20.04.25).

16. Цифровые двойники в промышленности: истоки, концепции, современный уровень развития и примеры внедрения [сайт] – Текст: электронный – URL: https://digitaltwin.ru/articles/digital-twins-in-industry/ (дата обращения: 22.04.25).

17. Будущее цифровых двойников в строительстве [сайт] – Текст: электронный – URL: https://digital-build.ru/budushhee-czifrovyh-dvojnikov-v-stroitelstve/ (дата обращения: 24.04.25).

18. Создание IoT-приложений с помощью tinyML и машинного обучения [сайт] – Текст: электронный – URL: https://controleng.ru/innovatsii/cifrovye-dvojniki/tinyml/ (дата обращения: 24.04.25).

19. Low-code платформа: кому нужна, в чем выгода и почему это тренд для компаний уровня Enterprise [сайт] – Текст: электронный – URL: https://habr.com/ru/companies/simpleone/articles/856860/ (дата обращения: 25.04.25).

20. Архитектура промышленных приложений IoT и протоколы AMQP, MQTT, JMS, REST, CoAP, XMPP [сайт] – Текст: электронный – URL: https://dzen.ru/a/X7JxwTktTBmepoyJ (дата обращения: 25.04.25).

21. Что такое CI/CD? Разбираемся с непрерывной интеграцией и непрерывной поставкой [сайт] – Текст: электронный – URL: https://habr.com/ru/companies/otus/articles/515078/ (дата обращения: 27.04.25).

22. Введение в архитектуру MLOps [сайт] – Текст: электронный – URL: https://habr.com/ru/companies/otus/articles/773320/ (дата обращения: 27.04.25).

23. Digital Twin — цифровая копия физической системы [сайт] – Текст: электронный – URL: https://habr.com/ru/articles/887936/ (дата обращения: 27.04.25).

24. Цифровой двойник: концепция, уровни, связь с Интернетом вещей и роль численного и системного моделирования [сайт] – Текст: электронный – URL: https://sapr.ru/article/26079 (дата обращения: 30.04.25).

25. Технология цифрового двойника в системном проектировании [сайт] – Текст: электронный – URL: https://www.geeksforgeeks.org/digital-twin-technology-in-system-design/ (дата обращения: 30.04.25).

ПРИЛОЖЕНИЕ А

Листинг кода приёмника данных от beacon для ESP32.

```
#include <BLEDevice.h>
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h> // Добавляем необходимый заголовочный файл

const std::string TARGET_BEACON_MAC_1 = "d0:b1:f7:50:e6:85";
const std::string TARGET_BEACON_MAC_2 = "c4:a4:d7:56:94:88";

#define RECEIVER_ID 1
const int WIFI_CHANNEL = 6;
uint8_t masterMac[] = {0xF4, 0x65, 0x0B, 0x55, 0x54, 0x88};

#pragma pack(push, 1)
struct SensorData {
    uint8_t receiver_id;
    int16_t rssi;
    char beacon_mac[18];
};
#pragma pack(pop)

class FilteredScanner : public BLEAdvertisedDeviceCallbacks {
    void onResult(BLEAdvertisedDevice device) {
        if           (device.getAddress().toString()           ==
TARGET_BEACON_MAC_1) {

                  device.getAddress().toString().c_str(),
                  sizeof(data.beacon_mac));

            esp_err_t result = esp_now_send(masterMac,
                                            (uint8_t*)&data,
                                            sizeof(data));

            Serial.printf("[%d]  MAC:  %s,  RSSI:  %d,  Status:
%s\n",
                        (result    ==    ESP_OK)    ?    "OK"    :
esp_err_to_name(result));
        }
    }
};

void setup() {
    Serial.begin(115200);

    // 1. Инициализация Wi-Fi
    WiFi.mode(WIFI_STA);
    WiFi.begin("Pentagon", "pinokkio", WIFI_CHANNEL);

    // Ожидание подключения к Wi-Fi
    if (WiFi.waitForConnectResult() != WL_CONNECTED) {
```

```
        Serial.println("Wi-Fi connection failed!");
        return;
    }

    // 2. Настройка параметров Wi-Fi
    // Замена устаревших функций
    wifi_country_t country = {
        .cc = "CN",
        .schan = 1,
        .nchan = 13,
        .max_tx_power = 84, // 20dBm
    };
    esp_wifi_set_country(&country);

    // 3. Инициализация ESP-NOW
    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW init failed");
        return;
    }

    // 4. Конфигурация пира с правильным типом интерфейса
    esp_now_peer_info_t peerInfo;
    memset(&peerInfo, 0, sizeof(peerInfo));
    memcpy(peerInfo.peer_addr, masterMac, 6);
    peerInfo.ifidx = WIFI_IF_STA; // Используем правильный тип
    peerInfo.encrypt = false;

    esp_err_t addStatus = esp_now_add_peer(&peerInfo);
    if (addStatus != ESP_OK) {
        Serial.printf("Failed     to     add     peer:     %s\n",
esp_err_to_name(addStatus));
        return;
    }

    // 5. Настройка BLE
    BLEDevice::init("");
    BLEScan* pScan = BLEDevice::getScan();
    pScan->setAdvertisedDeviceCallbacks(new
    pScan->setWindow(99);
}


void loop() {
    BLEScanResults   results   =   BLEDevice::getScan()->start(1,
false);
    delay(50);
    BLEDevice::getScan()->clearResults();
}
```

ПРИЛОЖЕНИЕ Б

Листинг кода обработчика данных сигналов от приёмников для ESP32.

```
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>

#pragma pack(push, 1)
struct SensorData {
    uint8_t receiver_id;
    int16_t rssi;
    char beacon_mac[18];
};
#pragma pack(pop)

// Статистика по приёмникам
struct ReceiverStats {
    int last_rssi = 0;
    unsigned long last_update = 0;
    String beacon_mac = "";
};

ReceiverStats receivers[4]; // Для 4 приёмников
unsigned long last_print = 0;
const int PRINT_INTERVAL = 1000; // 5 раз в секунду

void  OnDataRecv(const  uint8_t  *mac,  const  uint8_t  *data,  int
len) {
    if(len == sizeof(SensorData)) {
        SensorData packet;
        memcpy(&packet, data, sizeof(packet));

        if(packet.receiver_id >= 1 && packet.receiver_id <= 4) {
            int idx = packet.receiver_id - 1;
            receivers[idx].last_rssi = packet.rssi;
            receivers[idx].last_update = millis();
            receivers[idx].beacon_mac                          =
String(packet.beacon_mac);
        }
    }
}

void printStats() {
    Serial.println("\n=== Данные приёмников ===");
    Serial.println("ID  |  MAC  Beacon              |  RSSI    |
Актуальность");
    Serial.println("-------------------------------------------
");

    for(int i = 0; i < 4; i++) {
        String status = "OK";
        if(millis() - receivers[i].last_update > 1000) {
```

```
            status = "STALE";
        } else if(receivers[i].beacon_mac.length() == 0) {
            status = "OFFLINE";
        }

        Serial.printf("%2d | %17s | %5d | %s\n",
                      i+1,
                      receivers[i].beacon_mac.c_str(),
                      receivers[i].last_rssi,
                      status.c_str());
    }
}

void setup() {
    Serial.begin(115200);

    // 1. Инициализация Wi-Fi
    WiFi.mode(WIFI_STA);
    WiFi.begin("Pentagon","pinokkio",6);
    Serial.print("MAC обработчика: ");
    Serial.println(WiFi.macAddress());

    // 2. Настройка параметров Wi-Fi
    wifi_country_t country = {
        .cc = "CN",
        .schan = 1,
        .nchan = 13,
        .max_tx_power = 84,
        .policy = WIFI_COUNTRY_POLICY_AUTO
    };
    esp_wifi_set_country(&country);

    // 3. Инициализация ESP-NOW
    if(esp_now_init() != ESP_OK) {
        Serial.println("Ошибка инициализации ESP-NOW");
        return;
    }

    // 4. Регистрация callback
    esp_now_register_recv_cb(OnDataRecv);

    Serial.println("Обработчик готов к приёму данных");
}

void loop() {
    if(millis() - last_print >= PRINT_INTERVAL) {
        printStats();
        last_print = millis();
    }
}
```

ПРИЛОЖЕНИЕ В

Листинг кода симуляции получения данных для ESP32.

```
#include <Arduino.h>

float lerp(float a, float b, float t);

struct MovingObject {
  int id;
  float startX, startY;
  float phase1X, phase1Y;
  float phase2X, phase2Y;
};

// Настройки объектов
MovingObject objects[2] = {
  {0, -1.0f, -6.245f, 1.0f, -6.245f, -1.0f, -3.844f}, // Объект
0
  {1, -1.0f, -3.844f, -1.0f, -6.245f, 1.0f, -6.245f}  // Объект
1
};

const unsigned long FULL_CYCLE_TIME = 9000; // Полный цикл 9
секунд
const unsigned long PHASE_DURATION = 3000;   // Длительность
каждой фазы 3 секунды

void setup() {
  Serial.begin(115200);
}

void loop() {
  static unsigned long startTime = millis();
  unsigned long currentTime = millis() - startTime;
  currentTime %= FULL_CYCLE_TIME; // Зацикливаем время

  // Определяем текущую фазу
  int phase = currentTime / PHASE_DURATION;
  float t = (currentTime % PHASE_DURATION) /
(float)PHASE_DURATION;

  String packet = "";

  for(int i = 0; i < 2; i++)
   {
    MovingObject& obj = objects[i];
    float x, y;

    switch(phase) {
      case 0: // Фаза 1: движение по первой прямой
        x = lerp(obj.startX, obj.phase1X, t);
        y = lerp(obj.startY, obj.phase1Y, t);
```

```
        break;

      case 1: // Фаза 2: движение по второй прямой
        x = lerp(obj.phase1X, obj.phase2X, t);
        y = lerp(obj.phase1Y, obj.phase2Y, t);
        break;

      case 2: // Фаза 3: возврат в начальную позицию
        x = lerp(obj.phase2X, obj.startX, t);
        y = lerp(obj.phase2Y, obj.startY, t);
        break;
    }

    packet += String(obj.id) + ":" + String(x, 3) + "," +
String(y, 3) + ";";
  }

  Serial.println(packet);
  delay(50); // Задержка для стабильности
}

// Функция линейной интерполяции
float lerp(float a, float b, float t) {
  return a + (b - a) * t;
}
```